BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Công Nghệ Kỹ Thuật Điện, Điện Tử

ĐỀ TÀI

PRIM XÂY CÂY BAO TRÙM NHỎ NHẤT TỪ MỘT ĐỈNH

Mã đề DT080

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Phan Văn Tịnh
Mã số sinh viên	N24DCDT095
Lớp	D24CQDT01-N

THÀNH PHỐ HỒ CHÍ MINH THÁNG 10 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Công Nghệ Kỹ Thuật Điện, Điện Tử

ĐỀ TÀI
PRIM XÂY CÂY BAO TRÙM NHỎ NHẤT TỪ MỘT ĐỈNH

Mã đề DT080

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Phan Văn Tịnh
Mã số sinh viên	N24DCDT095
Lớp	D24CQDT01-N
Học phần và nhóm	INT13185 Nhóm 03
Phiên bản báo cáo	Ngày 25/10/2026. Commit mã được kiểm C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc

THÀNH PHỐ HỒ CHÍ MINH THÁNG 10 NĂM 2026

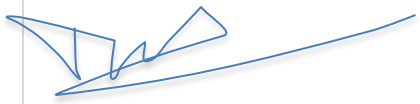
PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Sinh viên điền theo bảng phân công
Họ tên MSSV lớp	Phan Văn Tịnh N24DCDT095 D24CQDT01-N
Mã đề và tên đề	DT080 Prim xây cây bao trùm nhỏ nhất từ một đỉnh
Căn cứ giáo trình	Chương 6, mục 6.5.1 Thuật toán Prim; Bài 6.2; trang in 150–153; trang PDF 162–165
Ngày giao ngày nộp	Ngày giao: ……/……/…… Ngày nộp: 15/11/2026

Phiếu giao đề tài DT080 của sinh viên Phan Văn Tịnh được trình bày nguyên văn ở hai trang tiếp theo.

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	Giảng viên hướng dẫn
 Phan Văn Tịnh	(Ký và ghi rõ họ tên) .....

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH

PHIẾU GIAO ĐỀ TÀI

DT080 INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Sinh viên Phan Văn Tịnh MSSV N24DCDT095

Lớp D24CQDT01-N

Prim xây cây bao trùm nhỏ nhất từ một đỉnh

Chương 6, 6.5.1 Thuật toán Prim; Bài 6.2; trang in 150–153; trang PDF 162–165

Đề bài

Viết Prim để nối toàn bộ đỉnh của đồ thị vô hướng liên thông bằng cây có tổng trọng số nhỏ nhất. Sinh viên mở rộng tập đỉnh đã chọn, lấy cạnh nhẹ nhất nối ra ngoài và loại cạnh có đích đã nằm trong cây. Kết quả phải gồm các cạnh, không chỉ một tổng chi phí.

Dữ liệu vào

Nhập hoặc khai báo đồ thị vô hướng đơn n=1..30, trọng số nguyên 0..1000, nguồn bắt đầu 1. Chọn cạnh theo (w, đỉnh_mới, cha) tăng dần. Nếu đồ thị rời rạc, báo không có cây bao trùm toàn bộ.

Kết quả ra

In n−1 cạnh được chọn và tổng trọng số; một đỉnh cho cây rỗng tổng 0. Nếu không phủ đủ n đỉnh, báo KHONG LIEN THONG, không gọi phần thu được là MST toàn bộ.

Ví dụ 1

Dữ liệu vào	Kết quả chính
`n=3; canh=[(1,2,4),(1,3,1),(2,3,2)]`	`chon=[(1,3,1),(3,2,2)]; tong=3`

Từ đỉnh 1 chọn cạnh tới 3 nhẹ nhất; tiếp theo chọn 3–2 thay vì cạnh trực tiếp 1–2.

Ví dụ 2

Dữ liệu vào	Kết quả chính
`n=4; canh=[(1,2,1),(3,4,2)]`	`KHONG LIEN THONG`

Bắt đầu từ 1 chỉ thu được hai đỉnh; không tồn tại cây nối đủ cả bốn đỉnh bằng các cạnh đã nhập.

DT080 HƯỚNG DẪN THỰC HIỆN

N24DCDT095 Phan Văn Tịnh

Thực hiện lần lượt năm bước

1. Đọc định nghĩa MST và vẽ tập đỉnh trong/ngoài cây ở ví dụ ba đỉnh.

2. Khởi tập đã chọn rỗng và hàng ưu tiên có (0,1,0) theo khuôn Prim trong sách. Chạy riêng hàm này trước khi ghép vào chương trình chính.

3. Mỗi lần lấy cạnh tốt nhất, bỏ qua đỉnh đã chọn; nếu mới thì thêm đỉnh và cạnh cha. In trạng thái sau mỗi bước trên ví dụ đã cho.

4. Đưa các cạnh nối tới đỉnh ngoài vào hàng; cuối cùng kiểm đủ đỉnh và đúng n−1 cạnh.

5. Chạy hai ví dụ cùng năm ca, cộng lại trọng số và kiểm không có chu trình trong tập cạnh. Nộp mã có chú thích các biến và bảng kết quả năm ca kiểm.

Các hàm nên viết

```
prim(ke,s): chọn cạnh và tính tổng
them_ung_vien!(hang,u,ke,da_chon): đưa cạnh
kiem_cay(n,canh_chon): kiểm kết quả
```

Năm ca cần kiểm tra

1. n=1, không cạnh → tổng 0.

2. Hai đỉnh, một cạnh 7 → tổng 7.

3. Tam giác trọng số 4,1,2 → tổng 3.

4. Tam giác mọi cạnh 1 → hai cạnh, tổng 2.

5. Hai thành phần rời → báo không liên thông.

Phân tích và nhận xét

Đếm lần thêm/lấy cạnh và số đỉnh được nhận. Prim dùng heap O(m log n), lưu đồ thị/hàng O(n+m). Thử đồ thị liên thông n=5,10,20; khi hòa trọng số có thể có cây khác nhưng tổng nhỏ nhất bằng nhau.

Yêu cầu thêm sau khi phần cơ bản chạy đúng

Tùy chọn: cho đổi nguồn bắt đầu và kiểm tổng nhỏ nhất không đổi, dù thứ tự cạnh có thể đổi.

Sản phẩm: mã nguồn, các kết quả chạy ví dụ và ca kiểm tra, một lần vết bằng tay, báo cáo theo mẫu môn học. Chạy từ bàn phím hoặc truyền trực tiếp các giá trị vào hàm.

Ngày giao ./...../..... Ngày nộp ...../...../.....

Sinh viên thực hiện	**Giảng viên hướng dẫn**
	(Ký và ghi rõ họ tên)
	
Phan Văn Tịnh	Hồ Nhựt Minh

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

LỜI CAM ĐOAN

Em tên là Phan Văn Tịnh, mã số sinh viên N24DCDT095, lớp D24CQDT01-N. Em xin cam đoan các kết quả chạy chương trình, kiểm thử và hình ảnh minh chứng trong báo cáo đề tài DT080 – "Prim xây cây bao trùm nhỏ nhất từ một đỉnh" phản ánh quá trình thực hiện và chạy thử chương trình trên máy tính của em.

Trong quá trình thực hiện, em sử dụng phiếu giao đề tài, khung báo cáo và tài liệu học phần do giảng viên cung cấp. Khung trình bày được chỉnh sửa từ mẫu báo cáo của giảng viên; nội dung, dữ liệu và kết quả được bổ sung theo đề tài DT080. Các kiến thức và nội dung tham khảo được ghi nguồn trong báo cáo.

Em có sử dụng ChatGPT để hỗ trợ giải thích thuật toán, xây dựng và chỉnh sửa mã nguồn Python, xây dựng ca kiểm thử, hướng dẫn thao tác Git/GitHub và hỗ trợ biên soạn báo cáo. Em trực tiếp chạy chương trình, thực hiện kiểm thử, đối chiếu kết quả và lưu các ảnh minh chứng trên máy tính của mình.

Em không sử dụng ảnh kết quả của bài mẫu hoặc của người khác làm minh chứng cá nhân, không khai báo số liệu thực nghiệm khi chưa chạy chương trình. Em chịu trách nhiệm về nội dung báo cáo, mã nguồn, tính trung thực của các kết quả và việc cung cấp mã nguồn để giảng viên kiểm tra, chạy lại.

TP. Hồ Chí Minh ngày 25 tháng 10 năm 2026

Sinh viên thực hiện



Phan Văn Tịnh

LỜI CẢM ƠN

Em xin chân thành cảm ơn thầy Hồ Nhựt Minh đã cung cấp tài liệu, phiếu giao đề tài và hướng dẫn các yêu cầu thực hiện báo cáo. Những nội dung này giúp em xác định rõ mục tiêu, phạm vi bài toán và các bước cần thực hiện đối với đề tài DT080 – "Prim xây cây bao trùm nhỏ nhất từ một đỉnh".

Thông qua quá trình thực hiện đề tài, em có cơ hội củng cố kiến thức về đồ thị, danh sách kề, hàng đợi ưu tiên và thuật toán Prim. Đồng thời, em được thực hành xây dựng chương trình Python, kiểm thử, truy vết thuật toán và quản lý mã nguồn bằng Git/GitHub.

Do kiến thức và kinh nghiệm còn hạn chế, báo cáo có thể còn thiếu sót. Em kính mong nhận được những nhận xét và góp ý của thầy để tiếp tục hoàn thiện kiến thức và kỹ năng của mình.

Em xin chân thành cảm ơn!

TÓM TẮT

Đề tài DT080 – “Prim xây cây bao trùm nhỏ nhất từ một đỉnh” tập trung xây dựng chương trình Python tìm cây bao trùm nhỏ nhất của đồ thị đơn vô hướng có trọng số. Phạm vi dữ liệu gồm đồ thị có từ 1 đến 30 đỉnh, trọng số cạnh là số nguyên từ 0 đến 1000. Các ví dụ và ca khảo sát được thực hiện với đỉnh bắt đầu là đỉnh 1.

Chương trình biểu diễn đồ thị bằng danh sách kề và sử dụng hàng đợi ưu tiên dạng heap để lựa chọn cạnh mở rộng cây theo thuật toán Prim. Với đồ thị liên thông, chương trình trả về danh sách cạnh được chọn và tổng trọng số. Với đồ thị không liên thông, chương trình thông báo “KHONG LIEN THONG”. Các chức năng kiểm tra đầu vào, kiểm tra cấu trúc cây và truy vết được xây dựng để hỗ trợ phát hiện dữ liệu sai và đối chiếu quá trình thực hiện thuật toán.

Bộ kiểm thử gồm 12 ca độc lập với 33 phép kiểm tra, bao quát hai ví dụ trên phiếu giao đề tài, các trường hợp hòa trọng số, cạnh trọng số 0, dữ liệu biên và dữ liệu không hợp lệ. Kết quả thực hiện ghi nhận 12 ca PASS và 0 ca FAIL. Khảo sát số thao tác được tiến hành trên đồ thị đường đi và đồ thị đầy đủ với ba quy mô 5, 10 và 20 đỉnh. Cả 6 trường hợp khảo sát đều cho tổng trọng số cây bao trùm nhỏ nhất khớp với giá trị kỳ vọng của bộ dữ liệu đã thiết kế.

Mã nguồn được quản lý trên GitHub và đã được kiểm tra bằng cách clone sang thư mục mới, chuyển về commit C, sau đó chạy lại các ví dụ, bộ kiểm thử và chương trình khảo sát thành công. Kết quả báo cáo giới hạn trong miền dữ liệu của đề tài và các ca đã thực hiện; khảo sát dựa trên phép đếm thao tác ở quy mô nhỏ, chưa đánh giá hiệu năng trên đồ thị quy mô lớn.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG iv
CÁCH ĐỐI CHIẾU RUBRIC v
LỜI CAM ĐOAN vi
LỜI CẢM ƠN vii
TÓM TẮT viii
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 2
1.4. Cách thực hiện và cấu trúc báo cáo 5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 9
2.1. Khái niệm và mô hình dữ liệu 9
2.2. Cấu trúc dữ liệu sử dụng 10
2.3. Nguyên lý thuật toán 15
2.4. Cơ sở đánh giá thời gian và bộ nhớ 17
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 21
3.1. Phân tích yêu cầu chương trình 21
3.2. Thiết kế các hàm và dữ liệu 24
3.3. Thuật toán và mã nguồn chính 32
3.4. Lần vết ví dụ nhỏ 43
3.5. Giải thích tính đúng và điều kiện dừng 45
3.6. Phân tích độ phức tạp 50
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 55
4.1. Môi trường và kế hoạch kiểm thử 55
4.2. Bảng kết quả ít nhất 12 ca 58
4.3. Hai ví dụ trên phiếu 66
4.4. Hai ca sinh viên tự thiết kế 69
4.5. Ba ca biên phù hợp với đề tài 73
4.6. Kiểm tra input không hợp lệ — LOI01 77
4.7. Kết quả toàn bộ kiểm thử 79
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ 80
4.9. GitHub và chạy lại từ bản clone sạch 85
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 91
5.1. Mức hoàn thành so với phiếu 91

5.2. Giới hạn và bài học..... 92
5.3. Hướng phát triển phù hợp 94
TÀI LIỆU THAM KHẢO..... 97
PHỤ LỤC..... 1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy..... 1
PHỤ LỤC B. Danh mục bằng chứng..... 5
PHỤ LỤC C. Tự kiểm trước khi nộp..... 7
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM 8

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
MST	Cây bao trùm nhỏ nhất của đồ thị vô hướng có trọng số.
n	Số đỉnh của đồ thị, thuộc khoảng từ 1 đến 30.
m	Số cạnh của đồ thị.
s	Đỉnh bắt đầu của thuật toán; đề tài sử dụng s = 1.
Heap cực tiểu	Cấu trúc hàng đợi ưu tiên dùng để lấy ứng viên nhỏ nhất theo thứ tự (trọng số, đỉnh mới, đỉnh cha).
SHA commit	Mã băm dùng để xác định một phiên bản trong kho Git.
Commit C	Phiên bản mã nguồn dùng để chạy kiểm thử và đối chiếu ảnh minh chứng.
Commit S	Phiên bản chốt của lần nộp, được ghi bằng SHA đầy đủ trong tệp thông tin nộp.

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào..... 4
Bảng 1.2 Đặc tả kết quả đầu ra 5
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm..... 8
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì..... 12
Bảng 3.1 Tổ chức các hàm và tệp..... 26
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm..... 30
Bảng 3.3 Truy vết ca TC01 44
Bảng 3.4 Chi phí của từng pha trong bản cài..... 53
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu..... 59
Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập..... 60
Bảng 4.3 Actual kết luận và vị trí minh chứng..... 63
Bảng 4.4 Khảo sát Q1 Q2 Q3 82
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại..... 85
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu..... 95
Bảng PL.1 Bốn tệp cần nộp..... 8

DANH SÁCH HÌNH

Hình 4.1. Kết quả chạy ví dụ 1 — VD01 (TC01)..... 67
Hình 4.2. Kết quả chạy ví dụ 2 — VD02 (TC02)..... 68
Hình 4.3. Kết quả ca TK01 (TC06) — Đồ thị có cạnh trọng số 0 70
Hình 4.4. Kết quả ca TK02 (TC07) — Đồ thị liên thông bốn đỉnh..... 71
Hình 4.5. Kết quả ca BIEN01 (TC03) — Đồ thị một đỉnh, không có cạnh 74
Hình 4.6. Kết quả ca BIEN02 (TC04) — Đồ thị hai đỉnh, một cạnh 75
Hình 4.7. Kết quả ca BIEN03 (TC08) — Đồ thị 30 đỉnh, trọng số cạnh 1000..... 77
Hình 4.8. Kết quả ca LOI01 (TC09) — Từ chối trọng số âm 78
Hình 4.9. Kết quả bộ kiểm thử tự động — 12 ca độc lập, 33 phép kiểm..... 80
Hình 4.10a. Kiểm tra phiên bản, môi trường và chạy demo từ bản clone C 88
Hình 4.10b. Kết quả bộ kiểm thử từ bản clone C 88
Hình 4.10c. Clone kho mã và checkout commit C 89
Hình 4.11. Kho GitHub của đề tài DT080 và commit C 90

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Đề tài DT080 – "Prim xây cây bao trùm nhỏ nhất từ một đỉnh" yêu cầu xây dựng chương trình tìm cây bao trùm nhỏ nhất của đồ thị vô hướng có trọng số. Với đồ thị liên thông gồm n đỉnh, kết quả cần tìm là tập gồm n − 1 cạnh nối toàn bộ các đỉnh, không tạo chu trình và có tổng trọng số nhỏ nhất trong các cây bao trùm của đồ thị [1, mục 6.5, tr. 150].

Bài toán có thể mô hình hóa việc lựa chọn các tuyến kết nối giữa một nhóm thiết bị hoặc địa điểm với tổng chi phí nhỏ nhất. Trong mô hình này, mỗi đỉnh biểu diễn một thiết bị hoặc địa điểm, mỗi cạnh biểu diễn một kết nối có thể sử dụng và trọng số biểu diễn chi phí của kết nối đó.

Thuật toán Prim giải quyết bài toán bằng cách bắt đầu từ một đỉnh, sau đó mở rộng cây từng bước. Tại mỗi bước, thuật toán chọn cạnh có trọng số nhỏ nhất nối một đỉnh đã thuộc cây với một đỉnh chưa thuộc cây [1, mục 6.5.1, tr. 150]. Trong đề tài này, đỉnh bắt đầu là đỉnh 1; danh sách kề được dùng để biểu diễn đồ thị và hàng đợi ưu tiên được dùng để lựa chọn cạnh.

Việc thực hiện đề tài giúp củng cố kiến thức về đồ thị, cây bao trùm, thuật toán tham lam và hàng đợi ưu tiên. Đồng thời, quá trình cài đặt, truy vết và kiểm thử tạo cơ sở để đối chiếu giữa cách hoạt động của thuật toán với kết quả thực tế của chương trình.

1.2. Mục tiêu và sản phẩm

Mục tiêu của đề tài là cài đặt và kiểm chứng thuật toán Prim bằng Python theo đúng miền dữ liệu, quy tắc lựa chọn cạnh và yêu cầu đầu ra của phiếu DT080. Các mục tiêu cụ thể gồm:

1. Xây dựng chức năng tiếp nhận và kiểm tra dữ liệu của đồ thị đơn vô hướng có từ 1 đến 30 đỉnh, trọng số nguyên từ 0 đến 1000. Chương trình phải phát hiện dữ liệu sai như đỉnh ngoài phạm vi, cạnh tự nối, cạnh trùng hoặc trọng số không hợp lệ và đưa ra thông báo tương ứng.

2. Cài đặt thuật toán Prim bắt đầu từ đỉnh 1. Các ứng viên được xét theo thứ tự tăng dần của bộ (trọng số, đỉnh mới, đỉnh cha). Với đồ thị liên thông, chương trình phải trả về n − 1 cạnh được chọn theo thứ tự thực hiện và tổng trọng số. Trường hợp một đỉnh cho danh sách cạnh rỗng, tổng bằng 0; đồ thị không liên thông phải được thông báo rõ.

3. Kiểm nghiệm kết quả bằng cách kiểm tra cấu trúc cây, sự tồn tại của các cạnh trong đồ thị đầu vào và tổng trọng số. Xây dựng bộ kiểm thử gồm 12 ca độc lập, bao quát hai ví dụ trên phiếu, các ca bắt buộc, ca tự thiết kế, ca biên và dữ liệu sai. Kết quả thực tế phải được đối chiếu với kết quả kỳ vọng xác định trước.

4. Thực hiện lần vết bằng tay trên ví dụ thứ nhất và đối chiếu với chế độ truy vết của chương trình. Nội dung lần vết phải thể hiện phần tử được lấy khỏi hàng đợi, đỉnh được nhận, cạnh được chọn hoặc bị bỏ qua và tổng trọng số hiện tại.

5. Khảo sát số thao tác trên đồ thị đường đi và đồ thị đầy đủ với n = 5, 10, 20. Ghi nhận số lần thêm cạnh, lấy cạnh, nhận đỉnh và bỏ qua phần tử; kiểm tra tổng trọng số theo kỳ vọng của từng bộ dữ liệu, từ đó nhận xét chi phí thực hiện.

6. Tổ chức mã nguồn, hướng dẫn chạy và minh chứng để có thể kiểm tra lại. Mã nguồn được quản lý bằng Git/GitHub; phiên bản dùng trong báo cáo được xác định bằng mã commit và được kiểm chứng bằng việc clone sang thư mục mới rồi chạy lại.

Sản phẩm của đề tài gồm mã nguồn Python, chương trình chạy ví dụ và nhập dữ liệu, chức năng truy vết, bộ kiểm thử tự động, chương trình khảo sát, ảnh minh chứng, báo cáo và kho mã nguồn GitHub. Các tệp thực thi chính gồm demo.py, truy_vet.py, khao_sat.py và test/runtests.py; các hàm xử lý được tổ chức trong thư mục src. Tệp README.md trình bày cấu trúc dự án và các lệnh cần thiết để chạy lại chương trình.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Chương trình xử lý đồ thị đơn vô hướng có trọng số. Số đỉnh n là số nguyên thỏa mãn $1 \le n \le 30$; các đỉnh được đánh số từ 1 đến n. Mỗi cạnh được biểu diễn bằng bộ ba (u,

v, w), trong đó u và v là hai đỉnh đầu mút, w là trọng số nguyên thỏa mãn $0 \le w \le 1000$.

Đồ thị không có cạnh tự nối và không có cạnh trùng. Hai bộ ba có cùng cặp đỉnh đầu mút được xem là trùng cạnh, kể cả khi đảo thứ tự hai đỉnh hoặc có trọng số khác nhau. Vì đồ thị là vô hướng, cạnh (u, v, w) cho phép kết nối theo cả hai chiều.

Gọi m là số cạnh của đồ thị, ta có $0 \le m \le n(n-1)/2$. Dữ liệu không bắt buộc phải liên thông ngay từ đầu; chương trình tiếp nhận đồ thị hợp lệ và xác định khả năng xây cây bao trùm trong quá trình xử lý. Đỉnh bắt đầu trong phạm vi đề tài là $s = 1$ theo phiếu DT080.

Chương trình hỗ trợ hai cách cung cấp dữ liệu. Với cách khai báo trực tiếp, n và danh sách canh được gán trong mã hoặc truyền vào hàm; số cạnh được xác định bằng độ dài danh sách canh. Với cách nhập bàn phím, chạy lệnh python demo.py --nhap, sau đó nhập lần lượt n, m và m dòng cạnh. Mỗi dòng cạnh gồm ba số nguyên u v w, ngăn cách bằng khoảng trắng.

Khi truyền dữ liệu trực tiếp, danh sách canh và mỗi bộ ba cạnh có thể sử dụng kiểu list hoặc tuple. Các giá trị n, u, v, w phải là số nguyên; giá trị kiểu bool không được chấp nhận. Các dữ liệu đầu vào được đặc tả trong Bảng 1.1.

1.3.2. Kết quả và trường hợp đặc biệt

Với đồ thị liên thông, hàm prim trả về cặp kết quả (canh_chon, tong). Danh sách canh_chon gồm $n - 1$ cạnh theo thứ tự được thuật toán chọn; mỗi cạnh có dạng (cha, đỉnh mới, trọng số). Giá trị tong là tổng trọng số của các cạnh trong danh sách. Chương trình demo hiển thị hai kết quả này với nhãn chon và tong.

Các ứng viên trong hàng đợi ưu tiên được xét theo thứ tự tăng dần của bộ (w, đỉnh mới, cha). Thuật toán ưu tiên trọng số nhỏ hơn; nếu trọng số bằng nhau thì ưu tiên đỉnh mới có số hiệu nhỏ hơn; nếu tiếp tục bằng nhau thì ưu tiên đỉnh cha có số hiệu nhỏ hơn. Phần tử có đỉnh đích đã thuộc cây sẽ bị bỏ qua.

Với n = 1 và không có cạnh, kết quả là chon = [] và tong = 0. Với đồ thị không liên thông, hàm prim trả về None và chương trình demo in KHONG LIEN THONG. Các cạnh thu được trong thành phần chứa đỉnh 1 không được công nhận là cây bao trùm của toàn bộ đồ thị.

Dữ liệu vi phạm miền giá trị hoặc cấu trúc đầu vào được xử lý bằng ngoại lệ ValueError. Chương trình demo bắt ngoại lệ và in thông báo bắt đầu bằng DU LIEU KHONG HOP LE:, kèm nguyên nhân cụ thể. Trường hợp này được phân biệt với đồ thị hợp lệ nhưng không liên thông. Các kết quả đầu ra được tổng hợp trong Bảng 1.2.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	**Kiểu và kích thước**	**Miền và điều kiện**	**Cách nhập hoặc khai báo**
n	Số nguyên	$1 \le n \le 30$; các đỉnh mang số hiệu từ 1 đến n	Nhập đầu tiên từ bàn phím hoặc khai báo trực tiếp
m	Số nguyên	$0 \le m \le n(n-1)/2$	Nhập sau n; khi khai báo trực tiếp, m = len(canh)
canh	List hoặc tuple gồm m bộ ba (u, v, w)	u, v thuộc 1..n; $u \ne v$; w nguyên thuộc 0..1000; không trùng cặp đỉnh vô hướng	Nhập m dòng u v w hoặc khai báo danh sách cạnh
s	Số nguyên	s = 1 theo phiếu DT080	Gán bằng 1 khi gọi Prim trong chương trình demo

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
canh_chon, in với nhãn chon	Danh sách các bộ (cha, đỉnh mới, trọng số)	Gồm n − 1 cạnh thuộc đồ thị đầu vào, nối đủ n đỉnh, không tạo chu trình và đạt tổng trọng số nhỏ nhất	Hòa theo (w, đỉnh mới, cha); n = 1 cho danh sách rỗng
tong	Số nguyên, in với nhãn tong	Bằng tổng trọng số các cạnh trong canh_chon	n = 1 cho tổng 0; nhiều cây tối ưu có cùng tổng trọng số
Thông báo không liên thông	KHONG LIEN THONG	Dữ liệu hợp lệ nhưng không thể nối toàn bộ n đỉnh	Hàm prim trả về None; không xuất cây bao trùm toàn bộ
Thông báo dữ liệu sai	DU LIEU KHONG HOP LE: kèm nguyên nhân	Dữ liệu vi phạm kiểu, miền giá trị hoặc quy định về cạnh	Phát sinh ValueError; chương trình demo bắt lỗi và thông báo

1.4. Cách thực hiện và cấu trúc báo cáo

Đề tài được thực hiện theo trình tự: nghiên cứu tài liệu, phân tích bài toán, thực hiện bằng tay, cài đặt, kiểm thử và thu thập minh chứng. Trước hết, nội dung về cây bao trùm nhỏ nhất và thuật toán Prim trong giáo trình được sử dụng làm cơ sở lý thuyết [1, mục 6.5–6.5.1]. Các quy định về miền dữ liệu, đỉnh bắt đầu, thứ tự ưu tiên và kết quả đầu ra được xác định theo phiếu DT080.

Tiếp theo, ví dụ ba đỉnh trên phiếu được thực hiện bằng tay để xác định thứ tự chọn cạnh và tổng trọng số kỳ vọng. Chương trình Python được tổ chức thành các phần kiểm tra dữ liệu, tạo danh sách kề, thực hiện Prim, kiểm tra cấu trúc cây, truy vết và khảo sát số thao tác. Kết quả chạy được đối chiếu với kết quả kỳ vọng và các điều kiện của cây bao trùm.

Quá trình kiểm thử bao gồm hai ví dụ trên phiếu, các ca bắt buộc, ca tự thiết kế, ca biên và dữ liệu không hợp lệ. Khảo sát được thực hiện trên đồ thị đường đi và đồ thị đầy đủ với 5, 10 và 20 đỉnh. Mã nguồn được quản lý bằng Git/GitHub; ảnh minh chứng và kết quả chạy lại từ bản clone sạch được sử dụng để đối chiếu với phiên bản mã đã kiểm tra.

Báo cáo được tổ chức thành năm chương:

Chương 1 – Tổng quan đề tài: trình bày bài toán, mục tiêu, phạm vi, quy định đầu vào, đầu ra và phương pháp thực hiện.

Chương 2 – Cơ sở lý thuyết: trình bày các khái niệm về đồ thị, cây bao trùm nhỏ nhất, danh sách kề, hàng đợi ưu tiên và nguyên lý thuật toán Prim.

Chương 3 – Phân tích, thiết kế và cài đặt: trình bày tổ chức dữ liệu, chức năng các hàm, giả mã, mã nguồn chính, lần vết, lập luận tính đúng và phân tích độ phức tạp.

Chương 4 – Kết quả thử nghiệm và nhận xét: trình bày môi trường thực hiện, phương pháp xác định kết quả kỳ vọng, bộ kiểm thử, ảnh minh chứng, khảo sát số thao tác và việc chạy lại từ kho GitHub.

Chương 5 – Kết luận và hướng phát triển: đối chiếu kết quả với mục tiêu, nêu giới hạn, bài học và hướng phát triển phù hợp.

Mối liên hệ giữa các yêu cầu của đề tài, sản phẩm cần kiểm tra và vị trí trình bày trong báo cáo được tổng hợp tại Bảng 1.3.

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Tiếp nhận đồ thị đúng miền dữ liệu; bắt đầu từ đỉnh 1	Kiểm tra n, các cạnh và trọng số; từ chối cạnh tự nối, cạnh trùng và dữ liệu sai	1.3; 3.1; 4.6
Cài đặt Prim, mở rộng tập đỉnh đã chọn và bỏ qua đỉnh đã thuộc cây	Hàm prim và them_ung_vien; xét ứng viên theo thứ tự (w, đỉnh mới, cha)	3.2; 3.3
Trả về các cạnh được chọn và tổng trọng số	Với đồ thị liên thông, kết quả gồm n − 1 cạnh tạo thành cây bao trùm nhỏ nhất; tổng trả về bằng tổng trọng số các cạnh	3.3; 3.5; 4.2
Xử lý đồ thị một đỉnh và đồ thị không liên thông	Một đỉnh cho cây rỗng, tổng 0; đồ thị không liên thông in KHONG LIEN THONG	3.3.3; 4.3.2; 4.5.1
Lần vết ví dụ nhỏ và giải thích tính đúng	Bảng lần vết bằng tay đối chiếu với truy_vet.py; trình bày trạng thái, cạnh chọn, tổng trọng số và điều kiện dừng	3.4; 3.5
Chạy hai ví dụ, các ca bắt buộc và bổ sung đủ 12 ca độc lập theo hướng dẫn chung	Bộ kiểm thử có input, kỳ vọng, kết quả thực tế và PASS/FAIL; kiểm tra cấu trúc cây, cạnh đầu vào và tổng trọng số	4.1.2; 4.2–4.7
Phân tích chi phí và khảo sát với n = 5, 10, 20	Phân tích thời gian, bộ nhớ; bảng đếm thao tác trên đồ thị đường đi và đồ thị đầy đủ	3.6; 4.8

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Nộp mã nguồn, kết quả và báo cáo; bảo đảm khả năng chạy lại theo hướng dẫn chung	Mã nguồn, README, ảnh minh chứng, báo cáo; kho GitHub có commit xác định và bằng chứng clone chạy lại	4.9; Phụ lục A, B

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Bài toán DT080 được mô hình hóa bằng đồ thị vô hướng có trọng số G = (V, E). Trong đó, V là tập đỉnh, E là tập cạnh và mỗi cạnh có một trọng số biểu diễn chi phí kết nối giữa hai đỉnh. Các đỉnh được đánh số từ 1 đến n. Vì đồ thị vô hướng nên cạnh nối u với v cũng thể hiện kết nối từ v đến u.

Theo miền dữ liệu của đề tài, số đỉnh thỏa mãn $1 \leq n \leq 30$; trọng số các cạnh là số nguyên từ 0 đến 1000. Đồ thị không có cạnh tự nối và không có cạnh trùng. Hai cạnh (u, v) và (v, u) được xem là cùng một cạnh. Với m là số cạnh, ta có $0 \leq m \leq n(n-1)/2$.

Một đồ thị được gọi là liên thông nếu giữa mọi cặp đỉnh đều tồn tại đường đi. Cây là đồ thị vô hướng liên thông và không có chu trình. Cây bao trùm của G là một đồ thị con dạng cây chứa toàn bộ các đỉnh của G. Với n đỉnh, mỗi cây bao trùm có đúng $n - 1$ cạnh.

Cây bao trùm nhỏ nhất là cây bao trùm có tổng trọng số các cạnh nhỏ nhất trong tất cả các cây bao trùm của đồ thị. Nếu T là tập cạnh của cây và w(e) là trọng số cạnh e, tổng trọng số được xác định bởi $W(T) = \sum w(e)$, với e thuộc T. Mục tiêu của bài toán là tìm tập cạnh T thỏa mãn điều kiện cây bao trùm và làm cho W(T) nhỏ nhất. Một đồ thị có thể có nhiều cây bao trùm nhỏ nhất có cùng tổng trọng số.

Trong đề tài này, thuật toán Prim bắt đầu từ đỉnh 1 và từng bước mở rộng cây bằng cách chọn cạnh có trọng số nhỏ nhất nối một đỉnh đã thuộc cây với một đỉnh chưa thuộc cây. Khi có nhiều ứng viên cùng trọng số, chương trình ưu tiên đỉnh mới có số hiệu nhỏ hơn; nếu tiếp tục bằng nhau thì ưu tiên đỉnh cha có số hiệu nhỏ hơn. Quy tắc này giúp xác định thống nhất thứ tự lựa chọn cạnh.

Dữ liệu đầu vào được biểu diễn bằng danh sách canh, trong đó mỗi phần tử là bộ ba (u, v, w), tương ứng với hai đầu mút và trọng số của một cạnh. Trước khi chạy Prim, danh sách cạnh được chuyển thành danh sách kề. Mỗi cạnh vô hướng được lưu ở cả hai đầu

mút để chương trình có thể duyệt các đỉnh kề từ một đỉnh bất kỳ.

Ví dụ, với V = {1, 2, 3} và danh sách cạnh [(1, 2, 4), (1, 3, 1), (2, 3, 2)], thuật toán bắt đầu từ đỉnh 1, chọn cạnh (1, 3, 1), sau đó chọn cạnh (3, 2, 2). Hai cạnh được chọn kết nối đủ ba đỉnh, không tạo chu trình và có tổng trọng số bằng 1 + 2 = 3.

Nếu đồ thị không liên thông thì không tồn tại cây bao trùm chứa toàn bộ n đỉnh; chương trình thông báo KHONG LIEN THONG. Với trường hợp n = 1 và không có cạnh, cây bao trùm gồm một đỉnh, danh sách cạnh rỗng và tổng trọng số bằng 0.

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Chương trình sử dụng danh sách cạnh để tiếp nhận dữ liệu, danh sách kề để biểu diễn đồ thị và hàng đợi ưu tiên để lựa chọn cạnh. Ngoài ra, chương trình sử dụng mảng đánh dấu đỉnh, danh sách cạnh đã chọn và biến tổng trọng số để lưu trạng thái trong quá trình chạy Prim.

Danh sách cạnh canh lưu các bộ ba (u, v, w), trong đó u và v là hai đầu mút, w là trọng số của cạnh. Cấu trúc này thuận tiện cho việc nhập, khai báo các ví dụ và kiểm tra tính hợp lệ của dữ liệu. Mỗi cạnh vô hướng chỉ xuất hiện một lần trong danh sách đầu vào.

Danh sách kề ke gồm n + 1 danh sách con, sử dụng các chỉ số từ 1 đến n; phần tử ke[0] không dùng để biểu diễn đỉnh. Với cạnh (u, v, w), chương trình thêm (v, w) vào ke[u] và thêm (u, w) vào ke[v]. Nhờ đó, khi nhận một đỉnh vào cây, thuật toán có thể duyệt trực tiếp các cạnh kề với đỉnh đó mà không cần quét lại toàn bộ danh sách cạnh.

Hàng đợi ưu tiên hang được tổ chức bằng heap cực tiểu, sử dụng hai thao tác heappush và heappop của thư viện heapq. Mỗi phần tử có dạng (w, đỉnh mới, cha), trong đó cha là đỉnh đã thuộc cây khi ứng viên được đưa vào hàng đợi. Thao tác heappush bổ sung ứng viên; heappop lấy ứng viên nhỏ nhất theo thứ tự ưu tiên: trọng số, số hiệu đỉnh mới, rồi số hiệu đỉnh cha. Cấu trúc này phù hợp với yêu cầu chọn cạnh nhẹ nhất để mở rộng cây.

Mảng da_chon gồm n + 1 giá trị Boolean, ban đầu đều bằng False. Khi đỉnh u được nhận vào cây, da_chon[u] được đặt thành True. Nếu một ứng viên được lấy khỏi heap nhưng đỉnh đích đã được chọn, thuật toán bỏ qua ứng viên đó. Việc kiểm tra này ngăn một đỉnh được nhận nhiều lần và tránh bổ sung cạnh tạo chu trình.

Danh sách canh_chon lưu các cạnh được chấp nhận theo đúng thứ tự lựa chọn, mỗi cạnh có dạng (cha, u, w). Biến tong lưu tổng trọng số của các cạnh trong canh_chon. Khi nhận một cạnh mới, chương trình thêm cạnh vào danh sách và cộng trọng số tương ứng vào tong. Riêng đỉnh bắt đầu được nhận mà không bổ sung cạnh.

Để phục vụ khảo sát, từ điển dem lưu số lần đẩy cạnh vào heap, lấy phần tử khỏi heap, lấy cạnh khỏi heap, nhận đỉnh và bỏ qua ứng viên. Khi tham số thong_ke được cung cấp, các bộ đếm được ghi vào từ điển này sau vòng lặp. Các bộ đếm hỗ trợ đánh giá số thao tác và không tham gia quyết định lựa chọn cạnh.

2.2.2. Quy ước và điều kiện hợp lệ

Các đỉnh được đánh số liên tiếp từ 1 đến n, với $1 \le n \le 30$. Danh sách kề ke và mảng da_chon đều có n + 1 phần tử để truy cập trực tiếp bằng số hiệu đỉnh. Chỉ số 0 không đại diện cho đỉnh của đồ thị. Theo phiếu giao đề tài, chương trình demo bắt đầu thuật toán Prim từ đỉnh s = 1.

Danh sách cạnh đầu vào canh là một list hoặc tuple, mỗi phần tử là một list hoặc tuple gồm đúng ba giá trị (u, v, w). Số đỉnh, hai đầu mút và trọng số phải có kiểu số nguyên; giá trị Boolean không được chấp nhận thay cho số nguyên. Hai đầu mút thỏa mãn $1 \le u, v \le n$ và $u \ne v$; trọng số thỏa mãn $0 \le w \le 1000$. Trọng số bằng 0 là hợp lệ.

Đồ thị được quy ước là vô hướng, không có cạnh tự nối và không có cạnh trùng. Vì vậy, (u, v) và (v, u) được xem là cùng một cặp đỉnh, kể cả khi hai bản ghi có trọng số khác nhau. Hai cạnh khác nhau vẫn được phép có cùng trọng số. Với m là số cạnh, ta có $m = \text{len(canh)}$ và $0 \le m \le n(n-1)/2$.

Trong danh sách kề, mỗi cạnh đầu vào được lưu thành hai phần tử đối xứng: (v, w) trong ke[u] và (u, w) trong ke[v]. Đây là cách biểu diễn một cạnh vô hướng, không

phải hai cạnh trùng của dữ liệu đầu vào. Đỉnh không có cạnh kề được biểu diễn bằng danh sách rỗng [].

Các phần tử trong heap có dạng (w, đỉnh mới, cha) và được so sánh theo thứ tự tăng dần của ba thành phần. Khi trọng số bằng nhau, ưu tiên đỉnh mới có số hiệu nhỏ hơn; nếu tiếp tục bằng nhau, ưu tiên đỉnh cha có số hiệu nhỏ hơn. Heap có thể đồng thời chứa nhiều ứng viên hướng tới cùng một đỉnh. Khi lấy một ứng viên ra, thuật toán phải kiểm tra da_chon trước khi nhận đỉnh và bổ sung cạnh.

Phần tử khởi tạo (0, s, 0) chỉ dùng để đưa đỉnh bắt đầu vào quá trình xử lý, không đại diện cho cạnh thật. Giá trị cha = 0 đánh dấu trường hợp này. Do đó, phần tử nguồn không được thêm vào canh_chon và không được tính là một cạnh trong các bộ đếm đẩy hoặc lấy cạnh.

Trong quá trình xử lý, mỗi đỉnh chỉ được nhận một lần. Sau khi đã nhận k đỉnh, với $k \geq 1$, danh sách canh_chon có đúng $k - 1$ cạnh và tong bằng tổng trọng số của các cạnh đó. Mỗi cạnh được chấp nhận nối cây hiện tại với một đỉnh mới nên không tạo chu trình.

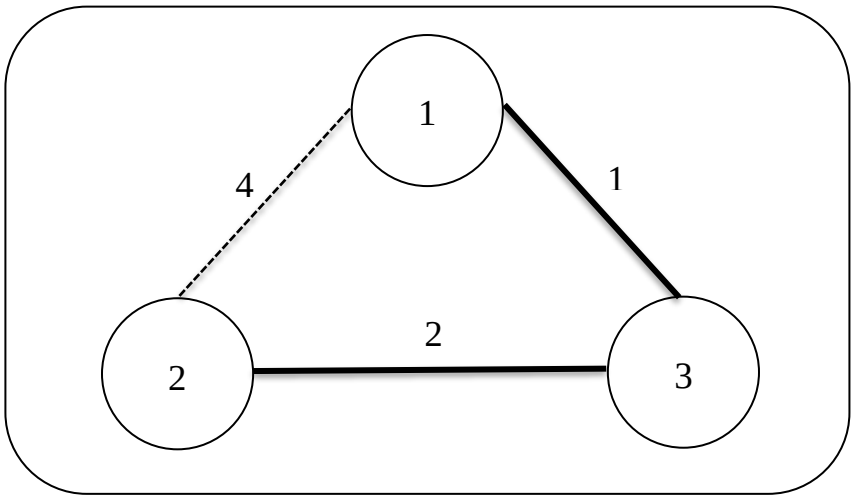
Đồ thị không liên thông vẫn là dữ liệu đầu vào hợp lệ, nhưng không có cây bao trùm chứa toàn bộ n đỉnh. Khi không nhận đủ n đỉnh, hàm prim trả về None; chương trình demo hiển thị KHONG LIEN THONG. Trường hợp $n = 1$ và không có cạnh trả về danh sách cạnh rỗng cùng tổng trọng số bằng 0. Dữ liệu vi phạm các quy định về kiểu, miền giá trị hoặc cấu trúc được báo bằng ngoại lệ ValueError.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	**Ý nghĩa**	**Miền / bất biến**
n	Số đỉnh của đồ thị.	Số nguyên, $1 \leq n \leq 30$; các đỉnh được đánh số từ 1 đến n.
m	Số cạnh của đồ thị.	$m = \text{len(canh)}$; $0 \leq m \leq n(n-1)/2$.
s	Đỉnh bắt đầu của thuật toán.	$s = 1$ theo phiếu DT080; được nhận vào cây mà không thêm cạnh.

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
canh	Danh sách cạnh đầu vào; mỗi cạnh có dạng (u, v, w).	$1 \le u, v \le n$; $u \ne v$; trọng số nguyên từ 0 đến 1000; không trùng cặp đỉnh vô hướng.
ke	Danh sách kề của đồ thị vô hướng.	Có n + 1 danh sách con; ke[0] không dùng. Mỗi cạnh (u, v, w) được lưu bằng (v, w) trong ke[u] và (u, w) trong ke[v].
hang	Heap cực tiểu lưu các ứng viên (w, đỉnh mới, cha).	Ưu tiên tăng dần theo trọng số, đỉnh mới, rồi cha. Khởi tạo bằng [(0, s, 0)]; ứng viên có đỉnh đích đã được chọn phải bị bỏ qua khi lấy ra.
da_chon	Mảng đánh dấu các đỉnh đã thuộc cây.	Có n + 1 giá trị Boolean; ban đầu đều False. Mỗi đỉnh chỉ chuyển từ False sang True một lần.
canh_chon	Danh sách cạnh được nhận, dạng (cha, u, w), theo thứ tự lựa chọn.	Sau khi nhận $k \ge 1$ đỉnh, có $k - 1$ cạnh; các cạnh nối những đỉnh đã nhận thành cây. Khi nhận đủ n đỉnh, có $n - 1$ cạnh.
tong	Tổng trọng số các cạnh đã nhận.	Khởi tạo bằng 0; luôn bằng tổng trọng số của các cạnh trong canh_chon.
dem	Từ điển lưu các bộ đếm thao tác phục vụ khảo sát.	Các giá trị là số nguyên không âm. lay_heap tính cả phần tử nguồn; day_canh và lay_canh không tính phần tử nguồn; nhan_dinh ≤ n.

Sơ đồ hoặc ví dụ mô hình



Hình 2.1. Đồ thị ví dụ và cây bao trùm nhỏ nhất bắt đầu từ đỉnh 1

Hình 2.1 minh họa đồ thị vô hướng gồm ba đỉnh và ba cạnh: (1, 2, 4), (1, 3, 1), (2, 3, 2). Bắt đầu từ đỉnh 1, thuật toán Prim chọn cạnh (1, 3, 1), sau đó chọn cạnh (3, 2, 2). Hai cạnh nét liền đậm kết nối đủ ba đỉnh, không tạo chu trình và tạo thành cây bao trùm nhỏ nhất có tổng trọng số bằng 3. Cạnh nét đứt có trọng số 4 không được chọn. Hình do sinh viên tự vẽ theo dữ liệu Ví dụ 1 trên phiếu DT080.

Cơ sở thuật toán và giới hạn áp dụng

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Thuật toán Prim xây dựng cây bao trùm nhỏ nhất theo phương pháp tham lam. Từ một đỉnh bắt đầu, thuật toán duy trì một cây và mở rộng cây bằng cách chọn cạnh có trọng số nhỏ nhất nối một đỉnh đã thuộc cây với một đỉnh chưa thuộc cây. Nguyên lý này được trình bày trong giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, mục 6.5.1, trang in 150–151 [1].

Gọi S là tập các đỉnh đã được nhận vào cây. Sau khi nhận đỉnh bắt đầu s = 1, ta có S = {1}. Tại mỗi bước mở rộng, thuật toán xét các cạnh có một đầu thuộc S và đầu còn lại thuộc V \ S. Cạnh nhẹ nhất trong nhóm này được chọn để bổ sung một đỉnh mới. Việc lựa chọn được thực hiện trên các cạnh nối từ toàn bộ cây hiện tại ra bên ngoài.

Trong chương trình, hàng đợi ưu tiên hang lưu các ứng viên dưới dạng (w, đỉnh mới, cha). Mỗi lần xử lý, thuật toán lấy bộ nhỏ nhất khỏi heap. Nếu đỉnh mới đã thuộc cây, ứng viên bị bỏ qua. Nếu đỉnh mới chưa thuộc cây, thuật toán đánh dấu đỉnh đó, thêm cạnh tương ứng vào canh_chon và cộng trọng số vào tong. Riêng phần tử nguồn (0, s, 0) chỉ dùng để nhận đỉnh bắt đầu, không bổ sung cạnh.

Sau khi nhận đỉnh u, chương trình duyệt danh sách kề ke[u]. Với mỗi đỉnh kề v chưa được chọn, ứng viên (w, v, u) được đưa vào heap. Các ứng viên cũ vẫn được giữ lại để tiếp tục so sánh với ứng viên mới. Vì một đỉnh có thể được tiếp cận từ nhiều cạnh, heap có thể chứa nhiều ứng viên cùng hướng tới đỉnh đó; mảng da_chon giúp loại bỏ các ứng viên không còn cần thiết khi lấy ra.

Khi các ứng viên có cùng trọng số, chương trình ưu tiên đỉnh mới có số hiệu nhỏ hơn; nếu tiếp tục bằng nhau thì ưu tiên đỉnh cha có số hiệu nhỏ hơn. Quy tắc này xác định thống nhất thứ tự lựa chọn nhưng vẫn giữ nguyên nguyên tắc ưu tiên trọng số nhỏ nhất.

Cơ sở của bước lựa chọn tham lam là tính chất lát cắt: khi các cạnh đã chọn còn có thể

mở rộng thành một cây bao trùm nhỏ nhất, việc thêm một cạnh nhẹ nhất nối S với V \ S vẫn duy trì khả năng đó. Đồng thời, cạnh được thêm luôn nối tới một đỉnh chưa thuộc cây nên không tạo chu trình. Với đồ thị liên thông, quá trình nhận đủ n đỉnh tạo ra cây gồm n − 1 cạnh có tổng trọng số nhỏ nhất.

Trong ví dụ ở Hình 2.1, từ đỉnh 1, thuật toán so sánh cạnh 1–2 trọng số 4 với cạnh 1–3 trọng số 1 và chọn cạnh 1–3. Khi đó S = {1, 3}; hai cạnh nối cây ra đỉnh 2 có trọng số 4 và 2, nên cạnh 3–2 trọng số 2 được chọn. Kết quả gồm hai cạnh (1, 3, 1), (3, 2, 2), với tổng trọng số bằng 3.

2.3.2. Điều kiện bắt đầu và kết thúc

Trước khi thực hiện thuật toán, dữ liệu đồ thị phải được kiểm tra tính hợp lệ và chuyển thành danh sách kề ke. Các đỉnh được đánh số từ 1 đến n; mỗi cạnh vô hướng được lưu ở cả hai đầu mút. Đỉnh bắt đầu phải là số nguyên thuộc phạm vi 1 đến n. Trong chương trình demo của đề tài, đỉnh bắt đầu được cố định là s = 1.

Ở trạng thái khởi tạo, mảng da_chon có n + 1 phần tử đều bằng False, danh sách canh_chon rỗng và tong bằng 0. Các bộ đếm thao tác trong dem được đặt bằng 0. Heap hang chứa duy nhất phần tử (0, s, 0), dùng để đưa đỉnh nguồn vào quá trình xử lý.

Trong lần lấy đầu tiên, đỉnh s được đánh dấu đã chọn và bộ đếm nhan_dinh tăng lên 1. Vì phần tử nguồn có cha = 0, chương trình không thêm cạnh vào canh_chon và không tăng tong. Sau đó, các cạnh nối từ s tới những đỉnh chưa được chọn được đưa vào heap.

Vòng lặp tiếp tục khi hang còn phần tử. Mỗi lần lặp lấy một ứng viên ra khỏi heap; nếu đỉnh đích đã được chọn thì bỏ qua, nếu chưa được chọn thì nhận đỉnh, cập nhật cạnh và tổng trọng số, rồi bổ sung các ứng viên mới từ danh sách kề của đỉnh đó.

Trong bản cài đặt này, điều kiện kết thúc vòng lặp là heap rỗng. Chương trình vẫn lấy ra và bỏ qua những ứng viên còn lại sau khi đã nhận đủ n đỉnh. Vì vậy, số lần lấy phần tử khỏi heap có thể lớn hơn số đỉnh được nhận. Cách xử lý này cũng được phản ánh trong các bộ đếm dùng cho phần khảo sát.

Thuật toán luôn kết thúc vì mỗi đỉnh chỉ được nhận một lần và danh sách kề của đỉnh đó chỉ được duyệt khi đỉnh được nhận. Với đồ thị vô hướng hợp lệ, mỗi cạnh được đưa vào heap nhiều nhất một lần: khi đầu mút được nhận trước duyệt tới đầu mút chưa được chọn. Số phần tử có thể được đưa vào heap là hữu hạn và mỗi vòng lặp đều lấy ra một phần tử.

Sau khi heap rỗng, chương trình kiểm tra dem["nhan_dinh"]. Nếu giá trị này bằng n, toàn bộ các đỉnh đã được nhận; hàm prim trả về cặp (canh_chon, tong). Khi đó, canh_chon chứa đúng n − 1 cạnh tạo thành cây bao trùm nhỏ nhất và tong là tổng trọng số của cây.

Nếu số đỉnh được nhận nhỏ hơn n, thuật toán chỉ tiếp cận được thành phần liên thông chứa đỉnh s. Hàm prim trả về None và chương trình demo hiển thị KHONG LIEN THONG. Các cạnh đã chọn trong thành phần đó không được trả về như một cây bao trùm của toàn bộ đồ thị.

Với trường hợp n = 1 và không có cạnh, phần tử nguồn được lấy ra, đỉnh duy nhất được nhận và heap trở thành rỗng. Hàm trả về ([], 0), tương ứng với cây gồm một đỉnh, không có cạnh và tổng trọng số bằng 0.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Quy mô đầu vào được xác định bởi n là số đỉnh và m là số cạnh của đồ thị. Hai tham số này ảnh hưởng đến số phần tử phải lưu, số lượt duyệt danh sách kề và số thao tác trên hàng đợi ưu tiên. Phân tích dưới đây xét bản cài đặt Prim sử dụng danh sách kề và heap cực tiểu, khi tắt chế độ in lần vết.

Việc xây dựng danh sách kề sau khi dữ liệu đã được kiểm tra cần khởi tạo n + 1 danh sách con và lưu hai phần tử kề cho mỗi cạnh vô hướng. Do đó, bước này có thời gian O(n + m) và sử dụng bộ nhớ O(n + m). Việc khởi tạo mảng đánh dấu trong hàm prim cần thời gian và bộ nhớ O(n).

Trong quá trình chạy Prim, mỗi đỉnh được nhận nhiều nhất một lần. Danh sách kề của

một đỉnh chỉ được duyệt khi đỉnh đó được nhận, nên tổng số phần tử kề được xét không vượt quá 2m. Mỗi cạnh được đưa vào heap nhiều nhất một lần vì chương trình chỉ thêm ứng viên hướng tới đỉnh chưa được chọn. Ngoài các ứng viên cạnh, heap còn có một phần tử nguồn (0, s, 0).

Heap có thể chứa O(m) ứng viên cạnh. Một thao tác thêm hoặc lấy phần tử trên heap có chi phí O(log(m + 1)). Vì tổng số thao tác heap là O(m + 1), kết hợp với khởi tạo và duyệt danh sách kề, thời gian của phần thuật toán được chặn trên bởi O(n + m log(m + 1)).

Với đồ thị đơn vô hướng, $m \leq n(n-1)/2$. Khi $n \geq 2$, có thể biểu diễn giới hạn trên theo n là O(n + m log n). Nếu đồ thị liên thông thì $m \geq n - 1$, nên độ phức tạp được rút gọn thành O(m log n), phù hợp với mức đánh giá Prim trong Bảng 6.5 của giáo trình, trang in 153 [1]. Riêng đồ thị một đỉnh, không có cạnh, quá trình xử lý cần thời gian O(1).

Về bộ nhớ, dữ liệu đầu vào dưới dạng danh sách cạnh chiếm O(m), còn danh sách kề chiếm O(n + m). Trong hàm prim, mảng da_chon chiếm O(n), heap hang chiếm O(m + 1) và danh sách kết quả canh_chon chiếm O(n). Các biến tổng và từ điển có số lượng bộ đếm cố định chiếm O(1). Vì vậy, bộ nhớ phụ của thuật toán có giới hạn O(n + m); nếu tính cả dữ liệu đầu vào và kết quả, tổng bộ nhớ vẫn là O(n + m).

Chi phí nhập dữ liệu, kiểm tra tính hợp lệ và in kết quả được xem xét riêng với chi phí lõi của Prim. Khi bật lan_vet, chương trình còn tạo danh sách các đỉnh đã nhận và in trạng thái sau từng bước nhận đỉnh. Những thao tác bổ sung này làm tăng thời gian thực hiện, nên giới hạn thời gian của phần lõi không được dùng để mô tả toàn bộ quá trình chạy có in lần vết.

Để khảo sát, chương trình đếm số lần đẩy cạnh vào heap, lấy cạnh khỏi heap, lấy phần tử khỏi heap, nhận đỉnh và bỏ qua ứng viên. Bộ đếm lấy phần tử khỏi heap tính cả phần tử nguồn; các bộ đếm đẩy và lấy cạnh không tính phần tử nguồn. Đây là số thao tác ở mức thuật toán, không phải số phép so sánh bên trong heap và không phải thời gian chạy.

Khảo sát sử dụng đồ thị đường đi và đồ thị đầy đủ tại ba quy mô n = 5, 10, 20. Đồ thị đường đi có m = n − 1, còn đồ thị đầy đủ có m = n(n − 1)/2. Với cùng số đỉnh, đồ thị đầy đủ dự kiến phát sinh nhiều ứng viên và nhiều lượt bỏ qua hơn. Kết quả thực tế được trình bày ở Chương 4 để đối chiếu với phân tích, trong phạm vi các trường hợp đã khảo sát.

Giả mã nền tảng

THUẬT TOÁN PRIM(ke, s)

Đầu vào:

ke: danh sách kề của đồ thị vô hướng hợp lệ;
các đỉnh được đánh số từ 1 đến n.
s: đỉnh bắt đầu hợp lệ; đề tài sử dụng s = 1.

Đầu ra:

(canh_chon, tong) nếu đồ thị liên thông;
None nếu đồ thị không liên thông.

n ← số phần tử của ke − 1
da_chon ← mảng n + 1 phần tử đều bằng False
hang ← heap cực tiểu chứa (0, s, 0)
canh_chon ← danh sách rỗng
tong ← 0
dem["nhan_dinh"] ← 0

TRONG KHI hang không rỗng:
(w, u, cha) ← lấy phần tử nhỏ nhất khỏi hang

NẾU da_chon[u] = True:
TIẾP TỤC vòng lặp kế tiếp

da_chon[u] ← True

```
    dem["nhan_dinh"] ← dem["nhan_dinh"] + 1

    NẾU cha ≠ 0:
        thêm (cha, u, w) vào cuối canh_chon
        tong ← tong + w

    VỚI MỖI (v, trong_so) trong ke[u]:
        NẾU da_chon[v] = False:
            thêm (trong_so, v, u) vào hang

NẾU dem["nhan_dinh"] ≠ n:
    TRẢ VỀ None

TRẢ VỀ (canh_chon, tong)
============================================================
```

Trong giả mã, ký hiệu ← biểu thị phép gán. Heap so sánh các bộ ba theo thứ tự tăng dần của trọng số, số hiệu đỉnh mới và số hiệu đỉnh cha. Phần tử (0, s, 0) dùng để nhận đỉnh nguồn, không đại diện cho một cạnh thật. Vòng lặp kết thúc khi heap rỗng, sau đó kiểm tra số đỉnh đã nhận để xác định đồ thị có liên thông hay không.

Nguồn tham khảo: giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, mục 6.5.1, trang in 150–151 [1]. Giả mã được điều chỉnh cho đề tài DT080 bằng cách sử dụng chỉ số đỉnh từ 1 đến n, bắt đầu từ đỉnh 1, dùng cha = 0 để nhận diện phần tử nguồn, bổ sung kiểm tra số đỉnh đã nhận và trả kết quả theo thứ tự (canh_chon, tong).

Giả mã tập trung vào phần xử lý chính, giả định dữ liệu đầu vào đã được kiểm tra hợp lệ và lược bỏ các lệnh in lần vết cùng những bộ đếm khảo sát khác. Trong mã Python, thao tác duyệt danh sách kề và thêm ứng viên được tách thành hàm them_ung_vien.

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Chương trình giải bài toán DT080: xây dựng cây bao trùm nhỏ nhất của đồ thị vô hướng có trọng số bằng thuật toán Prim, bắt đầu từ đỉnh 1. Hợp đồng input/output quy định dữ liệu được tiếp nhận, điều kiện để thực hiện thuật toán và kết quả phải trả về trong từng trường hợp.

Ở mức chương trình, dữ liệu đầu vào gồm số đỉnh n và danh sách cạnh canh. Số đỉnh là số nguyên thỏa mãn $1 \leq n \leq 30$. Mỗi cạnh được biểu diễn bởi bộ ba (u, v, w), trong đó u và v là hai đỉnh khác nhau thuộc phạm vi 1 đến n, w là số nguyên từ 0 đến 1000. Danh sách cạnh không được chứa cạnh tự nối hoặc cạnh trùng theo nghĩa vô hướng. Số cạnh m được xác định bằng len(canh).

Chương trình hỗ trợ dữ liệu khai báo trực tiếp trong mã và dữ liệu nhập từ bàn phím. Khi chạy chế độ nhập, người dùng lần lượt nhập n, m và m dòng cạnh; mỗi dòng gồm ba số nguyên u, v, w. Số cạnh phải thỏa mãn $0 \leq m \leq n(n-1)/2$. Đỉnh bắt đầu được gán bằng 1 theo yêu cầu của đề tài.

Hàm tao_danh_sach_ke(n, canh) tiếp nhận số đỉnh và danh sách cạnh, kiểm tra dữ liệu rồi trả về danh sách kề ke. Kết quả có n + 1 danh sách con, trong đó ke[0] không dùng để biểu diễn đỉnh. Mỗi cạnh (u, v, w) được lưu bằng hai phần tử đối xứng: (v, w) trong ke[u] và (u, w) trong ke[v].

Hàm xử lý chính có chữ ký prim(ke, s=1, lan_vet=False, thong_ke=None). Tiền điều kiện là ke biểu diễn một đồ thị vô hướng hợp lệ theo quy ước trên và s là đỉnh bắt đầu hợp lệ. Tham số lan_vet điều khiển việc in các bước xử lý; tham số thong_ke nhận một từ điển để lưu bộ đếm thao tác hoặc nhận None nếu không cần lấy thống kê. Trong chương trình demo, hàm được gọi với s = 1.

Nếu đồ thị liên thông, hàm prim trả về cặp (canh_chon, tong). Danh sách canh_chon

gồm đúng n − 1 cạnh, mỗi cạnh có dạng (cha, u, w), được lưu theo thứ tự thuật toán lựa chọn. Các cạnh phải thuộc đồ thị đầu vào theo nghĩa vô hướng, kết nối đủ n đỉnh và không tạo chu trình. Giá trị tong bằng tổng trọng số các cạnh đã chọn và là tổng nhỏ nhất trong tất cả các cây bao trùm của đồ thị. Chương trình demo hiển thị danh sách cạnh với nhãn chon và tổng trọng số với nhãn tong.

Thứ tự lựa chọn ứng viên được xác định theo bộ (trọng số, đỉnh mới, đỉnh cha), so sánh tăng dần từ trái sang phải. Quy tắc này giúp kết quả có thứ tự thống nhất khi các cạnh có trọng số bằng nhau. Một cạnh trong kết quả có thể được ghi theo chiều ngược với bản ghi đầu vào vì đồ thị là vô hướng.

Nếu đồ thị không liên thông, hàm prim trả về None và chương trình demo in KHONG LIEN THONG. Trường hợp này không trả danh sách cạnh của riêng thành phần chứa đỉnh 1 như một cây bao trùm của toàn bộ đồ thị. Với n = 1 và không có cạnh, kết quả hợp lệ là ([], 0).

Khi dữ liệu vi phạm điều kiện đầu vào, chương trình báo lỗi thay vì tiếp tục tính cây bao trùm. Các trường hợp không hợp lệ được phát hiện bằng ngoại lệ ValueError; chương trình demo hiển thị thông báo bắt đầu bằng DU LIEU KHONG HOP LE: kèm nguyên nhân. Việc kiểm tra cụ thể được trình bày ở mục 3.1.2.

3.1.2. Kiểm tra đầu vào

Việc kiểm tra đầu vào được thực hiện trước khi tạo danh sách kề và chạy thuật toán Prim. Chương trình sử dụng hàm kiem_tra_du_lieu(n, canh) trong tệp src/kiem_tra.py để kiểm tra dữ liệu khai báo trực tiếp hoặc dữ liệu đã đọc từ bàn phím. Khi phát hiện vi phạm, hàm phát sinh ngoại lệ ValueError kèm thông báo nguyên nhân và dừng kiểm tra tại lỗi đầu tiên.

Trước hết, n phải có kiểu số nguyên và thuộc khoảng từ 1 đến 30. Chương trình sử dụng điều kiện type(n) is int để không chấp nhận giá trị Boolean thay cho số nguyên. Sau đó, canh phải là một list hoặc tuple; mỗi phần tử trong canh cũng phải là list hoặc tuple gồm đúng ba giá trị (u, v, w).

Đối với từng cạnh, cả u, v và w phải có kiểu số nguyên, không chấp nhận Boolean. Hai đầu mút phải thuộc phạm vi 1 đến n và khác nhau. Điều kiện $u \neq v$ loại bỏ cạnh tự nối. Trọng số phải thuộc khoảng từ 0 đến 1000; do đó, cạnh trọng số 0 được chấp nhận, còn trọng số âm hoặc lớn hơn 1000 bị từ chối.

Để phát hiện cạnh trùng trong đồ thị vô hướng, chương trình chuẩn hóa hai đầu mút thành cap_dinh = (min(u, v), max(u, v)). Tập hợp da_gap lưu các cặp đỉnh đã xuất hiện. Nếu cap_dinh đã có trong da_gap, cạnh hiện tại bị từ chối; ngược lại, cặp đỉnh được thêm vào tập hợp. Cách kiểm tra này phát hiện cả cạnh trùng cùng chiều và ngược chiều, kể cả khi hai bản ghi có trọng số khác nhau. Các cạnh nối những cặp đỉnh khác nhau vẫn được phép có cùng trọng số.

Trong chế độ nhập bàn phím, hàm nhap_do_thi() kiểm tra thêm định dạng dữ liệu nhập. Giá trị n được chuyển thành số nguyên và kiểm tra miền hợp lệ trước khi hỏi số cạnh. Giá trị m cũng phải chuyển được thành số nguyên và thỏa mãn $0 \leq m \leq n(n-1)/2$. Mỗi dòng cạnh phải có đúng ba thành phần và cả ba phải chuyển được thành số nguyên. Sau khi đọc đủ m dòng, hàm gọi kiem_tra_du_lieu(n, canh) để kiểm tra toàn bộ đồ thị.

Khi dữ liệu được khai báo trực tiếp, số cạnh được xác định bằng len(canh), không có tham số m riêng trong hàm kiểm tra. Các điều kiện đầu mút hợp lệ, không tự nối và không trùng cặp đỉnh đã bảo đảm số cạnh không vượt quá giới hạn của đồ thị đơn vô hướng.

Hàm prim kiểm tra riêng đỉnh bắt đầu s: giá trị này phải có kiểu số nguyên và nằm trong khoảng từ 1 đến n. Nếu cung cấp tham số thong_ke, tham số đó phải là từ điển; giá trị None được dùng khi không cần nhận thống kê. Danh sách kề truyền vào prim được giả định đã được tạo từ dữ liệu hợp lệ bằng hàm tao_danh_sach_ke.

Chương trình demo bắt các ngoại lệ ValueError của quá trình nhập và kiểm tra dữ liệu, sau đó hiển thị DU LIEU KHONG HOP LE: kèm nguyên nhân. Với một lần nhập sai, chương trình kết thúc lượt chạy thay vì tự động yêu cầu nhập lại. Nếu luồng nhập kết

thúc trước khi đọc đủ dữ liệu, chương trình bắt EOFError và hiển thị DA DUNG: Chua nhap du du lieu.

Ví dụ, cạnh (1, 2, -5) bị từ chối vì trọng số âm; cạnh (2, 2, 3) bị từ chối vì tự nối; hai cạnh (1, 2, 4) và (2, 1, 7) bị xem là trùng cặp đỉnh. Với n = 3, một cạnh chứa đỉnh 4 cũng không hợp lệ. Các dạng lỗi này được đưa vào bộ kiểm thử để kiểm tra khả năng phát hiện dữ liệu sai.

Tính liên thông không phải điều kiện để chấp nhận dữ liệu đầu vào. Một đồ thị đúng cấu trúc và đúng miền giá trị nhưng không liên thông vẫn được chuyển thành danh sách kề và chạy Prim. Sau khi xử lý, hàm trả về None nếu không nhận đủ n đỉnh; chương trình hiển thị KHONG LIEN THONG. Như vậy, dữ liệu sai được phân biệt rõ với dữ liệu hợp lệ nhưng không tồn tại cây bao trùm.

3.2. Thiết kế các hàm và dữ liệu

Chương trình được tổ chức thành các mô-đun theo nhiệm vụ: kiểm tra dữ liệu, biểu diễn đồ thị, thực hiện Prim, kiểm tra cấu trúc cây và chạy các tình huống thử nghiệm. Cách tổ chức này cho phép sử dụng chung phần thuật toán trong chương trình demo, bộ kiểm thử, chương trình lần vết và chương trình khảo sát.

Tệp src/kiem_tra.py chứa hàm kiem_tra_du_lieu(n, canh), chịu trách nhiệm kiểm tra kiểu dữ liệu, miền giá trị, cạnh tự nối và cạnh trùng. Hàm không thay đổi dữ liệu đầu vào. Khi phát hiện vi phạm, hàm phát sinh ValueError; nếu dữ liệu hợp lệ, quá trình xử lý tiếp tục sang bước tạo danh sách kề.

Tệp src/prim.py chứa ba hàm chính. Hàm tao_danh_sach_ke(n, canh) gọi bước kiểm tra dữ liệu và tạo danh sách kề mới cho đồ thị vô hướng. Hàm them_ung_vien(hang, u, ke, da_chon) duyệt các đỉnh kề của u, đưa ứng viên hướng tới đỉnh chưa được chọn vào heap và trả về số cạnh đã đẩy. Hàm này cập nhật hang nhưng không thay đổi ke hoặc da_chon.

Hàm prim(ke, s=1, lan_vet=False, thong_ke=None) điều khiển toàn bộ quá trình xây dựng cây. Hàm khởi tạo mảng đánh dấu, heap, danh sách cạnh kết quả, tổng trọng số

và các bộ đếm. Trong vòng lặp, hàm lấy ứng viên, kiểm tra trạng thái đỉnh, cập nhật cây và gọi them_ung_vien để bổ sung ứng viên mới. Khi kết thúc, hàm trả về (canh_chon, tong) nếu nhận đủ các đỉnh hoặc None nếu đồ thị không liên thông. Danh sách kề đầu vào được giữ nguyên; nếu có từ điển thong_ke, hàm cập nhật các bộ đếm vào từ điển đó.

Tệp src/kiem_cay.py chứa hàm kiem_cay(n, canh_chon), trả về True hoặc False để xác định các cạnh có tạo thành cây trên tập đỉnh từ 1 đến n hay không. Hàm kiểm tra số cạnh, tính hợp lệ của từng cạnh, chu trình và tính liên thông bằng cấu trúc tập hợp rời nhau. Hàm này chỉ kiểm tra cấu trúc cây và miền dữ liệu của các cạnh; nó không tự xác nhận cạnh thuộc đồ thị đầu vào hoặc tổng trọng số là nhỏ nhất. Các yêu cầu đó cần được kiểm tra bổ sung khi đối chiếu kết quả.

Tệp demo.py đảm nhiệm việc nhập dữ liệu và hiển thị kết quả. Hàm nhap_do_thi() đọc đồ thị từ bàn phím; hàm chay_vi_du(ten, n, canh) tạo danh sách kề, gọi Prim và in kết quả hoặc thông báo lỗi; hàm main() lựa chọn chế độ chạy theo tham số dòng lệnh. Phần giao tiếp này được tách khỏi thuật toán để cùng một hàm prim có thể được gọi từ nhiều chương trình khác nhau.

Các tệp hỗ trợ gồm test/runtests.py để chạy bộ kiểm thử tự động, truy_vet.py để quan sát diễn biến thuật toán và khao_sat.py để thu thập số thao tác trên các đồ thị khảo sát. Những tệp này sử dụng lại phần cài đặt trong thư mục src, giúp kết quả thử nghiệm và kết quả demo dựa trên cùng một thuật toán.

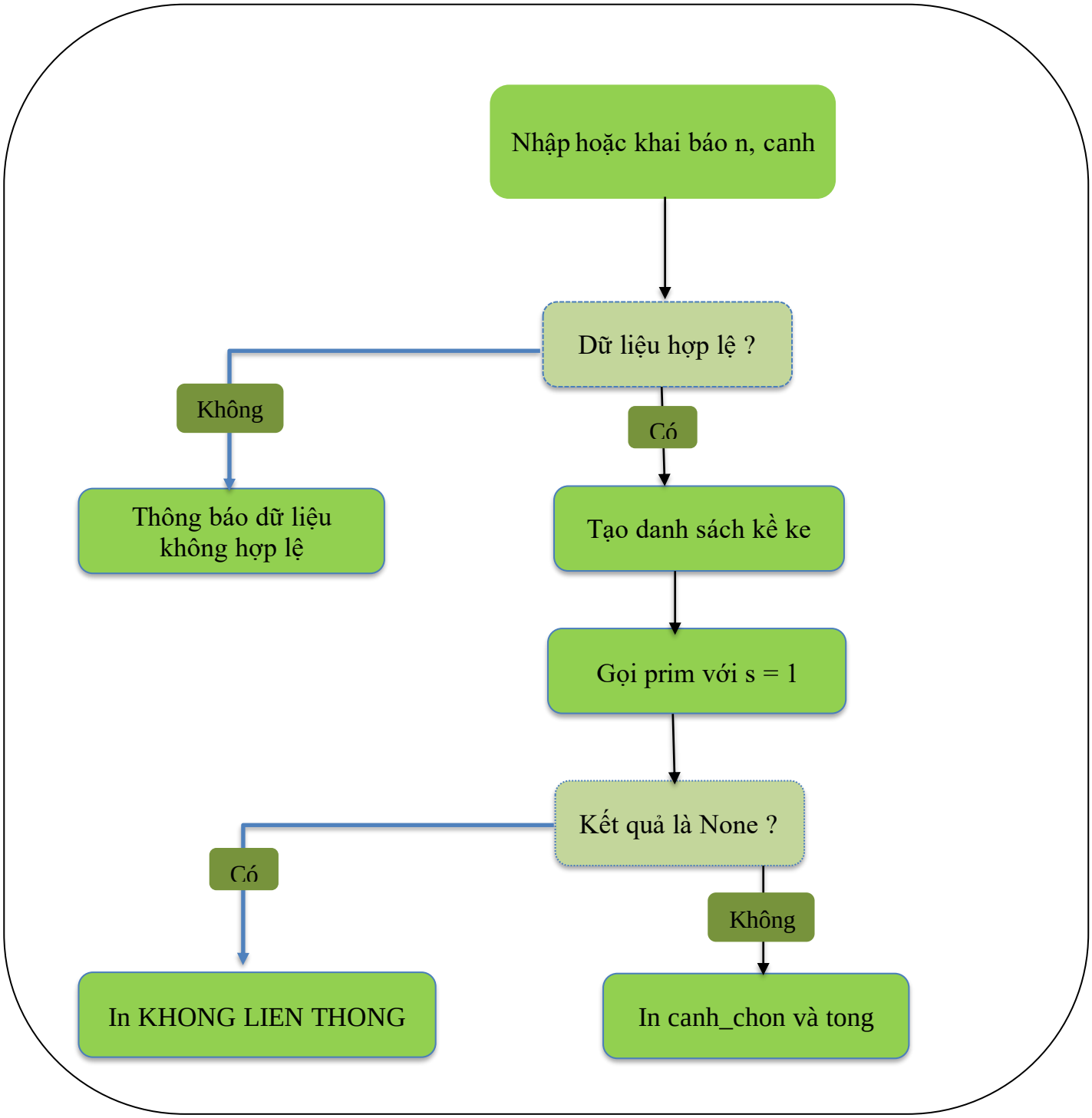
Phần cài đặt của đề tài bao gồm kiểm tra đầu vào, chuyển đổi danh sách cạnh thành danh sách kề, điều khiển các bước Prim, xử lý trường hợp đặc biệt và xây dựng các chương trình kiểm chứng. Hai thao tác heappush và heappop được sử dụng từ thư viện chuẩn heapq của Python để quản lý heap cực tiểu. Chương trình không tự cài đặt lại heap và không sử dụng thư viện đồ thị để tính sẵn cây bao trùm nhỏ nhất.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
kiem_tra_du_lieu — src/kiem_tra.py	n, canh.	Không trả dữ liệu khi hợp lệ; phát sinh ValueError khi vi phạm.	Kiểm tra kiểu, miền giá trị, cấu trúc cạnh, cạnh tự nối và cạnh trùng vô hướng.
tao_danh_sach_ke — src/prim.py	n, canh.	Danh sách kề ke gồm n + 1 danh sách con.	Gọi kiểm tra dữ liệu; lưu mỗi cạnh vô hướng tại cả hai đầu mút.
them_ung_vien — src/prim.py	hang, u, ke, da_chon.	Số cạnh đã đẩy; heap hang được cập nhật.	Duyệt các cạnh kề của u; dùng heappush để thêm ứng viên (w, v, u) khi v chưa được chọn.
prim — src/prim.py	ke; s = 1; lan_vet = False; thong_ke = None theo mặc định.	(canh_chon, tong) hoặc None; cập nhật thong_ke nếu được cung cấp.	Điều khiển Prim, bỏ qua đỉnh đã chọn, cập nhật cây và tổng trọng số, kiểm tra số đỉnh đã nhận. Sử dụng heapq để quản lý heap.
kiem_cay — src/kiem_cay.py	n, canh_chon.	True nếu các cạnh tạo thành cây hợp lệ trên n đỉnh; ngược lại là False.	Kiểm tra số cạnh, miền dữ liệu, chu trình và liên thông bằng tập hợp rời nhau; không tự kiểm tra tính tối ưu.
nhap_do_thi — demo.py	Dữ liệu bàn phím: n, m và m dòng u v w.	(n, canh) khi hợp lệ; phát sinh ngoại lệ khi nhập sai hoặc thiếu dữ liệu.	Đọc và chuyển đổi dữ liệu, kiểm tra số cạnh và gọi hàm kiểm tra đồ thị.

Hàm / tệp	Input	Output	Vai trò và phần tự cài
chay_vi_du — demo.py	ten, n, canh.	In tên ví dụ, dữ liệu đầu vào, kết quả hoặc thông báo lỗi.	Kết nối bước tạo danh sách kề với Prim; phân biệt dữ liệu sai, đồ thị không liên thông và kết quả hợp lệ.
main — demo.py	Tham số dòng lệnh và dữ liệu bàn phím nếu chọn chế độ nhập.	Điều phối việc chạy và hiển thị thông báo.	Chạy các ví dụ mặc định hoặc chế độ --nhap; xử lý lỗi nhập và tham số không được hỗ trợ.
test/runtests.py	Các ca kiểm thử và kết quả kỳ vọng được khai báo trong tệp.	Kết quả PASS/FAIL từng ca và thống kê toàn bộ bộ kiểm thử.	Gọi các hàm cần kiểm tra, đối chiếu kết quả và kiểm tra các trường hợp hợp lệ, biên, không liên thông, dữ liệu sai.
truy_vet.py	Đồ thị Ví dụ 1; đỉnh bắt đầu bằng 1.	Các bước lấy ứng viên, nhận hoặc bỏ qua đỉnh, cạnh được chọn và tổng trọng số.	Gọi Prim với chế độ lần vết để đối chiếu diễn biến thuật toán với phân tích bằng tay.
khao_sat.py	Đồ thị đường đi và đồ thị đầy đủ với n = 5, 10, 20.	Bảng số thao tác, tổng trọng số và kết quả đối chiếu.	Tạo dữ liệu khảo sát, gọi Prim để lấy thống kê và so sánh tổng trọng số với giá trị kỳ vọng n − 1.

Sơ đồ xử lý



Hình 3.1. Quy trình xử lý của chương trình DT080

Hình 3.1 mô tả quy trình xử lý từ dữ liệu đầu vào đến kết quả của chương trình DT080. Dữ liệu được nhập từ bàn phím hoặc khai báo trực tiếp, sau đó được kiểm tra tính hợp lệ. Nếu dữ liệu vi phạm quy định, chương trình thông báo lỗi và kết thúc lượt xử lý. Nếu hợp lệ, chương trình tạo danh sách kề và gọi thuật toán Prim với đỉnh bắt đầu s = 1. Sau khi thuật toán kết thúc, chương trình kiểm tra giá trị trả về. Nếu kết quả là None, chương trình hiển thị KHONG LIEN THONG. Ngược lại, chương trình hiển thị danh

sách cạnh được chọn và tổng trọng số của cây bao trùm nhỏ nhất. Sơ đồ thể hiện luồng xử lý tổng quát; các bước bên trong thuật toán Prim được trình bày tại mục 3.3.

Cài đặt phần quyết định

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
kiem_tra_du_lieu(n, canh)	Nhận n và canh cần kiểm tra; không giả định dữ liệu đã hợp lệ.	Nếu kết thúc bình thường, n và các cạnh thỏa mãn quy định DT080; dữ liệu đầu vào không bị thay đổi. Hàm trả về None.	Phát sinh ValueError khi sai kiểu, sai miền giá trị, sai cấu trúc cạnh, có cạnh tự nối hoặc cạnh trùng vô hướng.
tao_danh_sach_ke(n, canh)	Để tạo được danh sách kề, n và canh phải vượt qua kiem_tra_du_lieu. Hàm tự gọi bước kiểm tra này.	Trả về ke gồm n + 1 danh sách con; mỗi cạnh được lưu ở cả hai đầu mút. Không thay đổi canh.	Dữ liệu sai làm phát sinh ValueError. Danh sách cạnh rỗng vẫn hợp lệ; khi đó các danh sách con đều rỗng.
them_ung_vien(hang, u, ke, da_chon)	ke hợp lệ; u đã được nhận vào cây; da_chon phản ánh đúng các đỉnh đã nhận; hang là heap cực tiểu chứa các bộ ba cùng quy ước.	Thêm (w, v, u) vào hang với mỗi đỉnh kề v chưa được chọn; duy trì tính chất heap và trả về số cạnh đã đẩy. Không thay đổi ke hoặc da_chon.	Nếu không có đỉnh kề chưa được chọn thì trả về 0. Hàm không tự kiểm tra lại tính hợp lệ của toàn bộ dữ liệu.

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
prim(ke, s=1, lan_vet=False, thong_ke=None)	ke biểu diễn đồ thị vô hướng hợp lệ, dùng chỉ số 1 đến n; s là số nguyên từ 1 đến n; thong_ke là từ điển hoặc None.	Nếu liên thông, trả về (canh_chon, tong), gồm n − 1 cạnh tạo thành cây bao trùm nhỏ nhất và tổng trọng số. Giữ nguyên ke; cập nhật bộ đếm vào thong_ke nếu được cung cấp.	Trả về None nếu không liên thông; trả về ([], 0) khi n = 1 và không có cạnh. Phát sinh ValueError nếu s hoặc kiểu của thong_ke không hợp lệ.
kiem_cay(n, canh_chon)	Nhận n và danh sách cạnh cần kiểm tra; không giả định danh sách đã tạo thành cây.	Trả về True nếu dữ liệu đúng miền quy định, có n − 1 cạnh, không có chu trình và kết nối đủ n đỉnh; ngược lại trả về False. Không thay đổi danh sách đầu vào.	Với n = 1 và danh sách rỗng, trả về True. Không kiểm tra cạnh có thuộc đồ thị gốc hay không và không xác nhận tổng trọng số nhỏ nhất.

Các hàm được phân chia trách nhiệm để tránh kiểm tra hoặc cập nhật dữ liệu không đúng vị trí. Hàm kiem_tra_du_lieu chịu trách nhiệm xác thực đồ thị trước khi tạo danh sách kề; hàm prim giả định danh sách kề đã hợp lệ và chỉ kiểm tra riêng các tham số s, thong_ke. Hàm them_ung_vien thay đổi trực tiếp heap hang, còn prim cập nhật từ điển thong_ke nếu được cung cấp. Danh sách cạnh đầu vào và danh sách kề không bị thay đổi trong quá trình xây dựng cây. Hàm kiem_cay hỗ trợ kiểm chứng cấu trúc kết quả; tính tối ưu và sự tồn tại của các cạnh trong đồ thị đầu vào phải được đối chiếu riêng trong bộ kiểm thử.

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

Giả mã mô tả hai hàm them_ung_vien và prim trong tệp src/prim.py. Tên biến, thứ tự cập nhật, các bộ đếm và điều kiện kết thúc tương ứng với bản cài đặt Python. Dữ liệu đồ thị được giả định đã được kiểm tra hợp lệ và chuyển thành danh sách kề ke, sử dụng chỉ số đỉnh từ 1 đến n.

Hàng đợi ưu tiên hang là heap cực tiểu, chứa các bộ ba (trọng số, đỉnh mới, đỉnh cha). Các bộ được so sánh tăng dần từ trái sang phải, bảo đảm ưu tiên trọng số nhỏ nhất; khi bằng nhau, ưu tiên số hiệu đỉnh mới rồi đến số hiệu đỉnh cha.

a) Giả mã hàm thêm ứng viên

Hàm them_ung_vien duyệt danh sách kề của đỉnh u vừa được nhận vào cây. Với mỗi đỉnh kề v chưa được chọn, hàm đưa ứng viên (w, v, u) vào heap và tăng số cạnh đã đẩy. Giá trị trả về được dùng để cập nhật bộ đếm day_canh trong hàm prim.

```
HÀM them_ung_vien(hang, u, ke, da_chon):
    so_day ← 0

    VỚI MỖI (v, w) trong ke[u]:
        NẾU da_chon[v] = False:
            đẩy (w, v, u) vào heap hang
            so_day ← so_day + 1

    TRẢ VỀ so_day
```

Hàm chỉ thay đổi heap hang, không thay đổi danh sách kề ke hoặc mảng da_chon. Nếu không có đỉnh kề chưa được chọn, hàm không thêm ứng viên và trả về 0.

b) Giả mã hàm Prim

Hàm prim tiếp nhận danh sách kề, đỉnh bắt đầu và hai tham số tùy chọn phục vụ lần vết, thống kê. Trong đề tài, đỉnh bắt đầu là s = 1. Phần tử nguồn (0, s, 0) được đưa vào heap để khởi động quá trình xử lý; phần tử này không đại diện cho cạnh thật.

```
HÀM prim(ke, s = 1, lan_vet = False, thong_ke = None):
    n ← số phần tử của ke − 1

    NẾU kiểu của s không phải int hoặc s nằm ngoài [1, n]:
        PHÁT SINH ValueError: đỉnh bắt đầu không hợp lệ

    NẾU thong_ke khác None và không phải từ điển:
        PHÁT SINH ValueError: thong_ke không hợp lệ

    dem ← từ điển gồm:
        "day_canh": 0
        "lay_heap": 0
        "lay_canh": 0
        "nhan_dinh": 0
        "bo_qua": 0

    da_chon ← mảng n + 1 phần tử đều bằng False
    hang ← heap cực tiểu chứa (0, s, 0)
    canh_chon ← danh sách rỗng
    tong ← 0

    TRONG KHI hang không rỗng:
        (w, u, cha) ← lấy phần tử nhỏ nhất khỏi hang
        dem["lay_heap"] ← dem["lay_heap"] + 1

        NẾU cha ≠ 0:
```

```
    dem["lay_canh"] ← dem["lay_canh"] + 1

NẾU lan_vet = True:
    in số bước và bộ (w, u, cha) vừa lấy

NẾU da_chon[u] = True:
    dem["bo_qua"] ← dem["bo_qua"] + 1

    NẾU lan_vet = True:
        in thông báo bỏ qua đỉnh u

    TIẾP TỤC vòng lặp kế tiếp

da_chon[u] ← True
dem["nhan_dinh"] ← dem["nhan_dinh"] + 1

NẾU cha ≠ 0:
    thêm (cha, u, w) vào cuối canh_chon
    tong ← tong + w

    NẾU lan_vet = True:
        in cạnh được chọn và trọng số
NGƯỢC LẠI:
    NẾU lan_vet = True:
        in thông báo nhận đỉnh nguồn, không thêm cạnh

dem["day_canh"] ← dem["day_canh"]
    + them_ung_vien(hang, u, ke, da_chon)

NẾU lan_vet = True:
    tap_dinh ← các đỉnh v từ 1 đến n
                có da_chon[v] = True
```

```
            in tap_dinh và tong

    NẾU thong_ke khác None:
        cập nhật các cặp khóa–giá trị của dem vào thong_ke

    NẾU dem["nhan_dinh"] ≠ n:
        NẾU lan_vet = True:
            in số đỉnh đã nhận trên tổng số n đỉnh

        TRẢ VỀ None

    TRẢ VỀ (canh_chon, tong)
```

c) Giải thích thứ tự xử lý và các bộ đếm

Ký hiệu ← biểu thị phép gán. Mỗi vòng lặp lấy đúng một phần tử khỏi heap. Bộ đếm lay_heap tăng ngay sau thao tác này, kể cả khi phần tử lấy ra là phần tử nguồn hoặc ứng viên sẽ bị bỏ qua. Bộ đếm lay_canh chỉ tăng khi cha khác 0, tức phần tử đại diện cho một cạnh thật.

Nếu đỉnh u đã thuộc cây, chương trình tăng bo_qua và chuyển ngay sang vòng lặp tiếp theo. Nhánh này không bổ sung cạnh, không tăng tổng trọng số và không duyệt lại danh sách kề của u.

Nếu u chưa thuộc cây, chương trình đánh dấu da_chon[u] thành True và tăng nhan_dinh. Với ứng viên cạnh thật, cạnh (cha, u, w) được thêm vào canh_chon và w được cộng vào tong. Với phần tử nguồn có cha = 0, chương trình chỉ nhận đỉnh bắt đầu mà không thêm cạnh.

Sau khi đánh dấu đỉnh u, hàm them_ung_vien được gọi để bổ sung các cạnh hướng tới đỉnh chưa được chọn. Số cạnh được hàm này trả về được cộng vào day_canh. Việc

đánh dấu trước khi bổ sung ứng viên bảo đảm trạng thái đỉnh được cập nhật đúng tại thời điểm duyệt danh sách kề.

Vòng lặp kết thúc khi heap rỗng. Bản cài đặt không dừng ngay khi nhận đủ n đỉnh mà tiếp tục lấy và bỏ qua các ứng viên còn lại. Vì vậy, số lần lấy phần tử khỏi heap có thể lớn hơn số đỉnh được nhận.

Nếu thong_ke được cung cấp, các bộ đếm được ghi vào từ điển này trước khi kiểm tra tính liên thông. Nhờ đó, chương trình vẫn có thể thu được thống kê khi kết quả là None. Nếu số đỉnh đã nhận bằng n, hàm trả về (canh_chon, tong); ngược lại, hàm trả về None. Với đồ thị một đỉnh và không có cạnh, kết quả là ([], 0).

Các lệnh in khi bật lan_vet chỉ hiển thị diễn biến xử lý, không thay đổi quy tắc chọn cạnh hoặc kết quả trả về. Khi tắt chế độ này, thuật toán vẫn thực hiện các bước lựa chọn và cập nhật bộ đếm như trong giả mã.

3.3.2. Đoạn mã và giải thích

Các đoạn mã dưới đây được trích từ tệp src/prim.py tại phiên bản mã nguồn dùng để chạy kiểm thử, có mã commit C là 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc. Nội dung tập trung vào ba thao tác quyết định: bổ sung ứng viên, nhận hoặc bỏ qua đỉnh và xác định kết quả trả về.

a) Bổ sung ứng viên vào hàng đợi ưu tiên

```
def them_ung_vien(hang, u, ke, da_chon):
    """Thêm cạnh tới đỉnh chưa chọn; trả về số cạnh đã đẩy."""
    so_day = 0

    for v, w in ke[u]:
        if not da_chon[v]:
            # Ưu tiên: trọng số, đỉnh mới, đỉnh cha.
```

```
            heappush(hang, (w, v, u))
            so_day += 1

    return so_day
```

Hàm duyệt các cặp (v, w) trong danh sách kề của đỉnh u vừa được nhận vào cây. Điều kiện not da_chon[v] bảo đảm chỉ đưa vào heap các ứng viên hướng tới đỉnh chưa thuộc cây.

Bộ ba được đẩy vào heap có thứ tự (w, v, u), tương ứng với trọng số, đỉnh mới và đỉnh cha. Python so sánh các tuple theo thứ tự từ trái sang phải, nên heap ưu tiên trọng số nhỏ hơn; nếu bằng nhau thì ưu tiên đỉnh mới nhỏ hơn, sau đó là đỉnh cha nhỏ hơn.

Biến so_day đếm số cạnh thực sự được đưa vào heap và được trả về để cập nhật thống kê. Hàm không đánh dấu v ngay khi đưa ứng viên vào hàng đợi. Một đỉnh chỉ được nhận khi ứng viên của nó được lấy ra và vượt qua bước kiểm tra trong hàm prim.

b) Bỏ qua đỉnh đã chọn và cập nhật cây

Đoạn mã sau nằm trong vòng lặp while hang của hàm prim, sau khi chương trình đã lấy bộ (w, u, cha) khỏi heap và cập nhật các bộ đếm lấy phần tử:

```
        if da_chon[u]:
            dem["bo_qua"] += 1

            if lan_vet:
                print(f"Bo qua: dinh {u} da nam trong cay.")

            continue
```

```
        da_chon[u] = True
        dem["nhan_dinh"] += 1

        if cha != 0:
            canh_chon.append((cha, u, w))
            tong += w

            if lan_vet:
                print(f"Chon canh {cha} - {u}, trong so {w}.")
        elif lan_vet:
            print(f"Nhan dinh nguon {u}, khong them canh.")

        dem["day_canh"] += them_ung_vien(
            hang, u, ke, da_chon
        )
```

Nếu da_chon[u] đã bằng True, ứng viên hiện tại không còn được dùng để mở rộng cây. Chương trình tăng bộ đếm bo_qua rồi thực hiện continue. Nhờ đó, ứng viên bị loại không làm thay đổi canh_chon, tong và không khiến danh sách kề của u bị duyệt lại.

Nếu u chưa được chọn, chương trình đánh dấu u trước khi bổ sung ứng viên mới và tăng nhan_dinh. Với cha khác 0, bộ (cha, u, w) biểu diễn một cạnh thật nối cây hiện tại với đỉnh mới u. Cạnh này được thêm vào cuối canh_chon và trọng số w được cộng vào tong. Thứ tự các phần tử trong canh_chon vì vậy chính là thứ tự lựa chọn cạnh của thuật toán.

Trường hợp cha bằng 0 chỉ xuất hiện ở phần tử nguồn (0, s, 0). Chương trình nhận đỉnh bắt đầu nhưng không thêm cạnh giả vào kết quả. Điều này bảo đảm đồ thị một đỉnh có danh sách cạnh rỗng và tổng trọng số bằng 0.

Cuối đoạn mã, hàm them_ung_vien được gọi với trạng thái da_chon đã cập nhật. Số cạnh mới được đẩy vào heap được cộng vào dem["day_canh"]. Các lệnh in được đặt trong điều kiện lan_vet nên chỉ thực hiện khi bật chế độ lần vết.

c) Kiểm tra tính liên thông và trả kết quả

Đoạn mã sau được thực hiện sau khi vòng lặp xử lý heap kết thúc:

```
    if thong_ke is not None:
        thong_ke.update(dem)

    if dem["nhan_dinh"] != n:
        if lan_vet:
            print(f"\nChi nhan duoc {dem['nhan_dinh']}/{n} dinh.")

        return None

    return canh_chon, tong
```

Điều kiện thong_ke is not None cho phép cập nhật thống kê cả khi người gọi truyền vào một từ điển rỗng. Các bộ đếm được ghi trước bước kiểm tra tính liên thông, nên người gọi vẫn nhận được thống kê nếu thuật toán không tìm được cây bao trùm của toàn bộ đồ thị.

Sau khi heap rỗng, nếu nhan_dinh khác n thì vẫn còn đỉnh không thể tiếp cận từ đỉnh bắt đầu. Hàm trả về None để chương trình demo hiển thị KHONG LIEN THONG. Việc kiểm tra này tránh trả về cây của riêng một thành phần liên thông như kết quả hợp lệ cho toàn bộ đồ thị.

Nếu nhan_dinh bằng n, thuật toán đã nhận đủ các đỉnh và trả về canh_chon cùng tong.

Các cạnh được lưu theo thứ tự lựa chọn; danh sách kề đầu vào không bị thay đổi.

Mã nguồn đối chiếu: Tệp src/prim.py tại commit C : https://github.com/PhanVanTinh0209/N24DCDT095_DT080_PhanVanTinh/blob/4ab77e2cdf32c21c43180b53991dac8cebb3d0fc/src/prim.py.

3.3.3. Xử lý tình huống đặc biệt

Ngoài trường hợp đồ thị liên thông thông thường, chương trình cần xử lý các trường hợp biên, trọng số bằng nhau, ứng viên không còn phù hợp và dữ liệu đầu vào không hợp lệ. Các mã TC dưới đây tương ứng với bộ kiểm thử tự động; kết quả thực tế được trình bày ở Chương 4.

a) Đồ thị chỉ có một đỉnh

Với n = 1 và danh sách cạnh rỗng, heap ban đầu chứa phần tử nguồn (0, 1, 0). Thuật toán nhận đỉnh 1 nhưng không thêm cạnh vì cha = 0. Sau đó, heap rỗng và số đỉnh đã nhận bằng n, nên hàm trả về ([], 0). Đây là kết quả hợp lệ, không phải trường hợp không liên thông. Tình huống này được kiểm tra trong TC03.

b) Đồ thị không liên thông

Thuật toán bắt đầu từ đỉnh 1 nên chỉ nhận được các đỉnh thuộc thành phần liên thông chứa đỉnh này. Khi heap rỗng mà dem["nhan_dinh"] nhỏ hơn n, hàm trả về None. Chương trình demo kiểm tra kết quả và in KHONG LIEN THONG.

Ví dụ, đồ thị có n = 4 và các cạnh (1, 2, 1), (3, 4, 2) gồm hai thành phần rời nhau. Thuật toán chỉ nhận được đỉnh 1 và đỉnh 2, nên không trả về các cạnh đã chọn như một cây bao trùm của toàn bộ đồ thị. Tình huống này được kiểm tra trong TC02. Đồ thị có nhiều hơn một đỉnh nhưng không có cạnh cũng được xử lý theo nhánh không liên thông.

c) Đồ thị hai đỉnh với một cạnh

Với n = 2 và một cạnh hợp lệ nối hai đỉnh, cạnh đó là cạnh duy nhất của cây bao trùm nhỏ nhất. Thuật toán nhận đỉnh nguồn, đưa cạnh vào heap rồi nhận đỉnh còn lại. Kết quả có đúng một cạnh và tổng trọng số bằng trọng số của cạnh đó. Tình huống này được kiểm tra trong TC04.

d) Các cạnh có cùng trọng số

Khi nhiều ứng viên có cùng trọng số, heap tiếp tục so sánh số hiệu đỉnh mới rồi đến số hiệu đỉnh cha. Nhờ đó, thứ tự chọn cạnh được xác định thống nhất mà không phụ thuộc riêng vào thứ tự các cạnh được đưa vào heap.

Trong TC05, đồ thị tam giác có ba cạnh cùng trọng số 1. Bắt đầu từ đỉnh 1, thuật toán chọn cạnh (1, 2, 1), sau đó chọn cạnh (1, 3, 1), với tổng trọng số bằng 2. Ca kiểm này đối chiếu cả tổng trọng số và thứ tự cạnh theo quy tắc ưu tiên.

e) Cạnh có trọng số bằng 0 và trọng số ở biên trên

Trọng số bằng 0 thuộc miền dữ liệu hợp lệ. Chương trình phân biệt phần tử nguồn với cạnh thật bằng giá trị cha, không dựa vào việc trọng số có bằng 0 hay không. Vì vậy, cạnh thật có trọng số 0 vẫn được thêm vào canh_chon khi nối tới một đỉnh chưa được chọn. TC06 kiểm tra trường hợp cây bao trùm nhỏ nhất có tổng trọng số bằng 0.

Ở biên trên, TC08 sử dụng đồ thị đường đi gồm 30 đỉnh, với mỗi cạnh có trọng số 1000. Cây bao trùm gồm 29 cạnh và có tổng trọng số 29 × 1000 = 29000. Giới hạn 1000 áp dụng cho từng cạnh, không phải cho tổng trọng số của cây.

f) Ứng viên hướng tới đỉnh đã được chọn

Một đỉnh có thể xuất hiện trong nhiều ứng viên do có nhiều cạnh nối từ cây tới đỉnh đó. Sau khi một ứng viên được chấp nhận, các ứng viên khác hướng tới cùng đỉnh vẫn có thể còn trong heap.

Khi lấy một ứng viên như vậy ra, điều kiện da_chon[u] được kiểm tra trước khi cập nhật kết quả. Nếu đỉnh đã được chọn, chương trình tăng bo_qua và thực hiện continue. Nhánh này ngăn việc nhận đỉnh nhiều lần, cộng thừa trọng số hoặc bổ sung cạnh tạo chu trình.

Trong Ví dụ 1, sau khi chọn các cạnh (1, 3, 1) và (3, 2, 2), ứng viên tương ứng với cạnh 1–2 trọng số 4 vẫn được lấy khỏi heap nhưng bị bỏ qua vì đỉnh 2 đã thuộc cây. Diễn biến này được thể hiện trong phần lần vết và đối chiếu bằng TC01.

g) Dữ liệu không hợp lệ

Hàm kiem_tra_du_lieu phát sinh ValueError khi dữ liệu sai kiểu, sai miền giá trị, sai cấu trúc cạnh, có cạnh tự nối hoặc cạnh trùng vô hướng. Chương trình demo bắt lỗi và hiển thị DU LIEU KHONG HOP LE: kèm nguyên nhân, không tiếp tục chạy Prim trên dữ liệu đó.

Các ca TC09, TC10, TC11 và TC12 lần lượt kiểm tra việc từ chối trọng số âm, cạnh tự nối, cạnh trùng ngược chiều và đỉnh vượt phạm vi. Trọng số âm bị từ chối theo miền dữ liệu quy định trong phiếu DT080.

Ngoài ra, hàm prim kiểm tra đỉnh bắt đầu s và kiểu của tham số thong_ke. Nếu s không phải số nguyên hợp lệ hoặc thong_ke không phải từ điển hay None, hàm phát sinh ValueError. Đây là các kiểm tra bảo vệ khi gọi trực tiếp hàm; chương trình demo sử dụng s = 1 theo đề tài.

3.4. Lần vết ví dụ nhỏ

Ví dụ được chọn để lần vết là Ví dụ 1 trên phiếu DT080, tương ứng với ca kiểm thử TC01 và đồ thị ở Hình 2.1. Dữ liệu gồm n = 3, danh sách cạnh canh = [(1, 2, 4), (1, 3, 1), (2, 3, 2)] và đỉnh bắt đầu s = 1. Ví dụ này đủ nhỏ để kiểm tra bằng tay, đồng thời thể hiện được cả thao tác nhận đỉnh mới và bỏ qua ứng viên có đỉnh đích đã được chọn.

Kết quả kỳ vọng được xác định độc lập trước khi chạy chương trình. Đồ thị tam giác có ba cây bao trùm, tương ứng với ba cách chọn hai cạnh. Tổng trọng số của các cây lần lượt là 4 + 1 = 5, 4 + 2 = 6 và 1 + 2 = 3. Vì vậy, cây bao trùm nhỏ nhất gồm cạnh nối 1 với 3 và cạnh nối 3 với 2, có tổng trọng số bằng 3. Theo thứ tự lựa chọn của Prim từ đỉnh 1, danh sách kết quả kỳ vọng là [(1, 3, 1), (3, 2, 2)].

Trong bảng lần vết, S biểu diễn tập đỉnh đã được nhận vào cây. Các ứng viên trong heap có dạng (w, u, cha), còn cạnh được thêm vào canh_chon có dạng (cha, u, w). Hai cách biểu diễn này cần được phân biệt để tránh nhầm trọng số với số hiệu đỉnh. Khi trình bày trạng thái heap, các ứng viên được liệt kê theo thứ tự ưu tiên tăng dần để dễ đối chiếu; đây không nhất thiết là thứ tự toàn bộ phần tử trong danh sách lưu heap của Python.

Ở bước khởi tạo, S rỗng, canh_chon rỗng, tong = 0 và heap chỉ chứa phần tử nguồn (0, 1, 0). Lần lấy đầu tiên nhận đỉnh 1 nhưng không thêm cạnh. Sau đó, thuật toán lần lượt nhận đỉnh 3 qua cạnh trọng số 1 và nhận đỉnh 2 qua cạnh trọng số 2.

Sau khi đã nhận đủ ba đỉnh, heap vẫn còn ứng viên (4, 2, 1). Chương trình tiếp tục lấy ứng viên này ra rồi bỏ qua vì đỉnh 2 đã thuộc cây. Do đó, ví dụ có bốn lần lấy phần tử khỏi heap, nhưng chỉ nhận ba đỉnh và chọn hai cạnh. Khi heap rỗng, hàm trả về ([(1, 3, 1), (3, 2, 2)], 3).

Bảng 3.3 ghi trạng thái sau mỗi bước xử lý, gồm ứng viên được lấy ra, thao tác thực

hiện và các dữ liệu được cập nhật. Diễn biến này được dùng để đối chiếu với thông tin do chương trình truy_vet.py hiển thị.

Ca truy vết TC01 – Ví dụ 1 trên phiếu DT080. Input đầy đủ n = 3; m = 3; canh = [(1, 2, 4), (1, 3, 1), (2, 3, 2)]; đỉnh bắt đầu s = 1. Expected canh_chon = [(1, 3, 1), (3, 2, 2)]; tong = 3. Ba cây bao trùm có tổng trọng số lần lượt là 5, 6 và 3; giá trị nhỏ nhất là 3.

Bảng 3.3 Truy vết ca TC01

Quy ước: H là heap hang; S là tập đỉnh đã nhận; C là danh sách canh_chon; t là giá trị tong. Các phần tử trong H được trình bày theo thứ tự ưu tiên tăng dần. Ký hiệu ∅ biểu diễn tập rỗng, [] biểu diễn danh sách rỗng.

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
0	Khởi tạo	n = 3; s = 1; canh = [(1, 2, 4), (1, 3, 1), (2, 3, 2)].	H = [(0, 1, 0)]; S = ∅; C = []; t = 0. Các bộ đếm đều bằng 0.	Chưa nhận đỉnh nào. Phần tử nguồn dùng để khởi động thuật toán, không đại diện cho cạnh thật.
1	Lấy (0, 1, 0); nhận đỉnh 1; thêm hai ứng viên.	H = [(0, 1, 0)]; S = ∅; C = []; t = 0.	H = [(1, 3, 1), (4, 2, 1)]; S = {1}; C = []; t = 0.	Đỉnh 1 chưa được chọn nên được nhận. Vì cha = 0, không thêm cạnh. Đưa các ứng viên tới đỉnh 3 và đỉnh 2 vào heap.
2	Lấy (1, 3, 1); chọn cạnh (1, 3, 1); thêm ứng viên tới đỉnh 2.	H = [(1, 3, 1), (4, 2, 1)]; S = {1}; C = []; t = 0.	H = [(2, 2, 3), (4, 2, 1)]; S = {1, 3}; C = [(1, 3, 1)]; t = 1.	Ứng viên có trọng số 1 nhỏ nhất và đỉnh 3 chưa được chọn. Nhận đỉnh 3, cộng 1 vào tổng; thêm (2, 2, 3). Không thêm ứng viên về đỉnh 1 vì đỉnh này đã được chọn.

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
3	Lấy (2, 2, 3); chọn cạnh (3, 2, 2).	H = [(2, 2, 3), (4, 2, 1)]; S = {1, 3}; C = [(1, 3, 1)]; t = 1.	H = [(4, 2, 1)]; S = {1, 2, 3}; C = [(1, 3, 1), (3, 2, 2)]; t = 3.	Trọng số 2 nhỏ hơn 4 và đỉnh 2 chưa được chọn. Nhận đỉnh 2, cộng 2 vào tổng. Các đỉnh kề đều đã được chọn nên không thêm ứng viên.
4	Lấy (4, 2, 1); bỏ qua ứng viên.	H = [(4, 2, 1)]; S = {1, 2, 3}; C = [(1, 3, 1), (3, 2, 2)]; t = 3.	H = []; S = {1, 2, 3}; C = [(1, 3, 1), (3, 2, 2)]; t = 3.	Đỉnh 2 đã thuộc cây nên tăng bộ đếm bo_qua và chuyển sang lượt kiểm tra vòng lặp tiếp theo. Không thêm cạnh, không cộng trọng số. Heap đã rỗng nên vòng lặp kết thúc.
Cuối	Kiểm tra số đỉnh đã nhận và trả kết quả.	H = []; nhan_dinh = 3 = n; C = [(1, 3, 1), (3, 2, 2)]; t = 3.	Trả về ([(1, 3, 1), (3, 2, 2)], 3).	Đã nhận đủ ba đỉnh. Hai cạnh được chọn tạo thành cây bao trùm nhỏ nhất, có thứ tự và tổng trọng số khớp Expected.

Kết thúc lần vết, chương trình đã đẩy 3 ứng viên cạnh vào heap, lấy 4 phần tử khỏi heap gồm 1 phần tử nguồn và 3 ứng viên cạnh, nhận 3 đỉnh và bỏ qua 1 ứng viên. Các bộ đếm tương ứng là day_canh = 3, lay_heap = 4, lay_canh = 3, nhan_dinh = 3 và bo_qua = 1.

Danh sách cạnh trả về là [(1, 3, 1), (3, 2, 2)], với tổng trọng số bằng 3, khớp kết quả kỳ vọng đã tính trước. Bước 4 cho thấy chương trình vẫn xử lý ứng viên còn lại sau khi đã nhận đủ các đỉnh; điều kiện kiểm tra da_chon ngăn cạnh trọng số 4 được thêm vào cây và làm sai tổng trọng số.

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Để giải thích tính đúng của thuật toán, xét đồ thị đầu vào hợp lệ và liên thông. Gọi S là tập đỉnh đã được nhận, C là tập cạnh trong canh_chon. Sau khi nhận đỉnh nguồn và sau

mỗi vòng lặp hoàn chỉnh, thuật toán duy trì các bất biến sau:

– Các cạnh trong C tạo thành một cây trên tập đỉnh S, không có chu trình và có đúng $|S| - 1$ cạnh.

– Giá trị tong bằng tổng trọng số các cạnh trong C.

– Tồn tại ít nhất một cây bao trùm nhỏ nhất của đồ thị chứa toàn bộ các cạnh trong C.

– Heap chứa ứng viên cho mọi cạnh nối từ S tới V \ S. Heap có thể chứa thêm các ứng viên cũ có đỉnh đích đã thuộc S; những ứng viên này sẽ bị bỏ qua khi lấy ra.

a) Điều kiện khởi đầu

Ban đầu, heap chứa phần tử nguồn (0, s, 0), danh sách cạnh rỗng và tong = 0. Trong lần xử lý đầu tiên, thuật toán nhận đỉnh s mà không thêm cạnh vì cha = 0. Khi đó, S = {s}, C rỗng và tong = 0.

Một đỉnh cùng tập cạnh rỗng tạo thành cây có $0 = |S| - 1$ cạnh. Tập cạnh rỗng là tập con của mọi cây bao trùm nhỏ nhất. Sau khi gọi them_ung_vien, toàn bộ các cạnh nối từ s tới những đỉnh chưa được nhận đều có ứng viên trong heap. Vì vậy, các bất biến được thiết lập sau bước nhận đỉnh nguồn.

b) Duy trì khi bỏ qua ứng viên

Nếu ứng viên được lấy ra có đỉnh đích u đã thuộc S, chương trình thực hiện continue. Tập S, tập cạnh C và tong đều không thay đổi, nên tính chất cây, tổng trọng số và khả năng mở rộng thành cây bao trùm nhỏ nhất được giữ nguyên.

Ứng viên bị bỏ qua có cả đỉnh cha và đỉnh đích nằm trong S, nên không còn đại diện cho cạnh nối S với V \ S. Loại bỏ ứng viên này không làm mất ứng viên của bất kỳ cạnh nào đang nối cây ra bên ngoài.

c) Duy trì khi nhận một đỉnh mới

Nếu u chưa thuộc S, ứng viên được chấp nhận biểu diễn cạnh e = (cha, u), trong đó cha đã thuộc cây. Do heap ưu tiên trọng số nhỏ nhất và các ứng viên cũ bị bỏ qua, e là một cạnh có trọng số nhỏ nhất nối S với V \ S tại thời điểm lựa chọn.

Việc thêm e đưa đúng một đỉnh mới vào cây. Vì u chưa thuộc S, cạnh e không tạo chu trình. Số đỉnh và số cạnh đều tăng thêm một, nên quan hệ |C| = |S| − 1 tiếp tục đúng. Đồng thời, chương trình cộng trọng số của e vào tong, bảo đảm tong vẫn bằng tổng trọng số các cạnh đã chọn.

Để chứng minh tính tối ưu được duy trì, giả sử T là một cây bao trùm nhỏ nhất chứa C trước bước cập nhật. Nếu e đã thuộc T, thì C cùng cạnh e vẫn nằm trong T và bất biến được giữ nguyên.

Nếu e chưa thuộc T, thêm e vào T tạo ra một chu trình. Trên chu trình này tồn tại một cạnh khác f nối S với V \ S. Vì e là cạnh nhẹ nhất nối hai tập đỉnh đó, ta có w(e) ≤ w(f). Cạnh f không thuộc C vì mọi cạnh trong C đều có hai đầu mút thuộc S.

Thay f bằng e tạo ra một cây bao trùm T′ vẫn chứa C cùng cạnh e và có tổng trọng số không lớn hơn T. Do T đã là cây bao trùm nhỏ nhất, T′ cũng là cây bao trùm nhỏ nhất. Như vậy, các cạnh được chọn sau bước cập nhật vẫn có thể mở rộng thành một cây bao trùm nhỏ nhất.

Sau khi nhận u, hàm them_ung_vien bổ sung các cạnh nối từ u tới những đỉnh chưa được chọn. Các ứng viên của những cạnh vẫn nối cây ra bên ngoài được giữ lại trong heap. Vì vậy, bất biến về các ứng viên cũng tiếp tục được duy trì.

d) Trường hợp các trọng số bằng nhau

Lập luận trên chỉ cần w(e) ≤ w(f), không yêu cầu trọng số các cạnh phải khác nhau. Vì

vậy, khi có nhiều cạnh nhẹ nhất cùng trọng số, việc ưu tiên số hiệu đỉnh mới rồi đến đỉnh cha vẫn bảo đảm tính tối ưu. Quy tắc này xác định thứ tự lựa chọn thống nhất trong số các phương án có cùng trọng số.

3.5.2. Kết thúc và output

a) Thuật toán luôn kết thúc

Mỗi vòng lặp lấy đúng một phần tử ra khỏi heap. Một đỉnh chỉ được nhận khi da_chon của đỉnh đó bằng False; sau khi nhận, giá trị này được đặt thành True và không đổi lại. Vì vậy, mỗi đỉnh được nhận nhiều nhất một lần và danh sách kề của nó cũng chỉ được duyệt một lần.

Với một cạnh vô hướng, khi đầu mút thứ nhất được nhận, cạnh có thể được đưa vào heap nếu đầu mút còn lại chưa được chọn. Khi đầu mút thứ hai được nhận, đầu mút thứ nhất đã thuộc cây nên cạnh không được đẩy thêm theo chiều ngược lại. Do đó, mỗi cạnh được đưa vào heap nhiều nhất một lần.

Toàn bộ quá trình có nhiều nhất m ứng viên cạnh được đẩy vào heap, cộng với một phần tử nguồn được khởi tạo ban đầu. Vì mỗi vòng lặp lấy ra một phần tử, số lần lặp không vượt quá m + 1. Số phần tử trong heap không nhất thiết giảm ở mọi bước vì có thể phát sinh ứng viên mới, nhưng tổng số phần tử được đưa vào là hữu hạn. Vì vậy, heap cuối cùng sẽ rỗng và thuật toán kết thúc.

b) Kết quả đối với đồ thị liên thông

Giả sử heap đã rỗng nhưng vẫn còn đỉnh chưa được nhận. Vì đồ thị liên thông và tập S đã chứa đỉnh nguồn, phải tồn tại ít nhất một cạnh nối S với V \ S. Theo bất biến về heap, cạnh này phải có ứng viên trong hàng đợi, mâu thuẫn với việc heap đã rỗng. Do đó, khi kết thúc trên đồ thị liên thông, thuật toán đã nhận đủ n đỉnh.

Theo bất biến đã chứng minh ở mục 3.5.1, các cạnh trong canh_chon tạo thành cây trên tập đỉnh đã nhận và có thể được mở rộng thành một cây bao trùm nhỏ nhất. Khi S = V, danh sách này đã có đúng n − 1 cạnh và kết nối toàn bộ đồ thị. Vì một cây bao

trùm nhỏ nhất cũng chỉ có n − 1 cạnh, canh_chon chính là tập cạnh của một cây bao trùm nhỏ nhất.

Giá trị tong luôn bằng tổng trọng số các cạnh trong canh_chon. Vì vậy, cặp kết quả (canh_chon, tong) thỏa mãn yêu cầu về cấu trúc cây, tổng trọng số nhỏ nhất và thứ tự cạnh được lựa chọn.

c) Kết quả đối với đồ thị không liên thông

Thuật toán chỉ bổ sung đỉnh mới thông qua cạnh nối từ một đỉnh đã được nhận. Do đó, mọi đỉnh được nhận đều thuộc thành phần liên thông chứa đỉnh nguồn s.

Ngược lại, nếu còn một đỉnh trong thành phần này chưa được nhận, trên đường đi từ s đến đỉnh đó phải có một cạnh nối S với một đỉnh chưa được chọn. Cạnh này phải có ứng viên trong heap, nên heap chưa thể rỗng. Vì vậy, khi kết thúc, tập S chính xác là tập đỉnh của thành phần liên thông chứa s.

Nếu đồ thị không liên thông, số đỉnh trong thành phần đó nhỏ hơn n. Điều kiện dem["nhan_dinh"] != n được thỏa mãn và hàm trả về None. Chương trình demo hiển thị KHONG LIEN THONG, không công nhận các cạnh trong một thành phần riêng lẻ là cây bao trùm của toàn bộ đồ thị.

d) Trường hợp một đỉnh và vai trò của kiểm thử

Với n = 1 và không có cạnh, thuật toán nhận đỉnh nguồn rồi kết thúc vì heap rỗng. Số đỉnh đã nhận bằng n, danh sách cạnh rỗng và tổng trọng số bằng 0. Kết quả ([], 0) phù hợp với cây bao trùm gồm một đỉnh.

Lập luận về bất biến và điều kiện kết thúc xác lập tính đúng của thuật toán trên dữ liệu hợp lệ. Bộ kiểm thử và phần lần vết bổ sung bằng chứng rằng bản cài đặt thực hiện đúng các bước đã thiết kế, nhưng không thay thế cho lập luận tính đúng.

3.6. Phân tích độ phức tạp

Gọi n là số đỉnh và m là số cạnh của đồ thị. Phân tích được thực hiện theo các thao tác trong bản cài đặt Python: kiểm tra dữ liệu, tạo danh sách kề, duyệt cạnh kề và thao tác trên heap. Các phép kiểm tra kiểu, so sánh số nguyên và truy cập phần tử danh sách được xem là O(1); thao tác thêm cuối danh sách có chi phí khấu hao O(1). Với tập hợp set, thao tác tra cứu và thêm phần tử được đánh giá theo chi phí trung bình O(1).

a) Kiểm tra dữ liệu và tạo danh sách kề

Hàm kiem_tra_du_lieu duyệt m cạnh. Với mỗi cạnh, hàm kiểm tra cấu trúc, kiểu dữ liệu, miền giá trị, cạnh tự nối và cạnh trùng. Theo giả định về set, thời gian kiểm tra dữ liệu trung bình là O(m + 1). Tập da_gap lưu tối đa m cặp đỉnh nên sử dụng bộ nhớ O(m).

Sau khi dữ liệu hợp lệ, việc tạo danh sách kề gồm khởi tạo n + 1 danh sách con và thêm hai phần tử cho mỗi cạnh vô hướng. Riêng bước xây dựng này có thời gian O(n + m) và bộ nhớ O(n + m). Tính cả bước kiểm tra dữ liệu, hàm tao_danh_sach_ke có thời gian trung bình O(n + m).

b) Khởi tạo và duyệt danh sách kề trong Prim

Hàm prim khởi tạo mảng da_chon gồm n + 1 phần tử, nên cần thời gian và bộ nhớ O(n). Các biến tổng, danh sách kết quả rỗng, phần tử nguồn và từ điển có năm bộ đếm được khởi tạo với chi phí O(1).

Mỗi đỉnh được nhận nhiều nhất một lần. Hàm them_ung_vien chỉ được gọi khi một đỉnh vừa được nhận, nên mỗi danh sách kề được duyệt nhiều nhất một lần. Vì mỗi cạnh vô hướng xuất hiện hai lần trong danh sách kề, tổng số phần tử kề được xét không vượt quá 2m. Chi phí duyệt và kiểm tra trạng thái đỉnh, chưa tính thao tác heap, là O(m).

c) Chi phí thao tác heap và thời gian của phần lõi

Mỗi cạnh được đẩy vào heap nhiều nhất một lần. Ngoài các ứng viên cạnh, heap có một phần tử nguồn. Vì vậy, số lần đẩy cạnh không vượt quá m, số lần lấy phần tử không vượt quá m + 1 và kích thước heap là O(m + 1).

Một lần đẩy hoặc lấy phần tử trên heap có chi phí O(log(m + 1)) khi xét các ứng viên cạnh; thao tác với heap chỉ chứa phần tử nguồn có chi phí O(1). Kết hợp với khởi tạo và duyệt danh sách kề, thời gian của hàm prim khi tắt lần vết được chặn trên bởi:

$T(n, m) = O(n + m \log(m + 1))$.

Với đồ thị đơn vô hướng, $m \leq n(n - 1)/2$. Khi $n \geq 2$, có thể viết giới hạn trên dưới dạng $O(n + m \log n)$. Nếu đồ thị liên thông thì $m \geq n - 1$, nên thời gian được rút gọn thành $O(m \log n)$. Trường hợp $n = 1$ và $m = 0$ có thời gian $O(1)$.

Các thao tác tăng bộ đếm có chi phí O(1) mỗi lần và không làm thay đổi bậc độ phức tạp. Bản cài đặt xử lý đến khi heap rỗng nên vẫn phải tính chi phí lấy và bỏ qua các ứng viên còn lại sau khi đã nhận đủ các đỉnh.

d) Bộ nhớ sử dụng

Danh sách cạnh đầu vào chiếm O(m), còn danh sách kề chiếm O(n + m). Trong hàm prim, mảng da_chon chiếm O(n), heap hang chiếm O(m + 1), danh sách kết quả canh_chon chứa tối đa $n - 1$ cạnh nên chiếm O(n). Từ điển dem có số khóa cố định và các biến đơn chiếm O(1).

Do đó, bộ nhớ phụ của hàm prim có giới hạn O(n + m). Nếu tính cả dữ liệu đầu vào, danh sách kề và kết quả, tổng bộ nhớ của chương trình vẫn là O(n + m). Hàm không sử dụng đệ quy nên không phát sinh ngăn xếp lời gọi tăng theo số đỉnh.

e) Chi phí hiển thị và chế độ lần vết

Kết quả thành công chứa n − 1 cạnh, nên việc duyệt và hiển thị danh sách cạnh có O(n) phần tử. Chương trình demo còn hiển thị danh sách m cạnh đầu vào. Các chi phí nhập và xuất được xem xét riêng với phần lõi của Prim.

Khi bật lan_vet, sau mỗi lần nhận đỉnh, chương trình duyệt các chỉ số từ 1 đến n để tạo tap_dinh. Có nhiều nhất n lần nhận đỉnh, nên riêng việc tạo các danh sách này có chi phí $O(n^2)$. Chương trình còn in thông tin trong các lần lấy phần tử khỏi heap. Vì vậy, không dùng trực tiếp độ phức tạp của chế độ không lần vết để mô tả toàn bộ thời gian chạy có hiển thị chi tiết. Thời gian quan sát trên màn hình còn phụ thuộc vào môi trường xuất dữ liệu.

f) Dự đoán xu hướng khảo sát

Với đồ thị liên thông, mọi đỉnh đều được nhận và mỗi cạnh được đẩy vào heap đúng một lần. Do heap được xử lý đến rỗng, các bộ đếm thỏa mãn: day_canh = m, lay_canh = m, lay_heap = m + 1, nhan_dinh = n và bo_qua = m − n + 1.

Đồ thị đường đi có m = n − 1, nên số ứng viên cạnh tăng tuyến tính theo n và không có lượt bỏ qua. Đồ thị đầy đủ có m = n(n − 1)/2, nên số ứng viên và số lượt lấy heap tăng theo bậc hai của n; số lượt bỏ qua bằng (n − 1)(n − 2)/2.

Vì vậy, tại cùng số đỉnh, đồ thị đầy đủ dự kiến cần nhiều thao tác heap và nhiều lượt bỏ qua hơn đồ thị đường đi. Khảo sát với n = 5, 10, 20 ở Chương 4 dùng để đối chiếu các dự đoán này. Số thao tác đếm được không phải số phép so sánh bên trong heap và không thay thế trực tiếp cho phép đo thời gian chạy.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Kiểm tra và chuẩn bị	n đỉnh, m cạnh.	Kiểm tra m cạnh; tạo n + 1 danh sách con và lưu 2m phần tử kề.	Trung bình O(n + m), với thao tác tra cứu và thêm vào set có chi phí trung bình O(1).	O(n + m) cho tập kiểm tra cạnh trùng và danh sách kề được tạo.
Thuật toán chính	n đỉnh, m cạnh.	Nhận tối đa n đỉnh; xét tối đa 2m phần tử kề; đẩy tối đa m ứng viên cạnh và lấy tối đa m + 1 phần tử khỏi heap.	O(n + m log(m + 1)). Với đồ thị liên thông và n ≥ 2: O(m log n).	O(n + m) cho mảng đánh dấu, heap và danh sách cạnh kết quả.
Kiểm nghiệm hoặc truy vết nếu có	n, m; xét chế độ lan_vet = True.	Quét n vị trí sau mỗi lần nhận đỉnh để tạo tap_dinh; in thông tin trong tối đa m + 1 lần lấy heap.	Chi phí bổ sung do lần vết: O(n^2 + m), theo số phần tử xử lý và hiển thị.	O(n) bổ sung cho tap_dinh và chuỗi hiển thị trạng thái; không lưu toàn bộ lịch sử lần vết.
In và định dạng kết quả	m cạnh đầu vào; tối đa n − 1 cạnh kết quả.	Định dạng và hiển thị danh sách cạnh đầu vào, danh sách cạnh được chọn, tổng trọng số hoặc thông báo.	O(m + n), theo số phần tử được định dạng; không tính độ trễ thiết bị xuất.	O(m + n) cho các chuỗi định dạng danh sách trong chương trình demo.

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Toàn chương trình	Một đồ thị có n đỉnh, m cạnh; không bật lần vết.	Tổng hợp kiểm tra, tạo danh sách kề, chạy Prim và hiển thị dữ liệu, kết quả.	Trung bình $O(n + m \log(m + 1))$ theo giả định về set. Với đồ thị liên thông và $n \geq 2$: $O(m \log n)$.	$O(n + m)$; tính cả dữ liệu đầu vào và kết quả vẫn cùng bậc.

Bảng 3.4 đánh giá chi phí cho một lần xử lý đồ thị. Dòng về lần vết chỉ thể hiện chi phí bổ sung khi bật lan_vet; chi phí này không được cộng vào dòng toàn chương trình ở chế độ không lần vết. Việc chạy toàn bộ bộ kiểm thử gồm nhiều lần gọi hàm và các phép đối chiếu riêng, không nằm trong phạm vi một lần xử lý được xét ở bảng.

Đánh giá trung bình của bước kiểm tra dữ liệu dựa trên thao tác tra cứu và thêm phần tử vào set. Giới hạn thời gian của phần lõi Prim được xác định từ số lần duyệt danh sách kề và thao tác heap. Các biểu thức không bao gồm độ trễ chờ người dùng nhập hoặc độ trễ của thiết bị xuất.

Danh sách kết quả có tối đa $n - 1$ cạnh nên chiếm $O(n)$ bộ nhớ và đã được tính trong bảng. Các vùng nhớ tạm không được cộng dồn qua mọi bước; bộ nhớ được đánh giá theo lượng dữ liệu phải lưu đồng thời. Với $n = 1$ và $m = 0$, chương trình xử lý trong thời gian và bộ nhớ $O(1)$.

Các kết quả khảo sát trên ba quy mô nhỏ được sử dụng để đối chiếu số thao tác dự đoán, không được dùng riêng lẻ để chứng minh độ phức tạp tiệm cận.

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Chương trình được thực hiện bằng Python và chạy trên hệ điều hành Windows thông qua Windows PowerShell. Git được sử dụng để quản lý phiên bản mã nguồn; GitHub lưu trữ kho mã để có thể tải lại và kiểm tra khả năng tái hiện kết quả.

Thông tin môi trường đã được kiểm tra bằng các lệnh python --version và git --version:

– Hệ điều hành: Windows.

– Môi trường thực thi lệnh: Windows PowerShell.

– Phiên bản Python: Python 3.14.7.

– Phiên bản Git: git version 2.55.0.windows.4.

– Thư mục dự án: E:\N24DCDT095_DT080_PhanVanTinh.

– Thư mục clone dùng để kiểm tra lại: E:\DT080_KiemTra_C.

Phần cài đặt Prim sử dụng heappush và heappop từ thư viện chuẩn heapq để thao tác với heap cực tiểu. Chương trình demo sử dụng sys để đọc tham số dòng lệnh. Chương trình không sử dụng thư viện đồ thị để tính sẵn cây bao trùm nhỏ nhất.

Mã nguồn được tổ chức trong thư mục src, gồm các tệp kiem_tra.py, prim.py và kiem_cay.py. Tệp demo.py dùng để chạy các ví dụ và nhập đồ thị từ bàn phím; test/runtests.py chứa bộ kiểm thử tự động; truy_vet.py phục vụ lần vết; khao_sat.py phục vụ khảo sát số thao tác.

Kho GitHub của đề tài:

https://github.com/PhanVanTinh0209/N24DCDT095_DT080_PhanVanTinh

Phiên bản mã nguồn dùng để đối chiếu kết quả thử nghiệm được ký hiệu là commit C, có mã SHA đầy đủ:

4ab77e2cdf32c21c43180b53991dac8cebb3d0fc

Nội dung commit: Hoan thanh Prim, kiem thu, truy vet va khao sat DT080.

Để kiểm tra khả năng tái hiện, kho mã đã được clone vào thư mục E:\DT080_KiemTra_C và chuyển về đúng commit C bằng lệnh git checkout --detach. Lệnh git rev-parse HEAD xác nhận mã SHA của bản chạy trùng với commit đã chọn. Sau khi chạy demo, bộ kiểm thử và chương trình khảo sát, git status báo working tree clean, cho thấy các lệnh chạy không làm thay đổi những tệp được Git theo dõi.

Các lệnh thực thi được chạy tại thư mục gốc của dự án hoặc thư mục clone:

```
python demo.py
python demo.py --nhap
python -m test.runtests
python truy_vet.py
python khao_sat.py
```

Trong đó, python demo.py chạy các ví dụ có sẵn; tham số --nhap cho phép nhập đồ thị từ bàn phím. Các lệnh còn lại lần lượt chạy bộ kiểm thử, lần vết và khảo sát số thao tác.

Commit C được giữ làm mốc đối chiếu giữa mã nguồn, kết quả và ảnh minh chứng trong báo cáo. Sau khi hoàn thiện hồ sơ, mã SHA của phiên bản nộp cuối S được ghi

trong tệp ThongTinNop.txt nộp kèm. Việc chốt phiên bản S và kiểm tra chạy lại được thực hiện ở bước hoàn thiện hồ sơ.

4.1.2. Phương pháp xác định kỳ vọng

Kết quả kỳ vọng được xác định bằng tính toán thủ công, tính chất của cây bao trùm và các điều kiện dữ liệu trên phiếu DT080. Các giá trị này được khai báo trong danh sách CAC_CA của tệp test/runtests.py, không được tạo bằng cách gọi chính hàm prim đang kiểm thử. Việc đối chiếu gồm kiểm tra kết quả trả về và các điều kiện cấu trúc của cây.

a) Xác định kỳ vọng cho các ca có cây bao trùm

Với TC01, đồ thị tam giác có ba cách chọn hai cạnh tạo thành cây bao trùm. Tổng trọng số tương ứng là 4 + 1 = 5, 4 + 2 = 6 và 1 + 2 = 3. Vì vậy, tổng nhỏ nhất là 3; thứ tự cạnh kỳ vọng khi bắt đầu từ đỉnh 1 là [(1, 3, 1), (3, 2, 2)].

Với TC03, đồ thị chỉ có một đỉnh nên cây bao trùm không có cạnh và tổng trọng số bằng 0. Với TC04, đồ thị hai đỉnh chỉ có một cạnh trọng số 7 nên cạnh này bắt buộc được chọn, tổng bằng 7. Với TC08, đồ thị là đường đi gồm 30 đỉnh và 29 cạnh trọng số 1000; tất cả các cạnh đều cần thiết để giữ liên thông, nên tổng kỳ vọng là 29 × 1000 = 29000.

Với TC05, đồ thị tam giác có tất cả các cạnh trọng số 1. Mỗi cây bao trùm có hai cạnh nên tổng trọng số nhỏ nhất bằng 2. Áp dụng quy tắc ưu tiên (trọng số, đỉnh mới, đỉnh cha), danh sách cạnh kỳ vọng là [(1, 2, 1), (1, 3, 1)].

Với TC06, hai cạnh trọng số 0 nối đủ ba đỉnh thành cây. Vì mọi trọng số đầu vào đều không âm, không thể có cây bao trùm với tổng nhỏ hơn 0. Do đó, danh sách [(1, 2, 0), (2, 3, 0)] có tổng bằng 0 là kết quả tối ưu.

Với TC07, cây bao trùm của đồ thị bốn đỉnh phải có ba cạnh. Ba trọng số nhỏ nhất trong dữ liệu là 1, 2 và 3, có tổng bằng 6. Ba cạnh tương ứng là (1, 2, 1), (2, 3, 2), (3, 4, 3), đồng thời tạo thành cây nối đủ bốn đỉnh. Vì vậy, tổng nhỏ nhất bằng 6 và thứ tự cạnh kỳ vọng từ đỉnh 1 là thứ tự của đường đi này.

b) Xác định kỳ vọng cho đồ thị không liên thông và dữ liệu sai

TC02 gồm hai thành phần đỉnh {1, 2} và {3, 4}, không có cạnh nối giữa chúng. Vì không tồn tại cây bao trùm chứa cả bốn đỉnh, kết quả kỳ vọng của hàm prim là None; chương trình demo phải hiển thị KHONG LIEN THONG.

Các ca TC09, TC10, TC11 và TC12 lần lượt vi phạm điều kiện về trọng số không âm, không có cạnh tự nối, không có cạnh trùng vô hướng và số hiệu đỉnh nằm trong phạm vi hợp lệ. Kết quả kỳ vọng là phát sinh ngoại lệ ValueError. Bộ kiểm thử đối chiếu

loại ngoại lệ; nội dung chi tiết của thông báo không được dùng làm điều kiện so sánh chuỗi tự động.

c) Cách so sánh kết quả thực tế

Mỗi ca kiểm thử đều đối chiếu kết quả thực tế với giá trị kỳ vọng đã khai báo. Với các ca trả về cây, phép so sánh kiểm tra cả danh sách cạnh theo đúng thứ tự và tổng trọng số. Khi có nhiều cây bao trùm nhỏ nhất, bộ kiểm thử vẫn yêu cầu kết quả tuân thủ quy tắc ưu tiên đã quy định trong đề tài.

Ngoài phép so sánh trực tiếp, mỗi ca kỳ vọng có cây được kiểm tra thêm ba điều kiện:

– Hàm kiem_cay xác nhận các cạnh tạo thành cây trên tập đỉnh từ 1 đến n.

– Mỗi cạnh được chọn, bao gồm cả trọng số, phải thuộc đồ thị đầu vào. Hai đầu mút được chuẩn hóa thành (min(u, v), max(u, v), w) để đối chiếu cạnh vô hướng.

– Giá trị tong phải bằng tổng trọng số của các cạnh trong canh_chon.

Một ca chỉ được đánh dấu PASS khi tất cả các phép kiểm tra của ca đó đều đạt. Hàm kiem_cay chỉ xác nhận cấu trúc cây; tính tối ưu được đối chiếu với kết quả kỳ vọng đã tính độc lập.

d) Số ca kiểm thử và quy tắc đánh giá

Bộ kiểm thử gồm 12 ca độc lập. Trong đó, 7 ca kỳ vọng trả về cây, mỗi ca có 4 phép kiểm tra; 5 ca còn lại gồm một ca không liên thông và bốn ca dữ liệu sai, mỗi ca có 1 phép kiểm tra. Tổng số phép kiểm tra là $7 \times 4 + 5 \times 1 = 33$. Số phép kiểm tra được báo riêng, không được tính thành 33 ca kiểm thử độc lập.

Toàn bộ trọng số và tổng trọng số trong đề tài là số nguyên nên kết quả được so sánh chính xác, không sử dụng ngưỡng sai số số thực.

Đối với phần khảo sát, các cạnh liên tiếp có trọng số 1 và các cạnh bổ sung có trọng số 10. Mỗi cây bao trùm cần $n - 1$ cạnh và mỗi cạnh có trọng số ít nhất bằng 1, nên tổng không thể nhỏ hơn $n - 1$. Các cạnh liên tiếp tạo thành một đường đi bao trùm có đúng tổng này; vì vậy, tổng kỳ vọng của các đồ thị khảo sát là $n - 1$.

4.2. Bảng kết quả ít nhất 12 ca

Bộ kiểm thử gồm 12 ca độc lập, bao phủ hai ví dụ, toàn bộ năm ca bắt buộc trên phiếu DT080 và các trường hợp bổ sung về trọng số bằng 0, đồ thị tự thiết kế, biên dữ liệu

và dữ liệu không hợp lệ. Các ca có kết quả kỳ vọng được xác định trước và được kiểm tra bằng chương trình test/runtests.py.

Báo cáo sử dụng các mã VD, TK, BIEN, LOI và KT theo bố cục mẫu; bộ kiểm thử tự động sử dụng các mã TC01 đến TC12. Mỗi ca trong báo cáo được ghi kèm mã TC tương ứng để đối chiếu với mã nguồn và ảnh kết quả. Hai hệ mã cùng mô tả một bộ gồm 12 ca, không được tính thành hai bộ kiểm thử riêng.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Ca trên phiếu	**Input và yêu cầu/kỳ vọng theo phiếu**	**Mã ca trong bảng kiểm thử**	**Ảnh/log hoặc vị trí kết quả**
Ca phiếu 1	n = 1; canh = []. Kết quả: danh sách cạnh rỗng, tổng trọng số bằng 0.	BIEN01 – TC03	Ảnh A05; mục 4.5.1; dòng TC03 trong ảnh tổng kết A09.
Ca phiếu 2	n = 2; canh = [(1, 2, 7)]. Kết quả: chọn cạnh (1, 2, 7), tổng trọng số bằng 7.	BIEN02 – TC04	Ảnh A06; mục 4.5.2; dòng TC04 trong ảnh tổng kết A09.
Ca phiếu 3	n = 3; canh = [(1, 2, 4), (1, 3, 1), (2, 3, 2)]. Kết quả: chon = [(1, 3, 1), (3, 2, 2)]; tong = 3.	VD01 – TC01	Ảnh A01; mục 4.3.1; bảng lần vết 3.3; dòng TC01 trong ảnh A09.
Ca phiếu 4	n = 3; canh = [(1, 2, 1), (1, 3, 1), (2, 3, 1)]. Kết quả: hai cạnh, tổng bằng 2; theo quy tắc ưu tiên, chọn [(1, 2, 1), (1, 3, 1)].	KT09 – TC05	Dòng TC05 trong ảnh tổng kết A09; dữ liệu và kỳ vọng tại test/runtests.py ở commit C.
Ca phiếu 5	n = 4; canh = [(1, 2, 1), (3, 4, 2)]. Hai thành phần rời nhau; hàm trả về None, chương trình in KHONG LIEN THONG.	VD02 – TC02	Ảnh A02; mục 4.3.2; dòng TC02 trong ảnh tổng kết A09.

Ca phiếu 3 sử dụng cùng dữ liệu và mục tiêu với Ví dụ 1; ca phiếu 5 sử dụng cùng dữ liệu và mục tiêu với Ví dụ 2. Vì vậy, mỗi trường hợp này chỉ được tính là một ca kiểm thử độc lập, dù đồng thời đáp ứng cả yêu cầu ví dụ và yêu cầu ca bắt buộc.

Các ca bổ sung gồm kiểm tra cạnh trọng số 0, đồ thị bốn đỉnh tự thiết kế, biên 30 đỉnh với trọng số 1000 và bốn trường hợp dữ liệu sai. Các ca dữ liệu sai kiểm tra lớp xác thực đầu vào, với kỳ vọng phát sinh ValueError. Tổng cộng bộ kiểm thử có 12 ca độc lập và 33 phép kiểm tra.

Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
VD01 (TC01)	Ví dụ 1 phiếu / kiểm tra Prim trên đồ thị liên thông có các trọng số khác nhau.	n = 3; s = 1; canh = [(1,2,4), (1,3,1), (2,3,2)]	([(1,3,1), (3,2,2)], 3). Ba cây bao trùm có tổng trọng số lần lượt là 5, 6 và 3; do đó tổng nhỏ nhất là 3. Kiểm tra chính xác danh sách cạnh và tổng trả về.
VD02 (TC02)	Ví dụ 2 phiếu / kiểm tra phát hiện đồ thị không liên thông.	n = 4; s = 1; canh = [(1,2,1), (3,4,2)].	Hàm prim trả về None; chương trình demo thông báo KHONG LIEN THONG. Hai thành phần {1,2} và {3,4} không có cạnh nối nên không tồn tại cây bao trùm.
TK01 (TC06)	Tự thiết kế 1 / kiểm tra xử lý cạnh có trọng số bằng 0.	n = 3; s = 1; canh = [(1,2,0), (1,3,5), (2,3,0)].	([(1,2,0), (2,3,0)], 0). Hai cạnh trọng số 0 nối đủ ba đỉnh. Vì mọi trọng số đều không âm nên tổng 0 là nhỏ nhất.

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
TK02 (TC07)	Tự thiết kế 2 / kiểm tra lựa chọn cạnh trên đồ thị bốn đỉnh.	n = 4; s = 1; canh = [(1,2,1), (1,3,4), (2,3,2), (2,4,5), (3,4,3)].	([(1,2,1), (2,3,2), (3,4,3)], 6). Ba cạnh có trọng số nhỏ nhất là 1, 2, 3 và tạo thành cây bao trùm; vì vậy tổng nhỏ nhất là 6.
BIEN01 (TC03)	Biên 1 / đồ thị chỉ có một đỉnh và không có cạnh.	n = 1; s = 1; canh = [].	([], 0). Cây bao trùm của đồ thị một đỉnh có 0 cạnh và tổng trọng số bằng 0.
BIEN02 (TC04)	Biên 2 / đồ thị hai đỉnh có đúng một cạnh.	n = 2; s = 1; canh = [(1,2,7)]	([(1,2,7)], 7). Cạnh duy nhất bắt buộc thuộc cây bao trùm nên tổng bằng 7.
BIEN03 (TC08)	Biên 3 / kiểm tra 30 đỉnh và trọng số 1000.	n = 30; s = 1; canh = [(i,i+1,1000) for i in range(1,30)]. Đây là 29 cạnh liên tiếp từ đỉnh 1 đến đỉnh 30.	Danh sách cạnh được chọn là [(i,i+1,1000) for i in range(1,30)], tổng bằng 29000. Đồ thị đầu vào là một đường đi nên phải chọn đủ 29 cạnh; tổng là 29 × 1000.
LOI01 (TC09)	Input sai / kiểm tra bộ kiểm tra đầu vào với trọng số âm.	n = 3; s = 1; canh = [(1,2,-5), (2,3,2)]. Lý do sai: Cạnh (1, 2, −5) có trọng số âm, vi phạm yêu cầu trọng số là số nguyên trong đoạn [0, 1000].	Phát sinh ValueError khi tạo danh sách kề, trước khi chạy Prim. Trọng số −5 nằm ngoài miền [0,1000].

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
KT09 (TC05)	Ca phiếu 4 / kiểm tra thứ tự ưu tiên của heap khi các cạnh cùng trọng số	n = 3; s = 1; canh = [(1,2,1), (1,3,1), (2,3,1)].	([(1,2,1), (1,3,1)], 2). Mọi cây bao trùm đều có tổng 2. Thứ tự ưu tiên (trọng số, đỉnh mới, đỉnh cha) xác định chính xác danh sách cạnh được chọn như trên.
KT10 (TC10)	Ca bổ sung / kiểm tra từ chối cạnh tự nối.	n = 2; s = 1; canh = [(1,1,5)].	Phát sinh ValueError khi tạo danh sách kề. Cạnh (1,1,5) có hai đầu trùng nhau, vi phạm yêu cầu đồ thị đơn.
KT11 (TC11)	Ca bổ sung / kiểm tra phát hiện cạnh trùng theo chiều ngược lại.	n = 2; s = 1; canh = [(1,2,4), (2,1,7)].	Phát sinh ValueError khi tạo danh sách kề. Trong đồ thị vô hướng, hai cạnh có cùng cặp đỉnh {1,2} là trùng nhau, dù trọng số khác nhau.
KT12 (TC12)	Ca bổ sung / kiểm tra đỉnh nằm ngoài phạm vi cho phép.	n = 3; s = 1; canh = [(1,4,2)]	Phát sinh ValueError khi tạo danh sách kề. Đỉnh 4 nằm ngoài tập đỉnh {1,2,3}.

Bảng 4.3 Actual kết luận và vị trí minh chứng

Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01 (TC01)	chon = [(1,3,1), (3,2,2)]; tong = 3	PASS — Đúng danh sách cạnh, thứ tự chọn và tổng kỳ vọng; đạt 4 phép kiểm.	A01_ViDu1.png; A09_KiemThu_12Ca.png, dòng TC01. SHA C: 4ab77e2
VD02 (TC02)	Hàm trả về None; demo hiển thị KHONG LIEN THONG.	PASS — Phát hiện đúng đồ thị không liên thông; đạt 1 phép kiểm.	A02_ViDu2.png; A09_KiemThu_12Ca.png, dòng TC02. SHA C: 4ab77e2.
TK01 (TC06)	chon = [(1,2,0), (2,3,0)]; tong = 0	PASS — Xử lý đúng cạnh trọng số 0; đạt 4 phép kiểm	A03_TuThietKe_TrongSo0.png; A09_KiemThu_12Ca.png, dòng TC06. SHA C: 4ab77e2.
TK02 (TC07)	chon = [(1,2,1), (2,3,2), (3,4,3)]; tong = 6	PASS — Cây và tổng trọng số đúng kỳ vọng; đạt 4 phép kiểm.	A04_TuThietKe_BonDinh.png; A09_KiemThu_12Ca.png, dòng TC07. SHA C: 4ab77e2.
BIEN01 (TC03)	chon = []; tong = 0	PASS — Đúng trường hợp một đỉnh, không có cạnh; đạt 4 phép kiểm.	A05_Bien_MotDinh.png; A09_KiemThu_12Ca.png, dòng TC03. SHA C: 4ab77e2.
BIEN02 (TC04)	chon = [(1,2,7)]; tong = 7.	PASS — Chọn đúng cạnh duy nhất; đạt 4 phép kiểm	A06_Bien_HaiDinh.png; A09_KiemThu_12Ca.png, dòng TC04. SHA C: 4ab77e2.

Ca	Actual	Kết luận	Ảnh log và SHA chạy
BIEN03 (TC08)	Chọn đủ 29 cạnh (i,i+1,1000), với i từ 1 đến 29; tong = 29000	PASS — Đúng giới hạn 30 đỉnh và trọng số 1000; đạt 4 phép kiểm.	A07_Bien_30Dinh_TrongSo1000.png; A09_KiemThu_12Ca.png, dòng TC08. SHA C: 4ab77e2.
LOI01 (TC09)	Phát sinh ValueError; demo thông báo: DU LIEU KHONG HOP LE: Canh 1: trong so phai tu 0 den 1000.	PASS — Từ chối đúng trọng số âm; đạt 1 phép kiểm về loại ngoại lệ.	A08_InputSai_TrongSoAm.png; A09_KiemThu_12Ca.png, dòng TC09. SHA C: 4ab77e2.
KT09 (TC05)	chon = [(1,2,1), (1,3,1)]; tong = 2	PASS — Đúng thứ tự ưu tiên khi hòa trọng số; đạt 4 phép kiểm.	A09_KiemThu_12Ca.png, dòng TC05; đối chiếu test/runtests.py tại C. SHA C: 4ab77e2.
KT10 (TC10)	Phát sinh ValueError khi kiểm tra cạnh tự nối (1,1,5)	PASS — Từ chối đúng cạnh tự nối; đạt 1 phép kiểm về loại ngoại lệ.	A09_KiemThu_12Ca.png, dòng TC10; đối chiếu test/runtests.py tại C. SHA C: 4ab77e2.
KT11 (TC11)	Phát sinh ValueError khi kiểm tra hai cạnh (1,2,4) và (2,1,7)	PASS — Phát hiện đúng cặp đỉnh trùng trong đồ thị vô hướng; đạt 1 phép kiểm về loại ngoại lệ	A09_KiemThu_12Ca.png, dòng TC11; đối chiếu test/runtests.py tại C. SHA C: 4ab77e2.

Ca	Actual	Kết luận	Ảnh log và SHA chạy
KT12 (TC12)	Phát sinh ValueError khi kiểm tra cạnh (1,4,2) với n = 3	PASS — Từ chối đúng đỉnh ngoài phạm vi; đạt 1 phép kiểm về loại ngoại lệ.	A09_KiemThu_12Ca.png, dòng TC12; đối chiếu test/runtests.py tại C. SHA C: 4ab77e2.

Các ảnh minh chứng được lưu tại thư mục docs/anh_ket_qua. SHA C viết tắt trong bảng là 4ab77e2, tương ứng với SHA đầy đủ 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc. Bộ kiểm thử ghi nhận 12/12 ca PASS, 0 ca FAIL và 33/33 phép kiểm đạt. Với các ca dữ liệu không hợp lệ, bộ kiểm thử tự động đối chiếu loại ngoại lệ ValueError, không đối chiếu nguyên văn thông báo lỗi.

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Ca VD01 tương ứng với TC01 trong bộ kiểm thử tự động, sử dụng ví dụ 1 trên phiếu giao đề tài.

Dữ liệu đầu vào: n = 3; đỉnh bắt đầu s = 1; danh sách cạnh canh = [(1, 2, 4), (1, 3, 1), (2, 3, 2)].

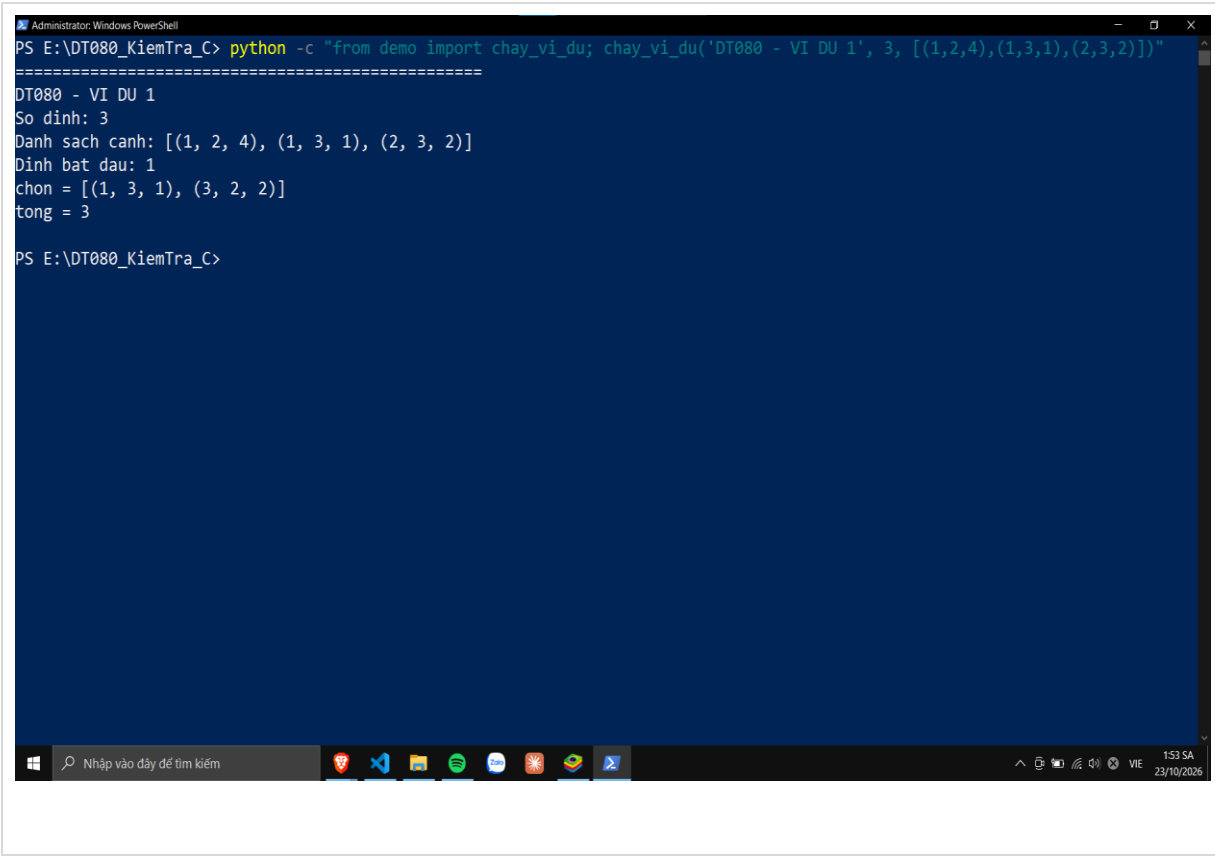
Kết quả kỳ vọng: Danh sách cạnh được chọn là [(1, 3, 1), (3, 2, 2)], tổng trọng số bằng 3. Đồ thị có ba cây bao trùm với tổng trọng số lần lượt là 4 + 1 = 5, 4 + 2 = 6 và 1 + 2 = 3. Vì vậy, tổng trọng số nhỏ nhất là 3. Khi bắt đầu tại đỉnh 1, thuật toán chọn cạnh (1, 3, 1), sau đó chọn cạnh (3, 2, 2).

Cách chạy tạo ảnh minh chứng: Sử dụng lệnh python -c để khai báo n = 3, canh = [(1,2,4), (1,3,1), (2,3,2)] và gọi trực tiếp hàm xử lý với đỉnh bắt đầu s = 1.

Kết quả thực tế: Chương trình hiển thị chon = [(1, 3, 1), (3, 2, 2)] và tong = 3, trùng với kết quả kỳ vọng.

Minh chứng: docs/anh_ket_qua/A01_ViDu1.png.

Phiên bản mã chạy — SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.



Hình 4.1. Kết quả chạy ví dụ 1 — VD01 (TC01)

Kết quả thực tế cho thấy hai cạnh được chọn nối đủ ba đỉnh, không tạo chu trình và có tổng trọng số bằng 3, đúng với giá trị nhỏ nhất đã tính trước. Cạnh (1, 2, 4) không được chọn vì sau khi nhận hai cạnh có trọng số 1 và 2, cả ba đỉnh đã thuộc cây; thêm cạnh này sẽ tạo chu trình. Diễn tiến lựa chọn cạnh và bỏ qua ứng viên dẫn tới đỉnh đã chọn được trình bày tại Bảng 3.3. Ca VD01 đạt yêu cầu; TC01 trong bộ kiểm thử tự động cũng đạt cả bốn phép kiểm.

4.3.2. Ví dụ 2 — VD02

Ca VD02 tương ứng với TC02 trong bộ kiểm thử tự động, sử dụng ví dụ 2 trên phiếu giao đề tài.

Dữ liệu đầu vào: n = 4; đỉnh bắt đầu s = 1; danh sách cạnh canh = [(1, 2, 1), (3, 4, 2)].

Kết quả kỳ vọng: Hàm prim trả về None; chương trình demo hiển thị KHONG LIEN THONG. Đồ thị gồm hai thành phần liên thông {1, 2} và {3, 4}, không có cạnh nối giữa hai thành phần nên không tồn tại cây bao trùm toàn bộ bốn đỉnh.

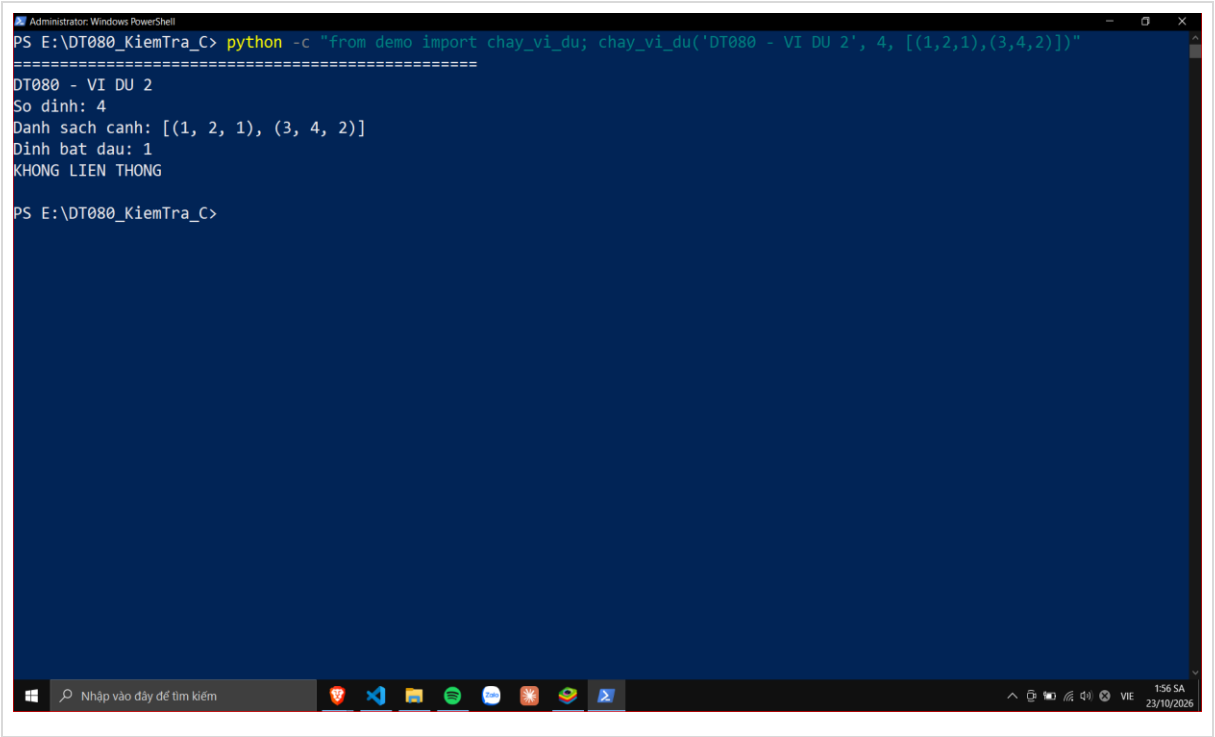
Cách chạy tạo ảnh minh chứng: Sử dụng lệnh python -c để khai báo n = 4, canh =

[(1,2,1), (3,4,2)] và gọi trực tiếp hàm xử lý với đỉnh bắt đầu s = 1.

Kết quả thực tế: Chương trình hiển thị KHONG LIEN THONG, đúng với kết quả kỳ vọng. Bộ kiểm thử TC02 xác nhận hàm prim trả về None.

Minh chứng: docs/anh_ket_qua/A02_ViDu2.png.

Phiên bản mã chạy — SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.



Hình 4.2. Kết quả chạy ví dụ 2 — VD02 (TC02)

Bắt đầu từ đỉnh 1, thuật toán chỉ nhận được hai đỉnh 1 và 2. Sau khi xử lý cạnh (1, 2, 1), heap rỗng nhưng số đỉnh đã nhận chỉ bằng 2, nhỏ hơn n = 4, nên hàm trả về None và chương trình thông báo KHONG LIEN THONG.

Khác với VD01, ca VD02 kết thúc ở nhánh phát hiện không đủ đỉnh để tạo cây bao trùm. Đây là dữ liệu đầu vào hợp lệ nhưng không tồn tại cây bao trùm, vì vậy không được xử lý như lỗi dữ liệu hoặc trả cây của riêng thành phần {1, 2} làm kết quả cho toàn đồ thị. Ca VD02 đạt yêu cầu; TC02 trong bộ kiểm thử tự động đạt phép kiểm kết quả trả về.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

Ca TK01 tương ứng với TC06 trong bộ kiểm thử tự động, được thiết kế để kiểm tra khả năng xử lý cạnh có trọng số bằng 0.

Mục đích: Kiểm tra chương trình chấp nhận trọng số 0 theo miền dữ liệu của đề tài và vẫn lựa chọn đúng các cạnh tạo thành cây bao trùm nhỏ nhất.

Dữ liệu đầu vào: n = 3; đỉnh bắt đầu s = 1; danh sách cạnh canh = [(1, 2, 0), (1, 3, 5), (2, 3, 0)].

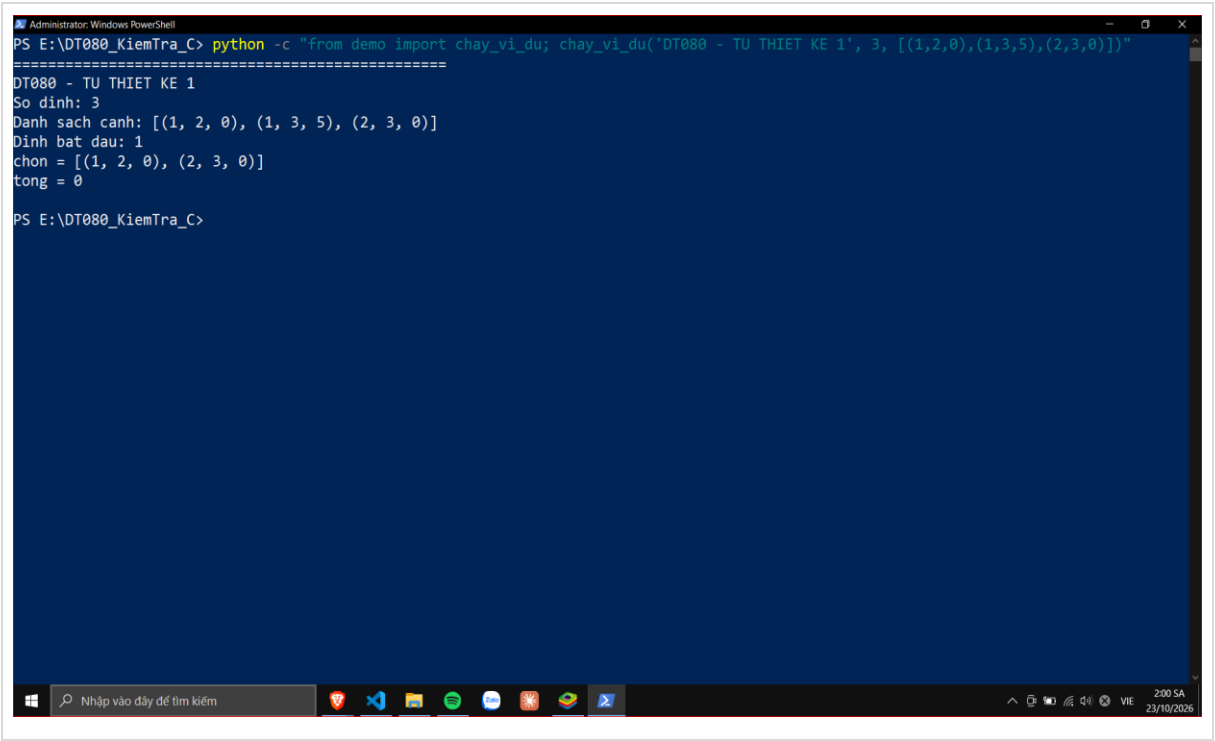
Kết quả kỳ vọng: Danh sách cạnh được chọn là [(1, 2, 0), (2, 3, 0)], tổng trọng số bằng 0. Hai cạnh trọng số 0 nối đủ ba đỉnh và không tạo chu trình. Do mọi trọng số đầu vào đều không âm, không thể có cây bao trùm với tổng trọng số nhỏ hơn 0; vì vậy đây là kết quả tối ưu.

Cách chạy tạo ảnh minh chứng: Sử dụng lệnh python -c để khai báo n = 3, canh = [(1,2,0), (1,3,5), (2,3,0)] và gọi trực tiếp hàm xử lý với đỉnh bắt đầu s = 1.

Kết quả thực tế: Chương trình hiển thị chon = [(1, 2, 0), (2, 3, 0)] và tong = 0, trùng với kết quả kỳ vọng.

Minh chứng: docs/anh_ket_qua/A03_TuThietKe_TrongSo0.png.

Phiên bản mã chạy — SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.



Hình 4.3. Kết quả ca TK01 (TC06) — Đồ thị có cạnh trọng số 0

Từ đỉnh 1, thuật toán chọn cạnh (1, 2, 0) trước cạnh (1, 3, 5). Sau khi nhận đỉnh 2, cạnh (2, 3, 0) được đưa vào heap và được chọn tiếp, tạo thành cây bao trùm có tổng trọng số bằng 0.

Ca này bổ sung khả năng phát hiện lỗi loại bỏ nhầm cạnh trọng số 0 hoặc coi trọng số 0 là không có cạnh. Kết quả thực tế cho thấy chương trình xử lý đúng trường hợp này; TC06 đạt cả bốn phép kiểm về kết quả kỳ vọng, cấu trúc cây, sự tồn tại của các cạnh trong đồ thị gốc và tính nhất quán của tổng trọng số.

4.4.2. Ca TK02

Ca TK02 tương ứng với TC07 trong bộ kiểm thử tự động, sử dụng đồ thị liên thông gồm bốn đỉnh và năm cạnh.

Mục đích: Kiểm tra việc cập nhật các cạnh ứng viên khi cây được mở rộng. Cạnh mới được đưa vào heap có thể có trọng số nhỏ hơn các cạnh đang chờ và phải được ưu tiên lựa chọn.

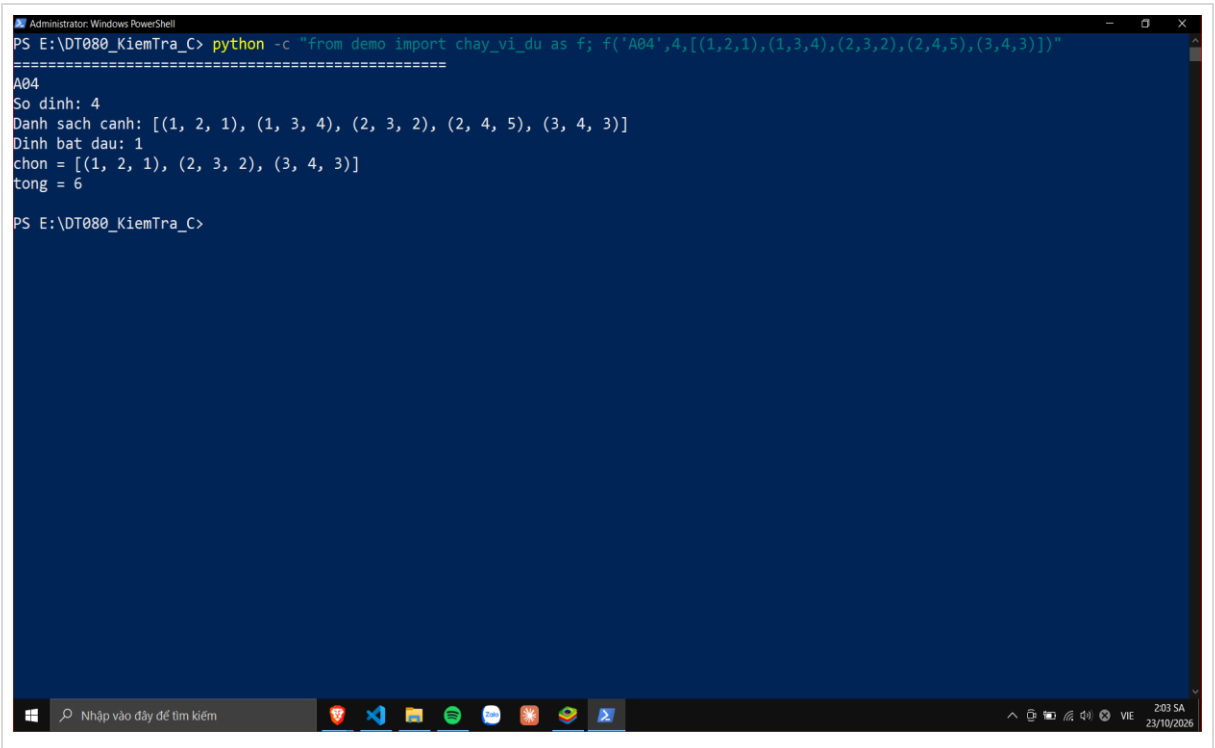
Dữ liệu đầu vào: n = 4; đỉnh bắt đầu s = 1; danh sách cạnh canh = [(1, 2, 1), (1, 3, 4), (2, 3, 2), (2, 4, 5), (3, 4, 3)].

Kết quả kỳ vọng: Danh sách cạnh được chọn là [(1, 2, 1), (2, 3, 2), (3, 4, 3)], tổng trọng số bằng 6. Một cây bao trùm bốn đỉnh phải có ba cạnh. Ba trọng số nhỏ nhất của đồ thị là 1, 2 và 3, tương ứng với các cạnh tạo thành đường đi 1–2–3–4. Do đó, tổng trọng số nhỏ nhất là 1 + 2 + 3 = 6.

Cách chạy tạo ảnh minh chứng: Sử dụng lệnh python -c để khai báo n = 4, canh = [(1,2,1), (1,3,4), (2,3,2), (2,4,5), (3,4,3)] và gọi trực tiếp hàm xử lý với đỉnh bắt đầu s = 1.
Kết quả thực tế: Chương trình hiển thị chon = [(1, 2, 1), (2, 3, 2), (3, 4, 3)] và tong = 6, trùng với kết quả kỳ vọng.

Minh chứng: docs/anh_ket_qua/A04_TuThietKe_BonDinh.png.

Phiên bản mã chạy — SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.



Hình 4.4. Kết quả ca TK02 (TC07) — Đồ thị liên thông bốn đỉnh

Sau khi chọn cạnh (1, 2, 1), thuật toán bổ sung cạnh (2, 3, 2) và (2, 4, 5) vào heap. Cạnh (2, 3, 2) được chọn trước cạnh (1, 3, 4) đang chờ. Khi nhận đỉnh 3, cạnh (3, 4, 3) tiếp tục được bổ sung và được chọn trước cạnh (2, 4, 5), đưa tổng trọng số của cây lên 6.

So với hai ví dụ trên phiếu và ca TK01, ca này sử dụng đồ thị liên thông lớn hơn, có nhiều cạnh ứng viên cạnh tranh qua các bước mở rộng cây. Vì vậy, ca kiểm tra giúp phát hiện lỗi không bổ sung cạnh từ đỉnh vừa nhận hoặc lấy cạnh không đúng thứ tự ưu tiên. Kết quả thực tế đúng kỳ vọng; TC07 đạt cả bốn phép kiểm trong bộ kiểm thử tự động.

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01

Ca BIEN01 tương ứng với TC03 trong bộ kiểm thử tự động, kiểm tra giới hạn nhỏ nhất của số đỉnh theo phiếu giao đề tài.

Mục đích: Kiểm tra chương trình xử lý đúng đồ thị chỉ có một đỉnh và không có cạnh, không nhầm trường hợp này với đồ thị không liên thông.

Dữ liệu đầu vào: n = 1; đỉnh bắt đầu s = 1; danh sách cạnh canh = [].

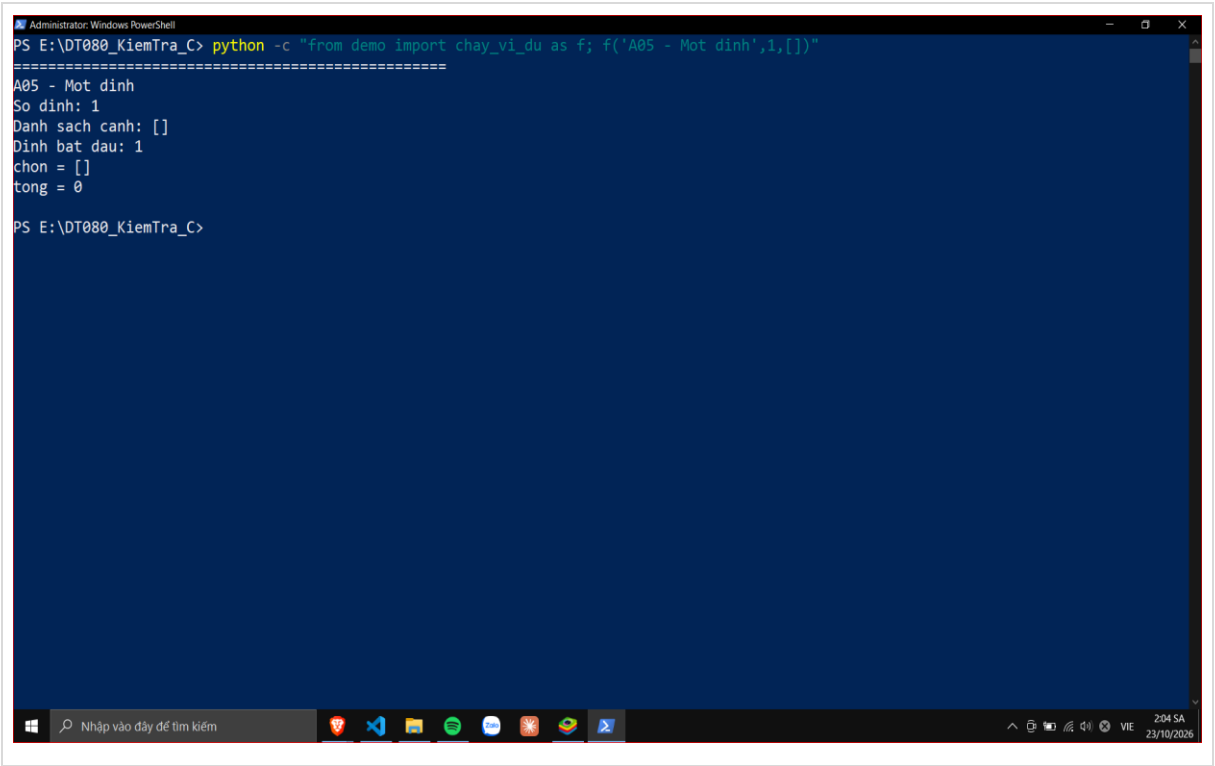
Kết quả kỳ vọng: Danh sách cạnh được chọn là [], tổng trọng số bằng 0. Với n = 1, cây bao trùm có n − 1 = 0 cạnh. Đỉnh duy nhất đã bao phủ toàn bộ đồ thị nên không cần chọn thêm cạnh nào.

Cách chạy tạo ảnh minh chứng: Sử dụng lệnh python -c để khai báo n = 1, canh = [] và gọi trực tiếp hàm xử lý với đỉnh bắt đầu s = 1.

Kết quả thực tế: Chương trình hiển thị chon = [] và tong = 0, trùng với kết quả kỳ vọng.

Minh chứng: docs/anh_ket_qua/A05_Bien_MotDinh.png.

Phiên bản mã chạy — SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.



Hình 4.5. Kết quả ca BIEN01 (TC03) — Đồ thị một đỉnh, không có cạnh

Thuật toán lấy phần tử nguồn (0, 1, 0) khỏi heap và đánh dấu đỉnh 1 đã được nhận. Vì phần tử nguồn không đại diện cho một cạnh, danh sách cạnh được chọn vẫn rỗng và tổng trọng số vẫn bằng 0. Đỉnh 1 không có đỉnh kề nên không có ứng viên mới được thêm vào heap.

Khi vòng lặp kết thúc, số đỉnh đã nhận bằng n = 1, do đó hàm trả về ([], 0) thay vì None. Ca kiểm tra xác nhận chương trình xử lý đúng biên dưới của số đỉnh và không thêm nhầm phần tử nguồn vào danh sách cạnh của cây. TC03 đạt cả bốn phép kiểm trong bộ kiểm thử tự động.

4.5.2. Ca BIEN02

Ca BIEN02 tương ứng với TC04 trong bộ kiểm thử tự động, kiểm tra đồ thị liên thông nhỏ nhất có cạnh.

Mục đích: Kiểm tra thuật toán chọn đúng cạnh duy nhất nối hai đỉnh và tính đúng tổng trọng số của cây bao trùm.

Dữ liệu đầu vào: n = 2; đỉnh bắt đầu s = 1; danh sách cạnh canh = [(1, 2, 7)].

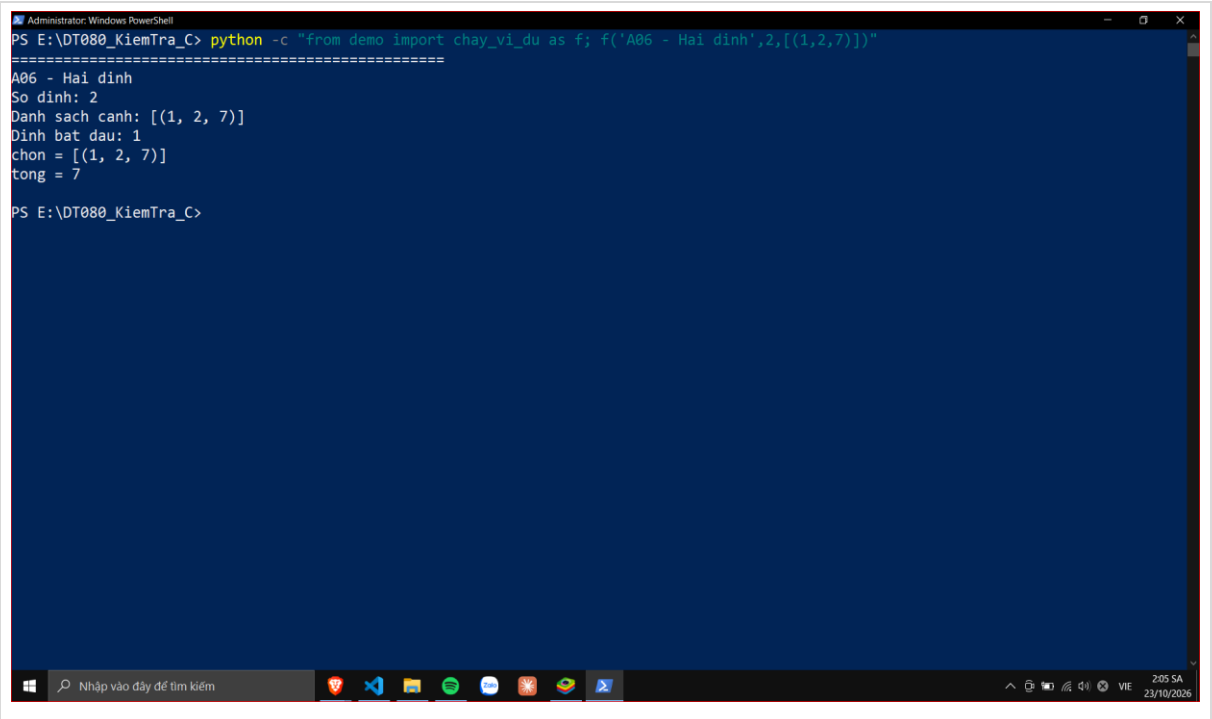
Kết quả kỳ vọng: Danh sách cạnh được chọn là [(1, 2, 7)], tổng trọng số bằng 7. Cây bao trùm hai đỉnh phải có n − 1 = 1 cạnh. Vì đồ thị chỉ có một cạnh nối hai đỉnh, cạnh này bắt buộc thuộc cây bao trùm và kết quả là duy nhất.

Cách chạy tạo ảnh minh chứng: Sử dụng lệnh python -c để khai báo n = 2, canh = [(1,2,7)] và gọi trực tiếp hàm xử lý với đỉnh bắt đầu s = 1.

Kết quả thực tế: Chương trình hiển thị chon = [(1, 2, 7)] và tong = 7, trùng với kết quả kỳ vọng.

Minh chứng: docs/anh_ket_qua/A06_Bien_HaiDinh.png.

Phiên bản mã chạy — SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.



Hình 4.6. Kết quả ca BIEN02 (TC04) — Đồ thị hai đỉnh, một cạnh

Sau khi nhận đỉnh nguồn 1, thuật toán đưa ứng viên (7, 2, 1) vào heap. Ở lượt tiếp theo, đỉnh 2 chưa được chọn nên thuật toán nhận đỉnh này, thêm cạnh (1, 2, 7) vào cây và cập nhật tổng trọng số thành 7. Cạnh ngược từ đỉnh 2 về đỉnh 1 không được đưa lại vào heap vì đỉnh 1 đã được chọn.

Kết quả có đúng một cạnh, nối đủ hai đỉnh và không tạo chu trình. So với BIEN01, ca

này kiểm tra thêm thao tác nhận một đỉnh mới qua cạnh thực, lưu cạnh vào kết quả và cộng trọng số. TC04 đạt cả bốn phép kiểm trong bộ kiểm thử tự động.

4.5.3. Ca BIEN03

Ca BIEN03 tương ứng với TC08 trong bộ kiểm thử tự động, kiểm tra đồng thời giới hạn trên của số đỉnh và trọng số cạnh theo phiếu giao đề tài.

Mục đích: Kiểm tra chương trình chấp nhận n = 30 và trọng số bằng 1000, chọn đủ số cạnh cần thiết và tính đúng tổng trọng số khi cây có nhiều cạnh.

Dữ liệu đầu vào: n = 30; đỉnh bắt đầu s = 1; danh sách cạnh được xác định bằng biểu thức Python: canh = [(i, i + 1, 1000) for i in range(1, 30)]. Đồ thị gồm 29 cạnh nối các cặp đỉnh liên tiếp từ 1–2 đến 29–30, mỗi cạnh có trọng số 1000.

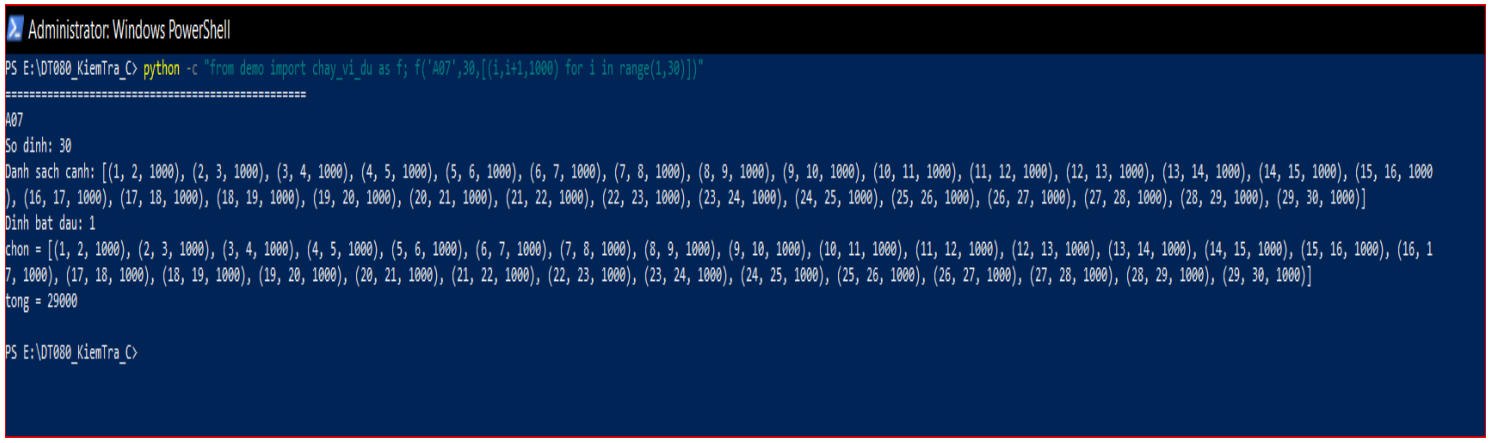
Kết quả kỳ vọng: Danh sách cạnh được chọn là [(i, i + 1, 1000) for i in range(1, 30)], theo thứ tự i tăng từ 1 đến 29. Tổng trọng số bằng 29 × 1000 = 29000. Đồ thị đầu vào là một đường đi qua đủ 30 đỉnh; bỏ bất kỳ cạnh nào cũng làm đồ thị mất liên thông. Vì vậy, cây bao trùm là duy nhất và phải chứa toàn bộ 29 cạnh.

Lệnh chạy bộ kiểm thử chứa ca TC08: python -m test.runtests

Kết quả thực tế: Chương trình chọn đủ 29 cạnh liên tiếp từ đỉnh 1 đến đỉnh 30 và trả về tổng trọng số 29000. TC08 hiển thị PASS với 4 phép kiểm đạt.

Minh chứng: docs/anh_ket_qua/A07_Bien_30Dinh_TrongSo1000.png và dòng TC08 trong ảnh docs/anh_ket_qua/A09_KiemThu_12Ca.png.

Phiên bản mã chạy — SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.



Hình 4.7. Kết quả ca BIEN03 (TC08) — Đồ thị 30 đỉnh, trọng số cạnh 1000

Bắt đầu tại đỉnh 1, thuật toán lần lượt mở rộng cây theo đường đi 1–2–3–…–30. Mặc dù các cạnh có cùng trọng số 1000, tại mỗi bước mở rộng chỉ có một cạnh nối tập đỉnh đã nhận với phần còn lại của đường đi, nên thứ tự lựa chọn được xác định duy nhất.

Kết quả gồm 29 cạnh, nối đủ 30 đỉnh và có tổng trọng số 29000, đúng với giá trị đã tính trước. Giá trị 1000 là giới hạn của từng trọng số cạnh, không phải giới hạn của tổng trọng số cây. TC08 đạt cả bốn phép kiểm, xác nhận chương trình xử lý đúng trường hợp biên trên này.

4.6. Kiểm tra input không hợp lệ — LOI01

Ca LOI01 tương ứng với TC09 trong bộ kiểm thử tự động, kiểm tra việc từ chối dữ liệu có trọng số âm.

Đối tượng kiểm tra: Hàm kiem_tra_du_lieu, được gọi khi tạo danh sách kề bằng hàm tao_danh_sach_ke, trước khi thực hiện thuật toán Prim.

Mục đích: Kiểm tra lớp xác thực đầu vào phát hiện đúng dữ liệu ngoài miền quy định và cung cấp thông báo nêu rõ nguyên nhân.

Dữ liệu đầu vào: n = 3; đỉnh bắt đầu s = 1; danh sách cạnh canh = [(1, 2, −5), (2, 3, 2)].

Lý do dữ liệu không hợp lệ: Cạnh (1, 2, −5) có trọng số âm, vi phạm yêu cầu trọng số là số nguyên trong đoạn [0, 1000] của đề tài DT080.

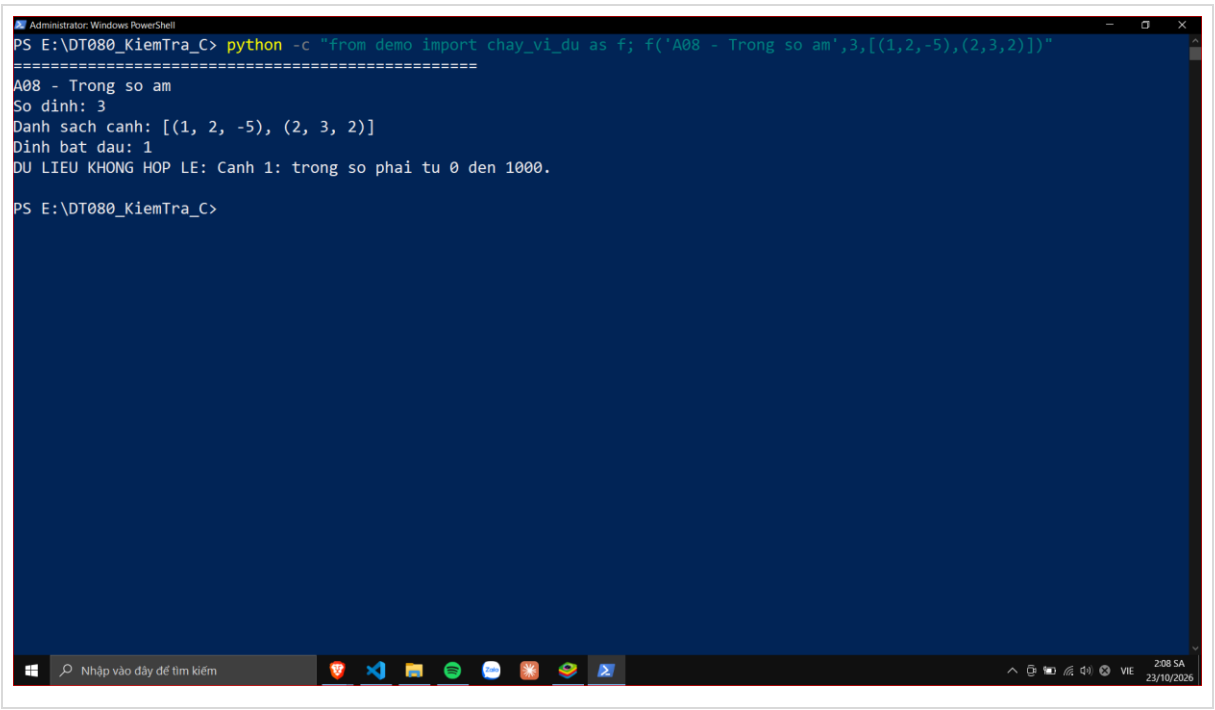
Kết quả kỳ vọng: Hàm kiểm tra dữ liệu phát sinh ngoại lệ ValueError. Chương trình demo bắt ngoại lệ và thông báo dữ liệu không hợp lệ do trọng số ngoài phạm vi cho phép; không gọi Prim cho ca này. Kỳ vọng được xác định trực tiếp từ điều kiện đầu vào trên phiếu, không dựa vào kết quả chạy thuật toán.

Cách chạy tạo ảnh minh chứng: Sử dụng lệnh python -c để khai báo n = 3, canh = [(1,2,-5), (2,3,2)] và gọi hàm xử lý để kiểm tra việc từ chối trọng số âm.

Kết quả thực tế: Chương trình hiển thị “DU LIEU KHONG HOP LE: Canh 1: trong so phai tu 0 den 1000.” Bộ kiểm thử TC09 xác nhận phát sinh đúng ngoại lệ ValueError và ghi nhận PASS.

Minh chứng: docs/anh_ket_qua/A08_InputSai_TrongSoAm.png.

Phiên bản mã chạy — SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.



Hình 4.8. Kết quả ca LOI01 (TC09) — Từ chối trọng số âm

Chương trình phát hiện trọng số −5 ngay ở cạnh đầu tiên và dừng xử lý ca dữ liệu này trước khi chạy Prim. Thông báo chỉ rõ vị trí cạnh sai và miền trọng số được chấp nhận, giúp người dùng xác định dữ liệu cần sửa.

Trường hợp này khác với VD02: VD02 có dữ liệu hợp lệ nhưng không tồn tại cây bao trùm, còn LOI01 vi phạm điều kiện đầu vào của đề tài. Việc từ chối trọng số âm ở đây xuất phát từ giới hạn của phiếu giao đề tài, không phải vì thuật toán Prim nói chung không xử lý được trọng số âm. TC09 đạt phép kiểm về loại ngoại lệ ValueError; nội dung thông báo được đối chiếu qua ảnh minh chứng, không được so sánh nguyên văn trong bộ kiểm thử tự động.

4.7. Kết quả toàn bộ kiểm thử

Bộ kiểm thử tự động được cài đặt trong tệp test/runtests.py, gồm 12 ca độc lập từ TC01 đến TC12. Các ca bao phủ hai ví dụ trên phiếu, năm trường hợp kiểm tra bắt buộc, các ca tự thiết kế, các giá trị biên và dữ liệu không hợp lệ. Những trường hợp trùng nhau giữa ví dụ và yêu cầu bắt buộc chỉ được tính là một ca độc lập.

Lệnh chạy: python -m test.runtests

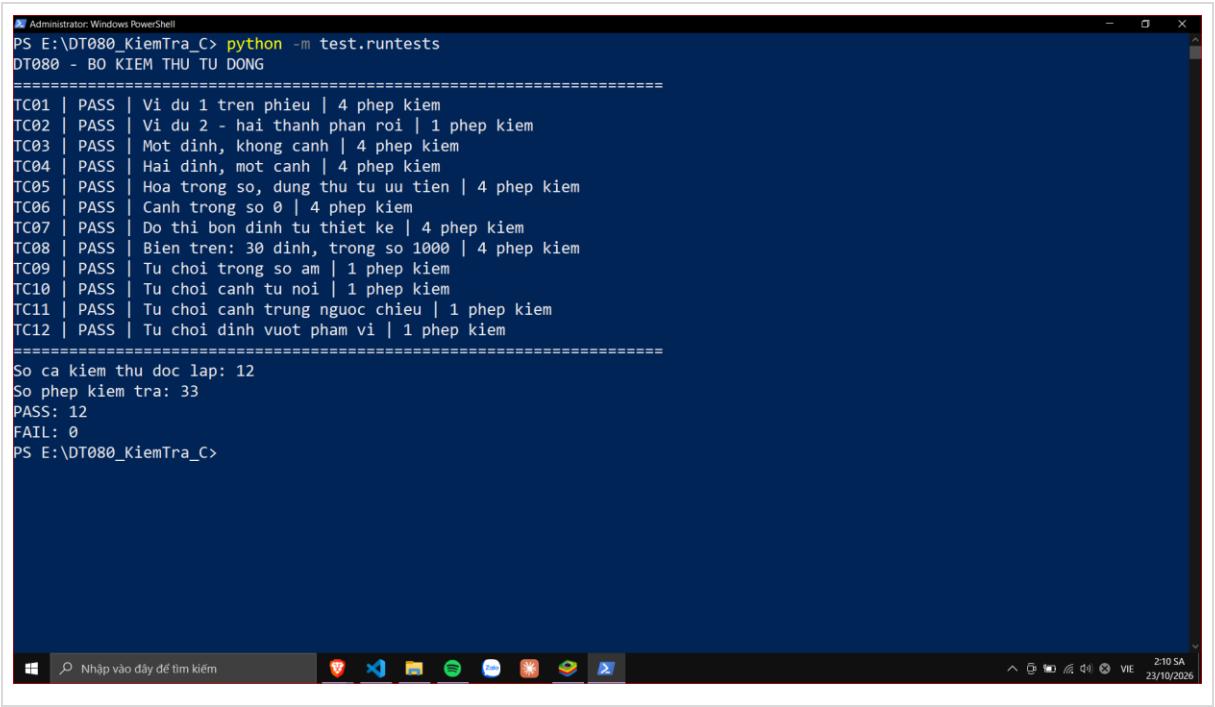
Kết quả thực tế: Tổng số ca kiểm thử là 12; số ca PASS là 12; số ca FAIL là 0. Tổng số phép kiểm là 33 và tất cả đều đạt.

Trong bộ kiểm thử, bảy ca trả về cây bao trùm gồm TC01, TC03, TC04, TC05, TC06, TC07 và TC08. Mỗi ca thực hiện bốn phép kiểm: kết quả trả về trùng với kỳ vọng đã xác định trước; các cạnh được chọn tạo thành cây; mỗi cạnh và trọng số tương ứng thuộc đồ thị đầu vào; tổng trả về bằng tổng trọng số của các cạnh được chọn.

Năm ca còn lại gồm TC02 và TC09–TC12, mỗi ca thực hiện một phép kiểm. TC02 kiểm tra kết quả trả về là None đối với đồ thị không liên thông. TC09–TC12 kiểm tra việc phát sinh ngoại lệ ValueError đối với dữ liệu vi phạm điều kiện đầu vào. Vì vậy, tổng số phép kiểm là $7 \times 4 + 5 \times 1 = 33$; đây là số phép kiểm, không phải số ca kiểm thử độc lập.

Minh chứng: docs/anh_ket_qua/A09_KiemThu_12Ca.png. Kết quả từng ca được đối chiếu tại Bảng 4.3; dữ liệu đầu vào và kết quả kỳ vọng được trình bày tại Bảng 4.2.

Phiên bản mã chạy — SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.



Hình 4.9. Kết quả bộ kiểm thử tự động — 12 ca độc lập, 33 phép kiểm

Kết quả chạy cho thấy cả 12 ca đều đạt kỳ vọng, thống nhất với các kết luận tại Bảng 4.3. Các ca trả về cây được kiểm tra thêm về cấu trúc, sự tồn tại của cạnh trong đồ thị gốc và tính nhất quán của tổng trọng số; nhờ đó, việc đánh giá không chỉ dựa vào một giá trị tổng.

Không ghi nhận ca thất bại hoặc ngoại lệ ngoài dự kiến trong lần chạy này. Tuy nhiên, 12 ca kiểm thử chỉ bao phủ các tình huống đã thiết kế, không thay thế lập luận tính đúng của thuật toán ở Chương 3 và không chứng minh rằng mọi dữ liệu hợp lệ đều đã được kiểm thử. Với các ca dữ liệu sai, kết luận PASS xác nhận đúng loại ngoại lệ ValueError, không xác nhận nguyên văn từng thông báo lỗi.

4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Chương trình khao_sat.py khảo sát thuật toán Prim tại ba quy mô Q1, Q2 và Q3, tương ứng với số đỉnh n = 5, 10 và 20. Các giá trị này đều nằm trong miền $1 \leq n \leq 30$ của đề tài. Ở mỗi quy mô, chương trình tạo hai dạng đồ thị liên thông là đồ thị đường đi và đồ thị đầy đủ, tạo thành sáu trường hợp khảo sát.

Cách tạo dữ liệu:

Đối với đồ thị đường đi, danh sách cạnh gồm các cạnh (i, i + 1, 1), với i chạy từ 1 đến n − 1. Đồ thị có m = n − 1 cạnh và mọi cạnh đều có trọng số 1.

Đối với đồ thị đầy đủ, mỗi cặp đỉnh u < v được nối bằng một cạnh. Cạnh nối hai đỉnh liên tiếp, tức v = u + 1, có trọng số 1; các cạnh còn lại có trọng số 10. Số cạnh là m = n(n − 1)/2.

Dữ liệu được tạo theo quy tắc cố định, không sử dụng số ngẫu nhiên. Trong tất cả trường hợp, đỉnh bắt đầu là s = 1.

Kết quả kỳ vọng và cách kiểm:

Mỗi cây bao trùm có n − 1 cạnh, trong khi mọi trọng số của dữ liệu khảo sát đều không nhỏ hơn 1, nên tổng trọng số không thể nhỏ hơn n − 1. Đường đi 1–2–…–n gồm toàn cạnh trọng số 1 tồn tại trong cả hai dạng đồ thị và đạt đúng tổng này. Vì vậy, tổng trọng số nhỏ nhất kỳ vọng là n − 1, lần lượt bằng 4, 9 và 19 tại ba quy mô.

Chương trình kiểm tra các cạnh trả về tạo thành cây, từng cạnh cùng trọng số thuộc đồ thị gốc, tổng trả về bằng tổng trọng số các cạnh đã chọn và bằng giá trị kỳ vọng n − 1.

Các phép toán được đếm:

– day_canh: số lần đẩy một ứng viên cạnh vào heap; không tính phần tử nguồn (0, s, 0).

– lay_canh: số lần lấy một ứng viên cạnh khỏi heap, bao gồm cả ứng viên bị bỏ qua; không tính phần tử nguồn.

– lay_heap: tổng số lần lấy phần tử khỏi heap, bao gồm phần tử nguồn.

– nhan_dinh: số đỉnh được nhận vào cây lần đầu, bao gồm đỉnh bắt đầu.

– bo_qua: số phần tử lấy khỏi heap có đỉnh đích đã được chọn nên bị bỏ qua.

Các bộ đếm được khởi tạo lại cho từng lần gọi Prim. Bản cài đặt tiếp tục xử lý đến khi heap rỗng, kể cả khi đã nhận đủ các đỉnh; do đó, các ứng viên còn lại vẫn được lấy ra và được tính vào số thao tác.

Điều kiện thực hiện:

Khảo sát được chạy bằng Python 3.14.7 trong Windows PowerShell, sử dụng mã tại SHA C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc. Chế độ in truy vết được tắt. Các bộ đếm chỉ ghi nhận thao tác trong thuật toán Prim, không tính việc tạo dữ liệu, kiểm tra đầu vào, kiểm tra kết quả hoặc in bảng.

Khảo sát sử dụng số thao tác thay cho thời gian thực thi, vì vậy không thực hiện đo thời gian hay lấy trung bình thời gian qua nhiều lần chạy. Bộ nhớ được nhận xét dựa trên các cấu trúc dữ liệu của bản cài đặt, không đo trực tiếp dung lượng RAM.

Lệnh chạy: python khao_sat.py

4.8.2. Kết quả và giới hạn nhận xét

Kết quả khảo sát ghi nhận cả sáu trường hợp đều đạt các điều kiện kiểm tra đã nêu tại mục 4.8.1. Với n lần lượt bằng 5, 10 và 20, tổng trọng số cây bao trùm nhỏ nhất tương ứng là 4, 9 và 19 ở cả hai dạng đồ thị.

Để trình bày gọn trong Bảng 4.4, các bộ đếm được ký hiệu: D là day_canh, L là lay_canh, H là lay_heap, N là nhan_dinh và B là bo_qua. Đơn vị của các bộ đếm là số lần thực hiện thao tác tương ứng.

Bảng 4.4 Khảo sát Q1 Q2 Q3

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	n = 5, s = 1. Đường đi: m = 4. Đầy đủ: m = 10. Cạnh liên tiếp có trọng số 1; cạnh khác có trọng số 10. Dữ liệu tạo theo mục 4.8.1.	Quy mô nhỏ nhất trong khảo sát. So sánh hai dạng đồ thị cùng 5 đỉnh nhưng khác số cạnh.	Đường đi: D = 4; L = 4; H = 5; N = 5; B = 0. Đầy đủ: D = 10; L = 10; H = 11; N = 5; B = 6. Cả hai: tổng MST = 4; PASS.	Bộ nhớ phân tích O(n + m), không đo RAM thực tế. Đồ thị đầy đủ có nhiều ứng viên cạnh hơn và phát sinh 6 lần bỏ qua.
Q2	n = 10, s = 1. Đường đi: m = 9. Đầy đủ: m = 45. Quy tắc trọng số giữ nguyên như Q1.	Tăng số đỉnh từ 5 lên 10; số cạnh tăng theo dạng đồ thị. Đỉnh bắt đầu và quy tắc trọng số không đổi.	Đường đi: D = 9; L = 9; H = 10; N = 10; B = 0. Đầy đủ: D = 45; L = 45; H = 46; N = 10; B = 36. Cả hai: tổng MST = 9; PASS.	Bộ nhớ phân tích O(n + m). Đồ thị đầy đủ lưu nhiều cạnh hơn đường đi; số lần bỏ qua tăng lên 36.
Q3	n = 20, s = 1. Đường đi: m = 19. Đầy đủ: m = 190. Quy tắc trọng số giữ nguyên như Q1.	Tăng số đỉnh lên 20, vẫn trong miền $n \leq 30$. Tiếp tục so sánh đường đi và đồ thị đầy đủ.	Đường đi: D = 19; L = 19; H = 20; N = 20; B = 0. Đầy đủ: D = 190; L = 190; H = 191; N = 20; B = 171. Cả hai: tổng MST = 19; PASS.	Bộ nhớ phân tích O(n + m). Với đồ thị đầy đủ, số cạnh lớn hơn nhiều so với đường đi và có 171 lần bỏ qua. Không suy ra dung lượng RAM cụ thể từ các bộ đếm.

Ở cả sáu trường hợp, số lần đẩy ứng viên cạnh và số lần lấy ứng viên cạnh đều bằng m. Do bản cài đặt xử lý đến khi heap rỗng, tổng số lần lấy heap bằng m + 1, trong đó một lần dành cho phần tử nguồn. Số đỉnh được nhận bằng n; số phần tử bị bỏ qua bằng $m - n + 1$.

Đối với đồ thị đường đi, $m = n - 1$ nên không có ứng viên bị bỏ qua. Số lần đẩy và lấy ứng viên cạnh tăng lần lượt từ 4 lên 9 rồi 19 khi số đỉnh tăng từ 5 lên 10 rồi 20.

Đối với đồ thị đầy đủ, $m = n(n - 1)/2$ nên số lần đẩy và lấy ứng viên tăng nhanh hơn, từ 10 lên 45 rồi 190. Sau khi một đỉnh đã được nhận qua cạnh được chọn, những ứng viên khác dẫn đến đỉnh đó trở thành ứng viên không còn hợp lệ và bị bỏ qua khi lấy khỏi heap. Điều này giải thích số lần bỏ qua lần lượt là 6, 36 và 171.

Kết quả phù hợp với phân tích rằng số cạnh ảnh hưởng trực tiếp đến số thao tác heap. Tuy nhiên, các bộ đếm chỉ ghi nhận số lần gọi thao tác, không đếm số phép so sánh bên trong heap. Vì vậy, không thể dùng riêng số liệu này để khẳng định thời gian chạy tuyến tính theo m hoặc kiểm chứng đầy đủ độ phức tạp $O(n + m \log(m + 1))$ đã phân tích ở Chương 3.

Về bộ nhớ, danh sách kề lưu $O(n + m)$ phần tử; heap có tối đa $O(m)$ ứng viên, còn các cấu trúc theo đỉnh và danh sách cạnh kết quả dùng $O(n)$ bộ nhớ. Với đồ thị đầy đủ, giới hạn tổng thể trở thành $O(n^2)$; với đường đi, tổng thể là $O(n)$. Đây là phân tích theo cấu trúc dữ liệu, không phải kết quả đo RAM thực tế.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Mã nguồn của đề tài được quản lý bằng Git và lưu trên GitHub tại kho N24DCDT095_DT080_PhanVanTinh. Các đường dẫn đối chiếu mã nguồn và bộ kiểm thử sử dụng SHA commit cụ thể để xác định chính xác phiên bản, thay vì chỉ dẫn đến nhánh main có thể tiếp tục thay đổi.

Commit C là phiên bản mã nguồn, kiểm thử và dữ liệu đã dùng để chạy chương trình và tạo các ảnh minh chứng. SHA đầy đủ của C là 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.

Commit S là phiên bản chốt cho từng lần nộp sau khi bổ sung báo cáo và ảnh minh chứng. Trong lần nộp tiến độ 1, S có SHA là 05b5cf16606cea70419c88009c0688139e38c48c. Phiên bản này đã được clone sang thư mục mới và chạy lại demo cùng bộ kiểm thử, ghi nhận hai ví dụ đúng kỳ vọng và 12/12 ca kiểm thử đạt.

SHA của phiên bản nộp cuối cùng được ghi trong tệp ThongTinNop.txt nộp rời. Việc tiếp tục chỉnh sửa báo cáo không làm thay đổi SHA C đã dùng cho các minh chứng trước đó. Nếu thay đổi mã nguồn, kiểm thử hoặc dữ liệu đầu vào sau C, cần chạy lại phần bị ảnh hưởng và cập nhật minh chứng tương ứng.

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

Thông tin	Giá trị thật phải điền
URL repo	https://github.com/PhanVanTinh0209/N24DCDT095_DT080_PhanVanTinh
Full SHA chạy ảnh C	4ab77e2cdf32c21c43180b53991dac8cebb3d0fc
SHA S ghi TXT nộp rời	Xem tệp N24DCDT095_DT080_PhanVanTinh_ThongTinNop.txt nộp kèm để xác định SHA S của lần nộp tương ứng.

Thông tin	Giá trị thật phải điền
Permalink hàm chính	https://github.com/PhanVanTinh0209/N24DCDT095_DT080_PhanVanTinh/blob/4ab77e2cdf32c21c43180b53991dac8cebb3d0fc/src/prim.py
Mã/kiểm thử giữa C và S	Trong phiên bản S của lần nộp tiến độ 1, bổ sung báo cáo và ảnh minh chứng; mã nguồn, kiểm thử và dữ liệu trong chương trình được giữ nguyên so với C. Demo và bộ kiểm thử đã được chạy lại từ bản clone S và đều đạt.
Lệnh chạy lại	Chạy hai ví dụ: python demo.py. Nhập dữ liệu mới: python demo.py --nhap. Chạy kiểm thử: python -m test.runtests. Chạy truy vết: python truy_vet.py. Chạy khảo sát: python khao_sat.py.

Permalink bộ kiểm thử tại C: https://github.com/PhanVanTinh0209/N24DCDT095_DT080_PhanVanTinh/blob/4ab77e2cdf32c21c43180b53991dac8cebb3d0fc/test/runtests.py

4.9.2. Clone sạch và tái hiện kết quả

Để kiểm tra khả năng tái hiện kết quả, kho mã được clone từ GitHub vào thư mục mới, tách biệt với thư mục phát triển. Sau khi clone, sử dụng lệnh git checkout --detach để chọn đúng commit C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc.

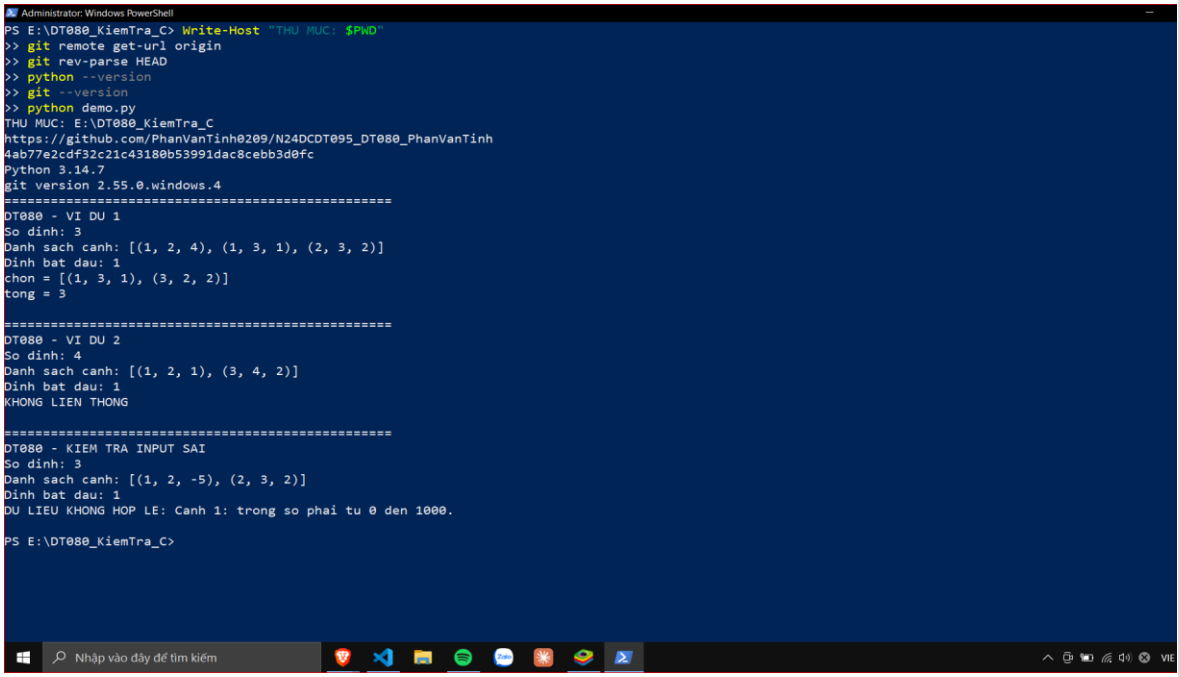
Trước khi chạy chương trình, lệnh git rev-parse HEAD được dùng để xác nhận phiên bản mã. Môi trường được kiểm tra bằng python --version và git --version, ghi nhận Python 3.14.7 và Git 2.55.0.windows.4.

Từ thư mục clone tại C, chạy python demo.py để thực hiện hai ví dụ trên phiếu và một ca dữ liệu sai. Ví dụ 1 trả về danh sách cạnh [(1, 3, 1), (3, 2, 2)] với tổng trọng số bằng 3. Ví dụ 2 hiển thị KHONG LIEN THONG. Ca trọng số âm được từ chối với thông báo trọng số phải từ 0 đến 1000.

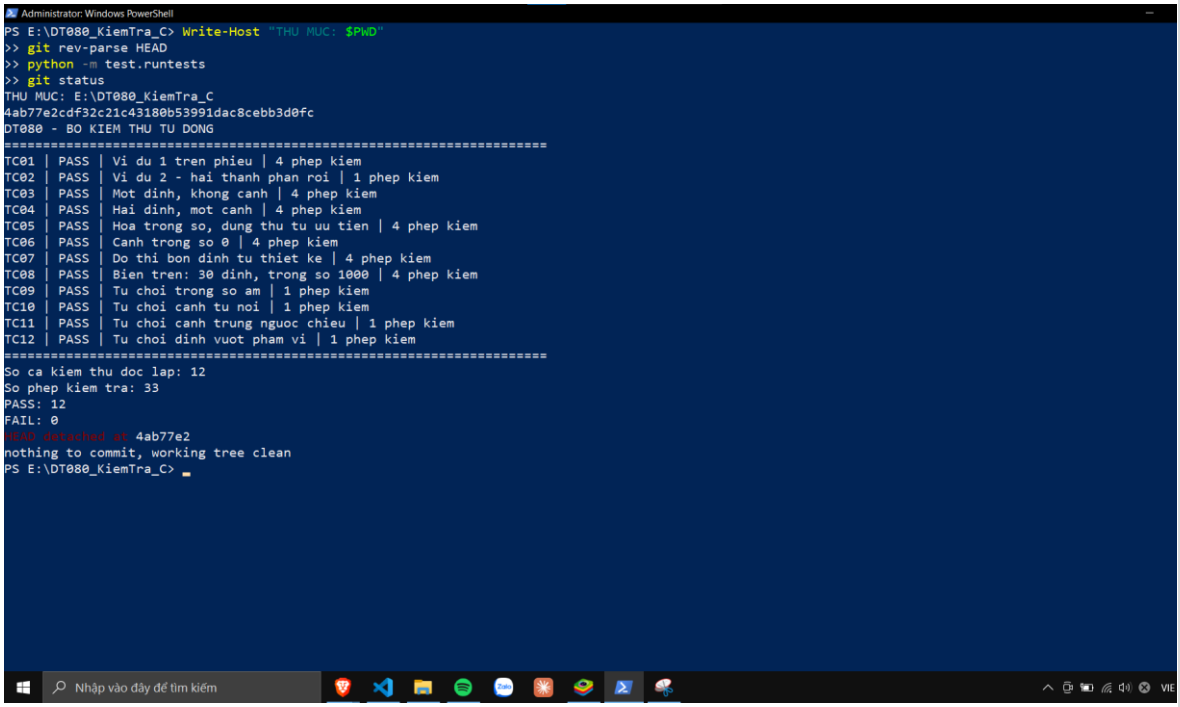
Tiếp theo, chạy python -m test.runtests. Kết quả ghi nhận 12 ca kiểm thử độc lập, 33 phép kiểm, 12 ca PASS và 0 ca FAIL. Kết quả này thống nhất với lần chạy tại thư mục

phát triển. Sau khi chạy, git status ghi nhận working tree clean, cho thấy các tệp được Git theo dõi không bị thay đổi.

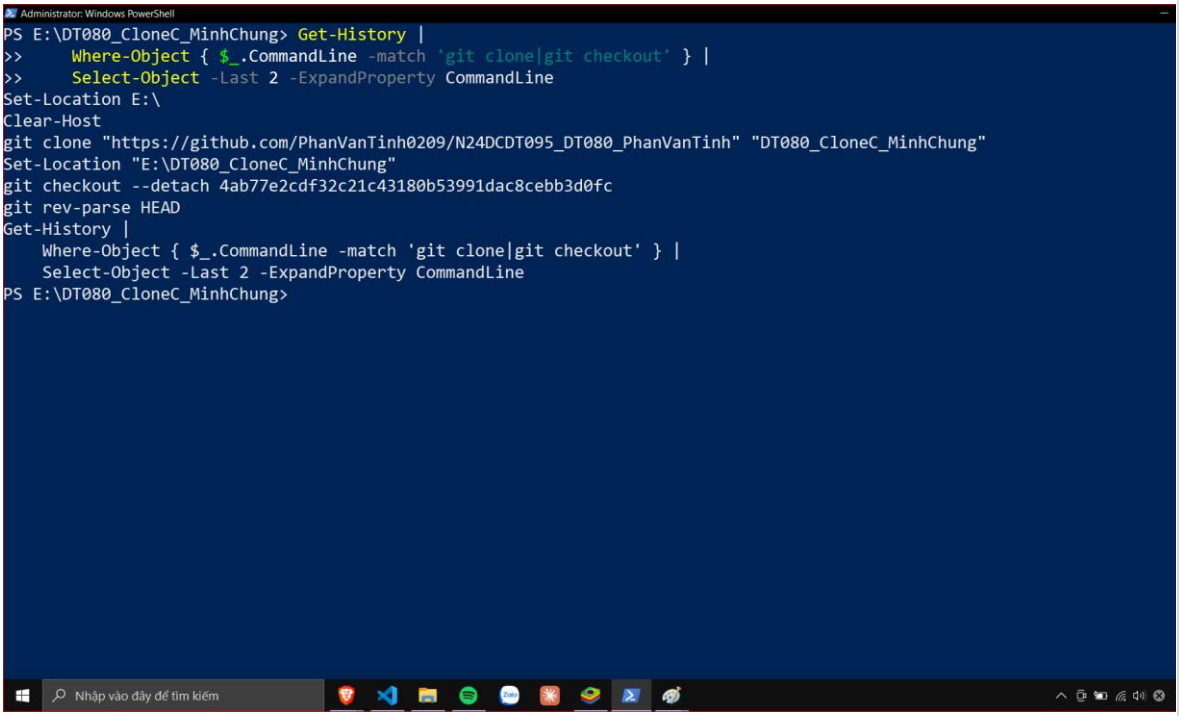
Nhóm minh chứng A10 gồm ba ảnh: A10c_LenhClone_CheckoutC.png thể hiện thao tác clone và chọn commit C; A10a_Clone_C_MoiTruong_Demo.png thể hiện phiên bản, môi trường và kết quả demo; A10b_Clone_C_KiemThu.png thể hiện kết quả bộ kiểm thử. Các ảnh được lưu trong thư mục docs/anh_ket_qua.



Hình 4.10a. Kiểm tra phiên bản, môi trường và chạy demo từ bản clone C



Hình 4.10b. Kết quả bộ kiểm thử từ bản clone C

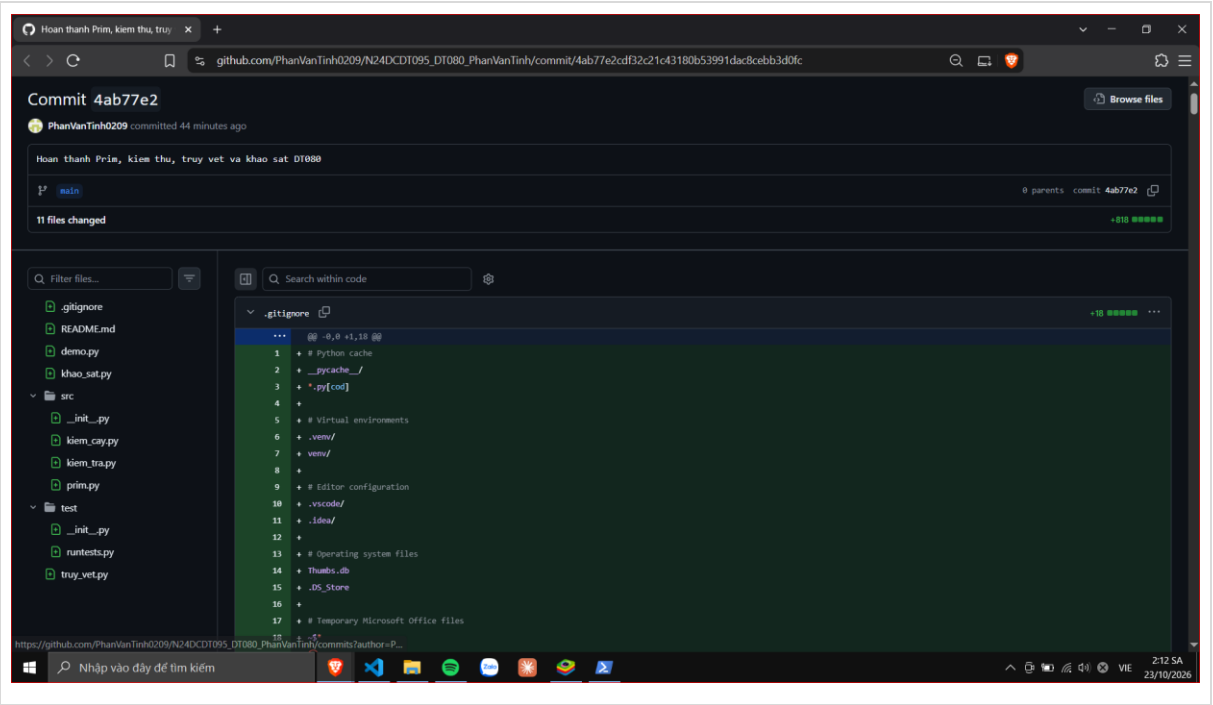


Hình 4.10c. Clone kho mã và checkout commit C

Các ảnh minh chứng cho thấy phiên bản mã được xác định bằng commit cụ thể và có thể chạy lại từ bản clone độc lập. Hai ví dụ và bộ kiểm thử cho kết quả thống nhất với các mục trước, giúp đối chiếu được giữa mã nguồn, dữ liệu kiểm thử và kết quả báo cáo.

Đối với lần nộp tiến độ 1, commit S là 05b5cf16606cea70419c88009c0688139e38c48c cũng đã được clone vào thư mục E:\DT080_KiemTra_S_Lan1 và chạy lại demo cùng bộ kiểm thử. Kết quả tiếp tục đạt 12/12 ca và 33/33 phép kiểm. Khi chốt phiên bản nộp cuối, cần thực hiện lại bước kiểm tra trên commit S tương ứng và ghi SHA cùng kết quả vào tệp ThongTinNop.txt nộp rời.

4.9.3. Bằng chứng kho GitHub thật



Hình 4.11. Kho GitHub của đề tài DT080 và commit C

Hình 4.11 là ảnh minh chứng kho GitHub N24DCDT095_DT080_PhanVanTinh thuộc tài khoản PhanVanTinh0209, dùng để lưu trữ mã nguồn của đề tài DT080. Phiên bản mã dùng cho các kết quả kiểm thử và ảnh minh chứng được xác định bởi commit C: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc, có thông điệp “Hoan thanh Prim, kiem thu, truy vet va khao sat DT080”.

Đường dẫn kho mã và các permalink tại commit C được trình bày ở mục 4.9.1. Khi nhánh main có thêm các cập nhật báo cáo hoặc ảnh, các permalink này vẫn trỏ đến đúng phiên bản mã đã được kiểm thử, giúp người đọc đối chiếu kết quả mà không nhầm với phiên bản mới hơn.

Ảnh GitHub cung cấp bằng chứng về kho mã và phiên bản được lưu trữ; khả năng chạy lại được thể hiện riêng qua nhóm ảnh A10 và kết quả tại mục 4.9.2. Tệp ảnh gốc được lưu tại docs/anh_ket_qua/A11_GitHub_CommitC.png.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đề tài đã cài đặt thuật toán Prim bằng Python để tìm cây bao trùm nhỏ nhất từ đỉnh bắt đầu s = 1 trên đồ thị vô hướng đơn, với số đỉnh từ 1 đến 30 và trọng số cạnh nguyên từ 0 đến 1000. Chương trình sử dụng danh sách kề và heap, lựa chọn ứng viên theo thứ tự ưu tiên (trọng số, đỉnh mới, đỉnh cha). Khi đồ thị liên thông, chương trình trả về danh sách n − 1 cạnh cùng tổng trọng số; khi đồ thị không liên thông, hàm trả về None và chương trình demo hiển thị KHONG LIEN THONG. Hai nhánh xử lý này được minh chứng lần lượt tại Hình 4.1 và Hình 4.2.

Các chức năng được phân tách thành các hàm kiểm tra dữ liệu, tạo danh sách kề, thêm ứng viên, thực hiện Prim và kiểm tra cấu trúc cây. Cách tổ chức mã và hợp đồng của các hàm được trình bày tại Bảng 3.1 và Bảng 3.2. Chương trình hỗ trợ chạy ví dụ có sẵn, nhập dữ liệu từ bàn phím, in truy vết và khảo sát số thao tác. Bảng 3.3 thể hiện diễn tiến thuật toán trên ví dụ 1; mục 3.5 trình bày lập luận tính đúng và Bảng 3.4 tổng hợp chi phí của từng pha trong bản cài đặt.

Về kiểm thử, đề tài đã xây dựng 12 ca độc lập, bao phủ hai ví dụ trên phiếu, năm trường hợp bắt buộc, hai ca tự thiết kế, các ca biên và dữ liệu không hợp lệ. Những trường hợp trùng nhau chỉ được tính một lần. Việc bao phủ đủ năm ca kiểm bắt buộc được đối chiếu cụ thể tại Bảng 4.1; dữ liệu đầu vào và kết quả kỳ vọng được trình bày tại Bảng 4.2. Kết quả thực tế tại Bảng 4.3 và Hình 4.9 ghi nhận 12/12 ca PASS, 0 ca FAIL và 33/33 phép kiểm đạt.

Các trường hợp đặc biệt đã có minh chứng riêng: cạnh trọng số 0 tại Hình 4.3; đồ thị một đỉnh, không có cạnh tại Hình 4.5; đồ thị hai đỉnh, một cạnh tại Hình 4.6; giới hạn 30 đỉnh và trọng số 1000 tại Hình 4.7. Hình 4.8 thể hiện việc từ chối trọng số âm theo điều kiện của phiếu giao đề tài. Không ghi nhận lỗi trong phạm vi các ca đã thực hiện; kết quả này không có nghĩa là mọi dữ liệu trong miền hợp lệ đều đã được kiểm thử.

Về khảo sát, chương trình đã chạy trên đồ thị đường đi và đồ thị đầy đủ tại ba quy mô

n = 5, 10 và 20, tạo thành sáu trường hợp hợp lệ. Cả sáu trường hợp đều trả về cây có tổng trọng số đúng kỳ vọng n − 1. Bảng 4.4 ghi nhận số lần đẩy cạnh, lấy cạnh, lấy phần tử heap, nhận đỉnh và bỏ qua ứng viên. Nhận xét tại mục 4.8.2 làm rõ ảnh hưởng của số cạnh đến số thao tác, đồng thời phân biệt số lần gọi thao tác heap với chi phí xử lý bên trong mỗi thao tác.

Về khả năng tái hiện, mã nguồn đã được lưu trên GitHub tại commit C xác định và được clone vào thư mục mới để chạy lại. Hình 4.10a thể hiện môi trường cùng kết quả demo; Hình 4.10b ghi nhận bộ kiểm thử đạt sau khi clone; Hình 4.10c minh chứng thao tác clone và chọn commit C. Hình 4.11 cung cấp bằng chứng về kho GitHub và commit dùng để đối chiếu. Phiên bản nộp tiến độ 1 cũng đã được clone và kiểm tra lại thành công như trình bày tại mục 4.9.2.

Các chức năng cốt lõi, kiểm thử, truy vết và khảo sát theo phạm vi đề tài đã được thực hiện và có bằng chứng đối chiếu. Bộ sản phẩm nộp được tổ chức theo Bảng PL.1. SHA của phiên bản nộp cuối S và kết quả chạy lại từ bản clone tương ứng được ghi trong tệp N24DCDT095_DT080_PhanVanTinh_ThongTinNop.txt nộp kèm.

5.2. Giới hạn và bài học

Giới hạn của chương trình

Chương trình được xây dựng theo miền dữ liệu của đề tài DT080: đồ thị vô hướng đơn, số đỉnh từ 1 đến 30, trọng số cạnh nguyên từ 0 đến 1000 và đỉnh bắt đầu s = 1 trong chương trình demo. Các dữ liệu có cạnh tự nối, cạnh trùng, đỉnh ngoài phạm vi hoặc trọng số ngoài miền quy định bị từ chối. Chương trình chưa hỗ trợ đồ thị có hướng, đa đồ thị hoặc tìm rừng bao trùm nhỏ nhất cho đồ thị không liên thông.

Hàm prim sử dụng danh sách kề đã được tạo và kiểm tra bởi tao_danh_sach_ke; bản thân hàm prim không kiểm tra lại toàn bộ cấu trúc danh sách kề. Vì vậy, khi sử dụng riêng hàm này, người gọi phải bảo đảm tiền điều kiện được nêu tại Bảng 3.2. Hàm kiem_cay chỉ xác nhận cấu trúc cây và tính hợp lệ của các cạnh được truyền vào, không tự xác nhận các cạnh thuộc đồ thị gốc hoặc tổng trọng số là nhỏ nhất. Các điều kiện này được kiểm tra bổ sung trong bộ kiểm thử bằng dữ liệu gốc và kết quả kỳ vọng

xác định trước.

Bộ kiểm thử gồm 12 ca độc lập đã bao phủ các yêu cầu bắt buộc và một số tình huống đặc biệt, nhưng chưa kiểm tra hết mọi đồ thị trong miền dữ liệu. Khảo sát mới thực hiện trên hai dạng đồ thị tại ba quy mô, sử dụng số thao tác thay cho thời gian chạy. Các bộ đếm không đo số phép so sánh bên trong heap hoặc dung lượng RAM thực tế, nên kết luận khảo sát chỉ giới hạn ở những đại lượng đã ghi nhận.

Bản cài đặt tiếp tục lấy các phần tử khỏi heap cho đến khi heap rỗng, kể cả sau khi đã nhận đủ các đỉnh. Cách xử lý này giúp thống kê đầy đủ các ứng viên bị bỏ qua, nhưng tạo thêm thao tác so với phương án dừng ngay khi đã nhận đủ n đỉnh trên đồ thị liên thông.

Bài học từ cài đặt và kiểm tra

Một điểm dễ gây sai khi cài Prim bằng heap là nhận lại đỉnh đã thuộc cây. Trong truy vết tại Bảng 3.3, sau khi chọn hai cạnh (1, 3, 1) và (3, 2, 2), heap vẫn còn ứng viên (4, 2, 1). Nếu tiếp tục nhận ứng viên này, danh sách kết quả sẽ có thêm cạnh (1, 2, 4), tạo chu trình và làm tổng trọng số sai. Bản cài đặt kiểm tra da_chon[u] ngay sau khi lấy phần tử khỏi heap và bỏ qua ứng viên nếu đỉnh đã được chọn. TC01 đồng thời kiểm tra kết quả kỳ vọng và cấu trúc cây, giúp phát hiện sai lệch nếu điều kiện này bị bỏ sót.

Quá trình kiểm thử cũng cho thấy cần phân biệt dữ liệu sai với dữ liệu hợp lệ nhưng không có cây bao trùm. VD02 có hai thành phần liên thông nên hàm trả về None, trong khi LOI01 có trọng số âm trái với quy định của đề tài nên bị từ chối trước khi chạy Prim. Hai cách xử lý được minh chứng tại Hình 4.2 và Hình 4.8.

Trong quá trình chuẩn bị phiên bản nộp, lệnh git add từng báo lỗi pathspec vì tệp PDF chưa có đúng tên và vị trí trong thư mục docs. Lỗi được xác định bằng cách kiểm tra đường dẫn với Test-Path, sau đó đặt tệp PDF đúng vị trí và thực hiện lại lệnh. Từ đó, việc kiểm tra tên tệp, đường dẫn và nội dung trước khi commit trở thành một bước cần thiết trong quy trình đóng gói.

Bài học chung là phải xác định kết quả kỳ vọng độc lập trước khi chạy, kiểm tra cả cấu trúc lẫn giá trị đầu ra và gắn minh chứng với phiên bản mã cụ thể. Việc phân biệt commit C dùng để chạy kiểm thử với commit S của từng lần nộp giúp đối chiếu kết quả rõ ràng và tái hiện được chương trình sau khi bổ sung báo cáo, hình ảnh.

5.3. Hướng phát triển phù hợp

Từ kết quả cài đặt và những giới hạn đã nêu, đề tài đề xuất ba hướng phát triển sau:

Thứ nhất, mở rộng bộ kiểm thử trong miền dữ liệu của đề tài. Bổ sung đồ thị có nhiều cạnh hòa trọng số, nhiều thành phần liên thông và các trường hợp sai kiểu dữ liệu. Có thể sinh thêm đồ thị với hạt giống ngẫu nhiên cố định để tái hiện dữ liệu. Với đồ thị nhỏ, sử dụng phương pháp liệt kê các tập $n - 1$ cạnh và kiểm tra cây để xác định tổng trọng số nhỏ nhất độc lập, sau đó đối chiếu với kết quả Prim. Khi có nhiều cây tối ưu, cần phân biệt việc kiểm tra tổng trọng số tối ưu với việc kiểm tra thứ tự chọn cạnh theo quy tắc của đề tài.

Thứ hai, xây dựng và so sánh một phiên bản dừng sớm. Phiên bản hiện tại xử lý đến khi heap rỗng để thống kê đầy đủ các ứng viên bị bỏ qua. Phiên bản mở rộng có thể dừng khi đã nhận đủ n đỉnh, qua đó tránh lấy tiếp các ứng viên còn lại. Hai phiên bản cần được chạy trên cùng dữ liệu, đối chiếu danh sách cạnh và tổng trọng số trước khi so sánh số thao tác. Nếu heap rỗng khi chưa nhận đủ n đỉnh, phiên bản dừng sớm vẫn phải trả về None. Kết quả khảo sát mới cần được trình bày riêng vì cách dừng làm thay đổi số lần lấy heap và số lần bỏ qua.

Thứ ba, bổ sung biểu diễn trực quan cho quá trình truy vết. Từ chức năng in truy vết đã có, có thể xây dựng hình minh họa theo từng bước, thể hiện tập đỉnh đã nhận, cạnh vừa chọn, các ứng viên đang chờ và tổng trọng số hiện tại. Phần trực quan cần lấy trạng thái từ quá trình chạy thực tế, giúp người học theo dõi thuật toán và đối chiếu với bảng truy vết mà không làm thay đổi kết quả tính toán.

Ba hướng trên mới là đề xuất, chưa được cài đặt hoặc khảo sát trong phiên bản mã dùng cho báo cáo này. Các kết quả đã thực hiện vẫn giới hạn ở chương trình Prim hiện

tại, bộ 12 ca kiểm thử, chức năng truy vết dạng văn bản và sáu trường hợp khảo sát tại ba quy mô.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Xác định bài toán và kiểm tra dữ liệu đầu vào	Đạt — Xác định miền dữ liệu, đầu ra; kiểm tra trọng số, đỉnh, cạnh tự nối và cạnh trùng.	Mục 1.3; 3.1; Bảng 4.2 và 4.3.	Áp dụng cho đồ thị vô hướng đơn, $1 \le n \le 30$, trọng số nguyên từ 0 đến 1000.
Cài đặt thuật toán Prim	Đạt — Chọn cạnh theo thứ tự ưu tiên (trọng số, đỉnh mới, đỉnh cha); trả danh sách cạnh và tổng trọng số.	Mục 3.2; 3.3; kết quả tại mục 4.3–4.5.	Sử dụng heapq của Python; đỉnh bắt đầu theo phiếu là s = 1.
Xử lý trường hợp đặc biệt	Đạt — Một đỉnh trả cây rỗng, tổng 0; chấp nhận trọng số 0; phát hiện đồ thị không liên thông và từ chối dữ liệu sai.	Các ca TC02, TC03, TC06, TC09–TC12 tại Bảng 4.2 và 4.3.	Đồ thị không liên thông không có MST toàn bộ; chương trình không trả rừng bao trùm nhỏ nhất.
Truy vết và lập luận tính đúng	Đạt — Có lần vết ví dụ, giải thích bất biến, lựa chọn cạnh và điều kiện kết thúc.	Mục 3.4; Bảng 3.3; mục 3.5.	Lần vết minh họa chi tiết trên ví dụ ba đỉnh.
Kiểm thử chương trình	Đạt — 12 ca độc lập, 12 PASS, 0 FAIL; 33/33 phép kiểm đạt tại C.	Bảng 4.2; Bảng 4.3; Hình 4.9; test/runtests.py tại C.	Bộ kiểm thử bao phủ các trường hợp đã thiết kế, không duyệt hết mọi đồ thị hợp lệ.

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Phân tích độ phức tạp và khảo sát	Đạt — Có phân tích thời gian, bộ nhớ; khảo sát hai dạng đồ thị tại n = 5, 10, 20, gồm sáu trường hợp.	Mục 3.6; mục 4.8; Bảng 4.4.	Khảo sát dựa trên số thao tác và các quy mô thuộc miền dữ liệu của đề tài.
Lưu mã trên GitHub và tái hiện từ bản clone sạch	Đạt tại C — Có kho mã, SHA xác định và kết quả chạy lại demo, kiểm thử từ bản clone C.	Bảng 4.5; mục 4.9.2; nhóm Hình 4.10 và Hình 4.11.	Kết quả tái hiện gắn với phiên bản đã kiểm tra. SHA S và kết quả chạy lại phiên bản nộp cuối được ghi trong tệp ThongTinNop.txt nộp kèm.
Tổ chức bộ sản phẩm nộp theo yêu cầu	Đã xác định bốn thành phần của bộ nộp: báo cáo Word, báo cáo PDF, ZIP mã nguồn kèm ảnh minh chứng và TXT thông tin nộp.	Bảng PL.1; Phụ lục D.	Bộ nộp chỉ được xác nhận hoàn tất sau khi kiểm tra đủ bốn tệp, thống nhất phiên bản và chạy lại từ commit S.

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf, Chương 6, mục 6.5.1 “Thuật toán Prim”, Bài 6.2, trang in 150 –153, tương ứng trang PDF 162 –165. Tài liệu học phần do giảng viên cung cấp.

[2] Hồ Nhựt Minh, Phiếu giao đề tài cá nhân học phần Cấu trúc dữ liệu và Giải thuật, INT13185, Nhóm 03, đề tài DT080 “Prim xây cây bao trùm nhỏ nhất từ một đỉnh”, giao cho sinh viên Phan Văn Tịnh, MSSV N24DCDT095. Tài liệu do giảng viên cung cấp.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

N24DCDT095_DT080_PhanVanTinh/

- README.md
- .gitignore
- demo.py
- truy_vet.py
- khao_sat.py
- src/
  - __init__.py
  - kiem_tra.py
  - prim.py
  - kiem_cay.py
- test/
  - __init__.py
  - runtests.py
- docs/
  - N24DCDT095_DT080_PhanVanTinh_BaoCao.docx
  - N24DCDT095_DT080_PhanVanTinh_BaoCao.pdf
  - anh_ket_qua/
    - A01_ViDu1.png
    - A02_ViDu2.png
    - A03_TuThietKe_TrongSo0.png
    - A04_TuThietKe_BonDinh.png
    - A05_Bien_MotDinh.png
    - A06_Bien_HaiDinh.png
    - A07_Bien_30Dinh_TrongSo1000.png
    - A08_InputSai_TrongSoAm.png
    - A09_KiemThu_12Ca.png
    - A10a_Clone_C_MoiTruong_Demo.png
    - A10b_Clone_C_KiemThu.png

A10c_LenhClone_CheckoutC.png

A11_GitHub_CommitC.png

Môi trường và cách thực hiện

Chương trình đã được kiểm tra bằng Python 3.14.7 trên Windows, sử dụng PowerShell. Mã nguồn chỉ sử dụng thư viện chuẩn của Python, không yêu cầu cài thêm gói bên ngoài. Git 2.55.0.windows.4 được sử dụng để quản lý phiên bản và clone kho mã.

Các lệnh dưới đây được chạy tại thư mục gốc của mã nguồn, nơi chứa demo.py, README.md, src và test. Sau khi giải nén ZIP, cần mở terminal tại đúng thư mục này. Không chạy trực tiếp các tệp bên trong src vì chúng được tổ chức thành một gói Python.

Danh sách trên mô tả cách tổ chức dự án có bổ sung tài liệu. Commit C chứa mã nguồn đã kiểm thử; báo cáo và ảnh trong docs được bổ sung ở các commit sau. Trong ZIP mã nguồn, ảnh minh chứng có thể được đặt tại thư mục anh_ket_qua ở gốc gói.

Chạy ví dụ 1

Lệnh: python demo.py

Kết quả tại phần “DT080 - VI DU 1”: chon = [(1, 3, 1), (3, 2, 2)] và tong = 3.

Chạy ví dụ 2

Lệnh: python demo.py

Kết quả tại phần “DT080 - VI DU 2”: KHONG LIEN THONG.

Một lần chạy demo.py hiển thị cả hai ví dụ trên phiếu và một trường hợp trọng số âm; không cần chạy hai lần nếu đã xem đủ các phần kết quả.
Nhập dữ liệu mới

Lệnh: python demo.py --nhap

Nhập số đỉnh n, số cạnh m, sau đó nhập m dòng cạnh. Mỗi dòng gồm ba số nguyên u, v, w, cách nhau bằng dấu cách. Chương trình sử dụng đỉnh bắt đầu s = 1.

Ví dụ, nhập lần lượt các dòng sau để tái hiện ví dụ 1:
3
3
1 2 4
1 3 1
2 3 2

Chạy bộ kiểm thử

Lệnh: python -m test.runtests

Kết quả đã kiểm tra: 12 ca độc lập, 33 phép kiểm, PASS: 12 và FAIL: 0. Chương trình kiểm thử trả mã thoát 0 khi toàn bộ ca đạt, mã thoát 1 nếu có ca không đạt.

Chạy truy vết

Lệnh: python truy_vet.py

Chương trình in từng bước xử lý ví dụ 1, gồm phần tử lấy khỏi heap, đỉnh được nhận, cạnh được chọn, tổng trọng số và ứng viên bị bỏ qua.

Chạy khảo sát

Lệnh: python khao_sat.py

Chương trình khảo sát đồ thị đường đi và đồ thị đầy đủ tại n = 5, 10, 20. Kết quả đã kiểm tra: sáu trường hợp đều đạt.

README.md tại commit C:
https://github.com/PhanVanTinh0209/N24DCDT095_DT080_PhanVanTinh/blob/4ab77e2cdf32c21c43180b53991dac8cebb3d0fc/README.md

PHỤ LỤC B. Danh mục bằng chứng

Các ảnh minh chứng được lưu trong thư mục docs/anh_ket_qua của dự án. Danh mục gồm 13 tệp ảnh, trong đó nhóm A10 được tách thành A10a, A10b và A10c để trình bày rõ môi trường, kết quả chạy và thao tác clone.

Ký hiệu C trong bảng tương ứng với commit mã nguồn đã dùng để chạy và đối chiếu kết quả: 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc. Các ảnh được bổ sung vào kho sau C; ký hiệu này xác định phiên bản mã được minh chứng, không phải commit chứa tệp ảnh.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01 / TC01 — Ví dụ 1; tổng trọng số bằng 3.	docs/anh_ket_qua/A01_ViDu1.png — C. Trình bày tại Hình 4.1.
A02	VD02 / TC02 — Đồ thị không liên thông.	docs/anh_ket_qua/A02_ViDu2.png — C. Trình bày tại Hình 4.2.
A03	TK01 / TC06 — Cạnh trọng số 0; tổng bằng 0.	docs/anh_ket_qua/A03_TuThietKe_TrongSo0.png — C. Trình bày tại Hình 4.3.
A04	TK02 / TC07 — Đồ thị bốn đỉnh; tổng bằng 6.	docs/anh_ket_qua/A04_TuThietKe_BonDinh.png — C. Trình bày tại Hình 4.4.
A05	BIEN01 / TC03 — Một đỉnh, không có cạnh; tổng bằng 0.	docs/anh_ket_qua/A05_Bien_MotDinh.png — C. Trình bày tại Hình 4.5.
A06	BIEN02 / TC04 — Hai đỉnh, một cạnh; tổng bằng 7.	docs/anh_ket_qua/A06_Bien_HaiDinh.png — C. Trình bày tại Hình 4.6.
A07	BIEN03 / TC08 — Đường đi 30 đỉnh, trọng số 1000; tổng bằng 29000.	docs/anh_ket_qua/A07_Bien_30Dinh_TrongSo1000.png — C. Trình bày tại Hình 4.7.
A08	LOI01 / TC09 — Từ chối trọng số âm.	docs/anh_ket_qua/A08_InputSai_TrongSoAm.png — C. Trình bày tại Hình 4.8.
A09	Toàn bộ TC01–TC12 — 12 ca PASS, 0 FAIL, 33 phép kiểm đạt.	docs/anh_ket_qua/A09_KiemThu_12Ca.png — C. Trình bày tại Hình 4.9.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A10a	Kiểm tra phiên bản, môi trường và chạy demo từ bản clone C.	docs/anh_ket_qua/A10a_Clone_C_MoiTruong_Demo.png — SHA chạy: C. Vị trí: Hình 4.10a.
A10b	Chạy bộ kiểm thử từ bản clone C.	docs/anh_ket_qua/A10b_Clone_C_KiemThu.png — SHA chạy: C. Vị trí: Hình 4.10b.
A10c	Thao tác clone kho mã và checkout commit C.	docs/anh_ket_qua/A10c_LenhClone_CheckoutC.png — SHA đối chiếu: C. Vị trí: Hình 4.10c.
A11	Kho GitHub và commit C của đề tài.	docs/anh_ket_qua/A11_GitHub_CommitC.png — C. Trình bày tại Hình 4.11.

PHỤ LỤC C. Tự kiểm trước khi nộp

Đối chiếu checklist 8 của hướng dẫn; xác nhận thông tin cá nhân/đề, >=12 ca, 11 ảnh, ba quy mô hoặc cấu hình hợp lệ, URL/SHA/permalink, clone sạch và DOCX/PDF cùng phiên bản. Đây là xác nhận sản phẩm đã chuẩn bị, không phải điểm chấm.

- ✓ MSSV và đề tài đúng bảng phân công.
- ✓ Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo.
- ✓ ≥12 ca độc lập; đủ ví dụ và ca phiếu; 13 ảnh minh chứng.
- ✓ Có truy vết, tính đúng, phân tích thời gian/bộ nhớ và ba quy mô hoặc cấu hình hợp lệ.
- ✓ URL, SHA C, permalink và quyền truy cập kiểm được; clone chạy lại.
- ✓ Đã thay toàn bộ hướng dẫn/ngoặc vuông; cập nhật trường Word và xuất PDF.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Bộ sản phẩm của đề tài sử dụng tiền tố thống nhất N24DCDT095_DT080_PhanVanTinh. Bốn tệp được nộp riêng trên Classroom, gồm báo cáo Word, báo cáo PDF, ZIP mã nguồn và TXT thông tin nộp. Tên tệp sử dụng họ tên không dấu, không có khoảng trắng và đúng mã đề DT080. Hạn nộp và lịch bảo vệ thực hiện theo thông báo của giảng viên trên Classroom.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT095_DT080_PhanVanTinh_BaoCao.docx	Báo cáo gồm năm chương, tài liệu tham khảo và phụ lục; có bảng, hình minh chứng và các danh mục được cập nhật. Giữ phần rubric dành cho giảng viên chấm.
PDF	N24DCDT095_DT080_PhanVanTinh_BaoCao.pdf	Xuất từ chính bản Word nộp; thống nhất nội dung, số trang, bảng và hình; kiểm tra khả năng đọc sau khi xuất.
Mã nguồn	N24DCDT095_DT080_PhanVanTinh_MaNguon.zip	Mã nguồn trong src, bộ kiểm thử trong test, demo.py, truy_vet.py, khao_sat.py, README.md và ảnh minh chứng. Không chứa .git, môi trường ảo, cache hoặc tệp tạm.
Thông tin	N24DCDT095_DT080_PhanVanTinh_ThongTinNop.txt	Thông tin sinh viên, URL GitHub, SHA C và S đầy đủ, permalink, môi trường, lệnh chạy, số ca và phép kiểm, kết quả chạy lại từ bản clone S.

ZIP mã nguồn chỉ chứa các thành phần của bài cá nhân, không đóng gói toàn bộ tài liệu mẫu do giảng viên cung cấp. Sau khi giải nén vào thư mục mới, chương trình phải chạy được theo hướng dẫn trong README.md. Ảnh gốc được đặt trong ZIP; các ảnh minh chứng tương ứng được trình bày trong báo cáo nên không cần đính kèm từng ảnh riêng trên Classroom.

Commit C xác định phiên bản mã nguồn, kiểm thử và dữ liệu dùng để tạo kết quả minh chứng. Commit S xác định phiên bản chốt của lần nộp sau khi bổ sung báo cáo và ảnh. Phiên bản S phải được clone vào thư mục mới để chạy lại; nếu mã nguồn, kiểm thử hoặc dữ liệu thay đổi so với C, các kết quả chịu ảnh hưởng phải được kiểm tra và cập nhật.

SHA S đầy đủ và kết quả xác minh được ghi trong tệp ThongTinNop.txt nộp rời. Không đưa tệp TXT chứa SHA của chính commit S vào commit đó, nhằm tránh việc phải cập nhật SHA lặp lại.

Các thông tin để chép vào TXT

URL GitHub thật
https://github.com/PhanVanTinh0209/N24DCDT095_DT080_PhanVanTinh

SHA C đầy đủ 4ab77e2cdf32c21c43180b53991dac8cebb3d0fc SHA S đầy đủ Xem tệp N24DCDT095_DT080_PhanVanTinh_ThongTinNop.txt nộp kèm.

Permalink hàm chính và test tại C Permalink tệp thuật toán chính tại C:

https://github.com/PhanVanTinh0209/N24DCDT095_DT080_PhanVanTinh/blob/4ab77e2cdf32c21c43180b53991dac8cebb3d0fc/src/prim.py

Permalink bộ kiểm thử tại C:

https://github.com/PhanVanTinh0209/N24DCDT095_DT080_PhanVanTinh/blob/4ab77e2cdf32c21c43180b53991dac8cebb3d0fc/test/runtests.py

Môi trường và phiên bản Windows PowerShell; Python 3.14.7; Git 2.55.0.windows.4. Chương trình sử dụng thư viện chuẩn của Python, không yêu cầu cài thêm gói bên ngoài.

Lệnh chạy hai ví dụ và bộ test Chạy hai ví dụ: python demo.py. Chạy bộ kiểm thử: python -m test.runtests.

Kết quả clone S Xem kết quả chạy lại và SHA S tương ứng trong tệp N24DCDT095_DT080_PhanVanTinh_ThongTinNop.txt nộp kèm. Số ca và phép kiểm 12 ca độc lập; 12 PASS, 0 FAIL; 33/33 phép kiểm đạt tại C.

Vào đúng bài tập trên Classroom, đính kèm đủ bốn tệp, mở kiểm tra lại rồi bấm Nộp bài. Kiểm tra trạng thái đã nộp; ảnh rời đã nằm trong báo cáo và ZIP nên không cần đính kèm từng ảnh riêng.